RTL Design Sherpa

DDR2 / LPDDR2 Family Controller Micro-Architecture Specification 0.1

June 13, 2026

Table of Contents

List of Figures

No figures in this document.

List of Tables

No tables in this document.

1 Ddr2 Lpddr2 Mas Index

Generated: 2026-06-16

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

2.1 DDR2 / LPDDR2 Family Controller Micro-Architecture Specification

Document Number: DDR2-LPDDR2-MAS-001 **Version:** 0.1 **Status:** Draft - Skeleton **Classification:** Open Source - Apache 2.0 License

---

2.2 Document Purpose

This Micro-Architecture Specification (MAS) is the implementation-level companion to the DDR2 / LPDDR2 Hardware Architecture Specification (HAS). Where the HAS answers “what is the architecture and why,” this MAS answers “what is in the RTL and how does it work.”

The MAS is the document RTL designers, verification engineers, and integrators consult when writing or reviewing SystemVerilog code, building testbenches, and stitching the controller into an SoC.

Target Audience:

- RTL designers implementing the FUBs described in this document
- Verification engineers writing per-FUB cocotb tests
- Integrators stitching the controller into an SoC
- Software engineers writing low-level memory configuration code (driver authors, bring-up engineers)

Companion Documents:

- DDR2/LPDDR2 Family Controller HAS (`../ddr2_lpddr2_has/`) — high-level architecture and design rationale

- DDR2/LPDDR2 DFI BFM documentation (in `RTLDesignSherpa-DV`) — verification-side reference

2.3 Document Scope

The MAS is the **micro-architecture** view: per-FUB inputs/outputs, internal state, FSM diagrams, datapath timing, register-level pipeline stages, and per-block timing budgets. It assumes the reader has read the HAS for context.

This document covers:

- Top-level integration wiring (`ddr2_lpddr2_ctrl`)
- Each FUB's interface signal list, internal storage, datapath flow, FSM, and timing budget
- AXI4 and DFI v2.1 wire-level protocol details specific to this controller
- APB programming interface and the full CSR register map
- Programming sequences (init, refresh, power-down, multi-rank, error handling)
- Build-time and runtime configuration reference

This document does **not** cover:

- Higher-level architectural rationale (see HAS)
- Verification plans or coverage models (see `dv/README.md`)
- Floorplan or layout guidance (project-specific)
- Bit-level CSR YAML — that lives in `regs/` and is the source of truth for CSR generation

2.4 Revision History

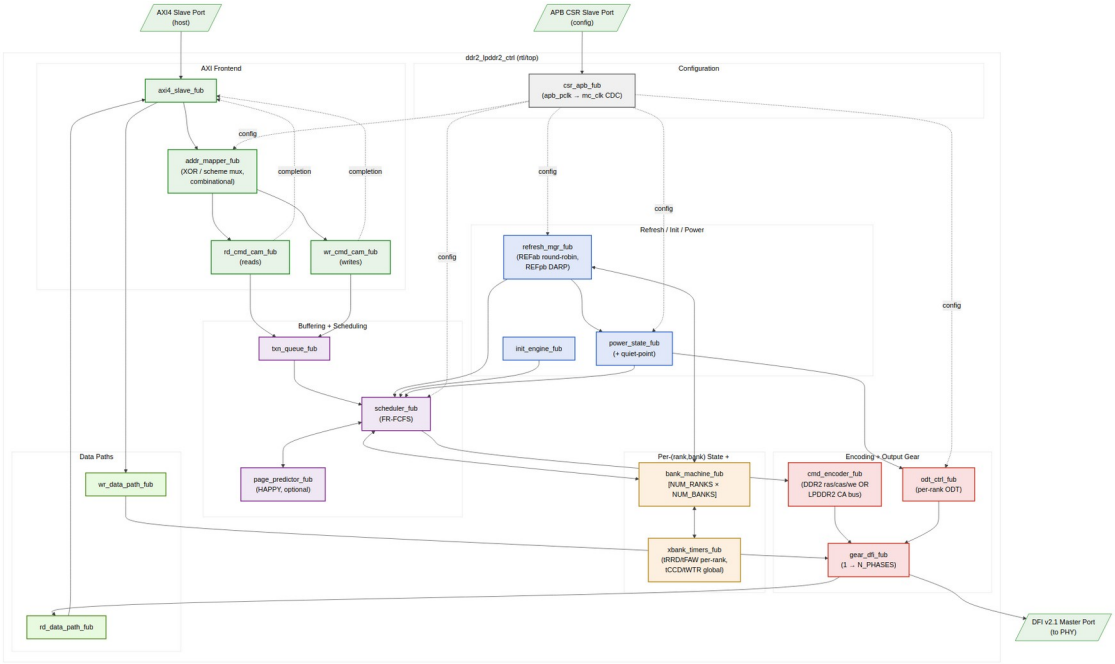
Version	Date	Author	Notes
0.1	2026-06-14	RTL Design Sherpa	Initial skeleton — chapter outline, FUB list, port list scaffold

3 Architecture and Datapath

This chapter is the implementation-level architectural orientation: the top-level FUB stitching, the read/write datapath flow, and the clocking topology. Per-FUB detail is in §2.

3.1 Top-Level FUB Topology

The controller's top is `ddr2_lpddr2_ctrl.sv` — a pure structural integration of leaf FUBs. The 18 leaf FUBs are partitioned into 7 functional groups (see HAS §2.5 for the SWAG-level group description; this section is the wire-level topology view).



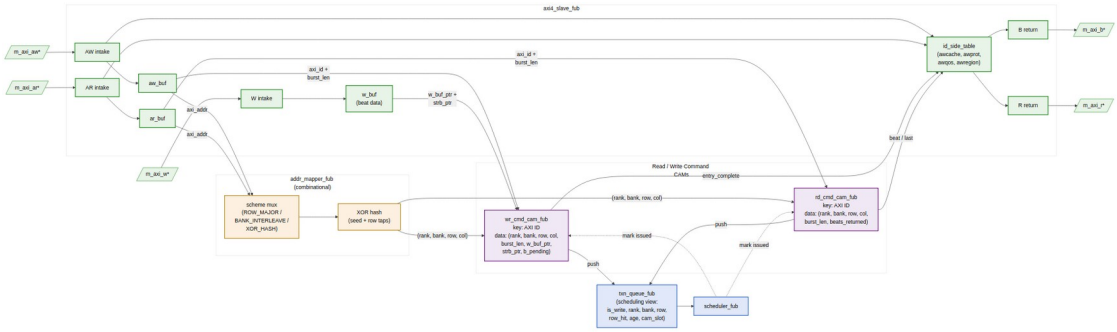
Top-Level FUB Topology

Source: 01_top_fub_topology.mmd

The salient property: there is exactly **one** stage where AXI-layer concepts (burst, ID, write strobe) cross into DRAM-layer concepts (rank, bank, row, column). That stage is `addr_mapper` → `{rd,wr}_cmd_cam`. Upstream of `addr_mapper` everything is AXI; downstream of the CAMs everything is DRAM.

3.2 Read / Write Datapath: AXI → addr-hash → CAM

The user-facing entry point is `axi4_slave_fub`. From the slave, the read and write paths share `addr_mapper` (one cycle) and then split into two separate CAMs:



AXI → addr_mapper → CAMs Datapath

Source: 02_axi_to_cam_datapath.mmd

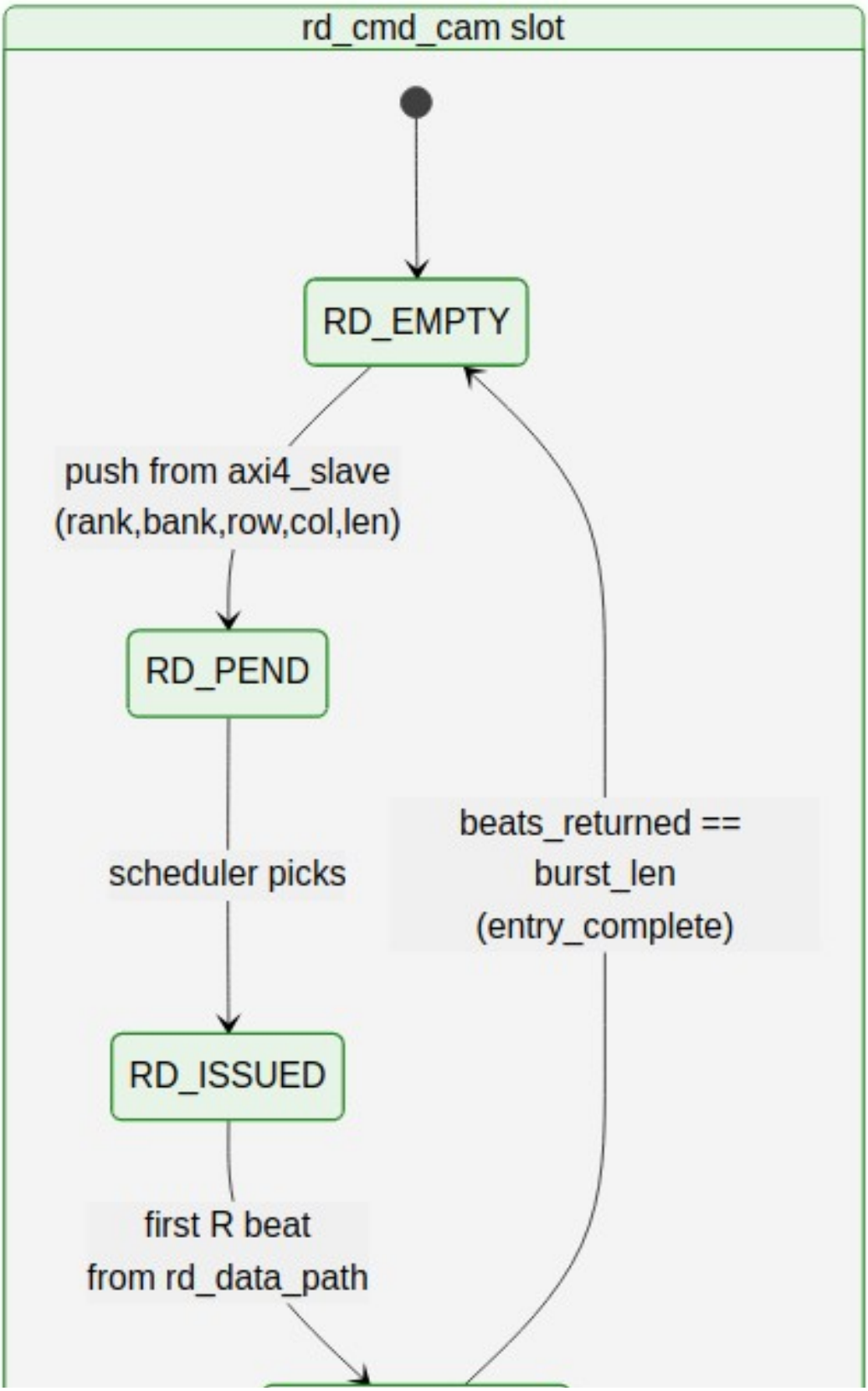
3.2.1 Why Two CAMs, Not One

The CAM split happens here, not at the scheduler, because read and write CAMs hold **different metadata** and have **different lifetimes**:

CAM	Key	Data	Cleared on
`wr_cmd_cam`	AXI ID	(rank, bank, row, col, burst_len, **w_buf_ptr**, **strb_ptr**)	B-channel response push
`rd_cmd_cam`	AXI ID	(rank, bank, row, col, burst_len, **rid_counter**, **expected_beats**)	last R beat of the burst

A single unified CAM would either carry both metadata sets (wasteful) or force an awkward "is_write" predicate on every lookup. Keeping the CAMs separate also means the read and write paths can be sized independently — the read path typically deeper because read latency is longer; the write path can be shallower because write completion is fire-and-forget.

The two CAMs have visibly different lifecycles — note in particular that the write side stays "alive" through the post-issue write window (waiting on `b_complete` from `xbank_timers`) while the read side just counts beats:


rd_cmd_cam slot
RD_EMPTY
push from axi4_slave
(rank,bank,row,col,len)
RD_PEND
scheduler picks
RD_ISSUED
first R beat
from rd_data_path
beats_returned ==
burst_len
(entry_complete)

CAM Slot Lifecycles (read vs. write)

Source: 03_cam_lifecycles.mmd

3.2.2 Why the XOR Happens Before the CAM

Address XOR / hash (`addr_mapper`) **must** happen before the CAM push because the CAM stores the post-decode (rank, bank, row, col) tuple — not the raw AXI address. Two reasons:

1. **Stable CAM key data.** The CAM is queried by the scheduler in the (rank, bank) dimension — for example, "do any pending writes target rank 0 bank 3, row R?" Storing the post-decode tuple makes this a direct comparison; storing the raw AXI address would force the scheduler to re-XOR on every match.

2. **Address-mapping scheme is per-controller, not per-burst.** The `ADDR_MAP_TUNING.scheme_or` field is global to the controller. If we stored raw addresses, a runtime scheme change (which can happen at any quiet point) would force a CAM walk to re-decode every entry. Storing the decoded tuple means a scheme change only affects **new** AXI bursts — old in-flight bursts continue with their decoded address. This matches the quiet-point semantics in HAS §6.3.

The `addr_mapper` is **combinational** — no pipeline stage between AXI intake and CAM push — so the timing budget for the XOR network is one cycle minus the AXI register-slice slack. See `ch02_blocks/03_addr_mapper.md` for the timing breakdown.

3.2.3 Why the AXI-side metadata stays in `axi4_slave_fub`

Some AXI fields (`awcache`, `awprot`, `awqos`, `awregion`, `bid`/`rid`) never need to cross into DRAM-layer state. These are held in small **side tables** inside `axi4_slave_fub`, indexed by the same AXI ID that keys the CAMs. The CAM holds the *DRAM* state; the side table holds the *AXI* state. On B/R completion, the CAM produces the completion signal and the side table produces the response metadata; the two are joined at the AXI return port.

This split keeps the CAM narrow (16 ID slots × ~36 bits in the default config) and pushes the AXI-specific clutter into the FUB that already owns the protocol.

3.3 Clocking Topology

Two external clocks; one CDC.

Clock	Frequency Range	Drives
`mc_clk`	100–500 MHz	All DRAM-layer FUBs: scheduler, bank machines, refresh, init, power, cmd_encoder, gear_dfi, odt_ctrl, data paths, and `axi4_slave`
`apb_pclk`	25–100 MHz	CSR slave only

Clock	Frequency Range	Drives

`csr_apb_fub` owns the single CDC between `apb_pclk` and `mc_clk`. CSR overrides are handed off across this CDC into a quiet-point staging register; the actual override-application happens at the next quiet point on `mc_clk`. Quiet-point detection is in `power_state_fub` (no commands in flight, no refresh in progress).

There is **no CDC** in the AXI4 datapath — `axi4_slave_fub` runs in the `mc_clk` domain. If the SoC's AXI master is on a different clock, an external clock converter is required upstream of the controller.

3.4 DFI Phase / Gear Topology

The DFI v2.1 master interface is `N_PHASES`-deep. The MC clock drives one frame of `N_PHASES` DFI cycles per MC clock cycle. The `gear_dfi_fub` rewinds scheduler-issued commands and AXI data beats into the right per-phase slot per `WRPHASE` / `RDPHASE`. The scheduler does not see the phase dimension — its issue rate is one command per MC clock; the gear absorbs all of the phase-level scheduling.

The block diagrams and FSM diagrams referenced inline above are in `assets/mermaid/` (TBD; placeholders until the per-block detail in §2 is complete).

---

4 Top-Level Port List

This chapter is the **wire-level** port list of the `ddr2_lpddr2_ctrl` top module — the exact signal names, directions, widths, and parameter dependencies that the integration script and SoC top-level use.

The implementation-level signal list is the same as the HAS overview view (HAS §2.4); this chapter restates it in the SystemVerilog-derivable form, with notes on bit-ordering, default tie-offs, and synthesis attributes where they matter.

The canonical machine-generated source is `regs/ddr2_lpddr2_ctrl_ports.svh` — what follows is the human-readable mirror.

4.1 Parameter-Dependent Widths (re-stated)

- `AW` = `AXI_ADDR_WIDTH`
- `DW` = `AXI_DATA_WIDTH`
- `SW` = `DW/8`
- `IDW` = `AXI_ID_WIDTH`
- `DFI_DW` = `DFI_DATA_WIDTH`
- `DFI_AW` = `DFI_ADDR_WIDTH`
- `NP` = `N_PHASES`
- `NR` = `NUM_RANKS`
- `NB` = `NUM_BANKS`
- `BA` = `$clog2(NUM_BANKS)`

4.2 AXI4 Slave Port List

See HAS §2.4 §1 for the table. The MAS adds nothing beyond what the HAS already published; the SystemVerilog declaration uses the same field names directly. AXI side-band signals (`awqos`, `awregion`, `arqos`, `arregion`) are observed but do not currently drive scheduler priority in v1; the wiring exists for the v2 QoS feature.

4.3 DFI v2.1 Master Port List

See HAS §2.4 §2. Additional MAS-level notes:

- The `dfi_cs_n[NR]` packed dimension is **outer-most** in SV declaration: `output [NP-1:0][NR-1:0] dfi_cs_n;`
- Per-rank `dfi_cke[NR]` is **outer-most**: `output [NR-1:0] dfi_cke;`
- The DDR2 ras/cas/we strobes are present even in LPDDR2 builds (driven all-1s, tied off by synthesis); the elaboration-time `MEMTYPE` parameter does *not* remove them from the port list, so the same SV header is usable for both flavors.

4.4 APB CSR Slave Port List

See HAS §2.4 §3. The APB clock and reset (`apb_pclk`, `apb_presetn`) are in the misc port group below; only the protocol signals are in this APB block.

4.5 Clocks, Resets, IRQs, Debug

See HAS §2.4 §4.

Additional MAS notes:

- `mc_rst_n` is asynchronous-assert, synchronous-deassert inside the controller (the deassert synchronizer lives in `power_state_fub`'s reset domain).
- `apb_presetn` is independently synchronized in `csr_apb_fub`.
- `irq_*` outputs are asserted in the `mc_clk` domain; the SoC's interrupt controller is responsible for re-synchronizing if it runs on a different clock.

4.6 SystemVerilog Header Snippet

The first 30 lines of the generated port header — the rest is per the tables above:

```
module ddr2_lpddr2_ctrl #(
    parameter string MEMTYPE          = "DDR2",
    parameter int    N_PHASES         = 4,
    parameter int    NUM_RANKS        = 1,
    parameter int    NUM_BANKS        = 8,
    parameter int    AXI_DATA_WIDTH   = 64,
    parameter int    AXI_ID_WIDTH     = 4,
    parameter int    AXI_ADDR_WIDTH   = 32,
    parameter int    ROW_WIDTH        = 14,
    parameter int    COL_WIDTH        = 10,
    parameter int    DFI_DATA_WIDTH   = 32,
    parameter int    DFI_ADDR_WIDTH   = 14,
    parameter int    WRPHASE          = 0,
    parameter int    RDPHASE          = 0,
    parameter int    TXN_QUEUE_DEPTH  = 16,
    parameter string SCHEDULER_MODE   = "OOO",
    parameter int    LOOKAHEAD_DEPTH_MAX = 4,
    parameter string PAGE_POLICY      = "HAPPY_HYBRID",
    parameter int    PAGE_PREDICTOR_TABLE_BITS = 12,
    parameter int    AGE_MAX          = 256,
    parameter int    REFRESH_DEFER_MAX = 8,
    parameter string REFPB_POLICY     = "DARP",
    parameter string ODT_RULE_MULTIRANK = "JEDEC_DDR2",
    parameter int    SIM_INIT_SCALE   = 1
) (
    // Clocks and resets
    input  logic                        mc_clk,
    input  logic                        mc_rst_n,
    input  logic                        apb_pclk,
    input  logic                        apb_presetn,
```

```
    // AXI4 slave, DFI master, APB slave, IRQs, debug — see chapter
tables.

    // ...
);
```

The full module declaration is in `regs/ddr2_lpddr2_ctrl_ports.svh` (machine-generated).

---

5 Clocks and Reset

5.1 Clock Domains

Clock	Frequency	Polarity	Domain Members
`mc_clk`	100–500 MHz	Posedge	All DRAM-layer FUBs, `axi4_slave`, data paths
`apb_pclk`	25–100 MHz	Posedge	`csr_apb_fub` only

The DFI v2.1 interface is driven on `mc_clk` (the controller side of DFI is always controller-clock; the DFI PHY runs at the DRAM clock and the gear mapping is the gear FUB's responsibility).

5.2 Reset Topology

Reset	Polarity	Type	Domain
`mc_rst_n`	Active-low	Async assert, sync deassert	`mc_clk`
`apb_presetn`	Active-low	Async assert, sync deassert	`apb_pclk`

Each reset has its own 2-flop synchronizer for the deassert edge, instantiated in the FUB that owns the domain:

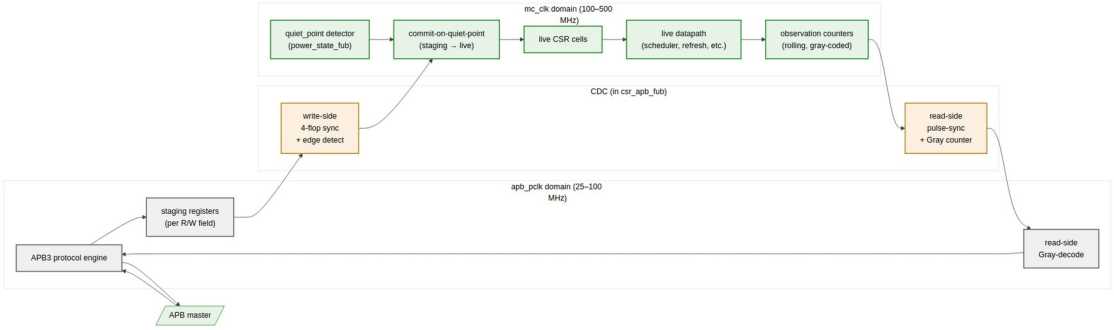
- `mc_rst_n` sync — in `power_state_fub`
- `apb_presetn` sync — in `csr_apb_fub`

Both synchronizers are clear-on-async-assert, hold-during-async-low. No reset-glitch filter is in the controller — the SoC's PMU is expected to drive a clean reset.

5.3 CDC

Exactly one CDC in the controller: APB → MC for CSR overrides and MC → APB for CSR readback. Both are handled in `csr_apb_fub`. The two directions use different mechanisms:

- **APB → MC** (write side): 4-flop synchronizer per register field, with a per-field `apb_write_strobe` edge-detect on the MC side to latch the new value into the staging register. Staging held until next quiet point.
- **MC → APB** (read side): per-counter pulse synchronizers feed Gray-coded saturating counters; APB reads sample the Gray code and decode locally. The MC-side counters never wrap synchronously between reads.



Clock Domains and CDC Topology

Source: 04_clocks_cdc.mmd

5.4 Quiet Points

A **configuration quiet point** is the only safe moment to propagate CSR overrides from staging registers into the live datapath. Quiet points are defined by:

- `txn_queue_fub` is empty *or* all pending entries are in `PENDING` state (no `ISSUED`/`COMPLETING`)
- `refresh_mgr_fub` is in IDLE (not WAIT_BANK_GNTS, DO_REFRESH, DO_ZQCS)
- `power_state_fub` is in ACTIVE (not transitioning)
- No DFI command is in flight (the encoder's per-phase pipeline has drained)

When all four hold, `power_state_fub` asserts `quiet_point`. Override-staging in `csr_apb_fub` watches this signal and commits all queued overrides in one MC-clock edge.

Software triggers quiet-point drain via `CTRL.config_apply`. The CSR slave returns `STATUS.config_settled` when commit is complete.

5.5 Reset Sequence

On power-on:

1. `mc_rst_n` and `apb_presetn` are asserted (both low). PHY drives `dfi_init_complete = 0`.
2. SoC deasserts `apb_presetn`. `csr_apb_fub` comes out of reset; CSRs are R/W-able. Software loads timing CSRs (MR0..MR3, timings, address-map scheme, refresh tuning).
3. SoC deasserts `mc_rst_n`. All DRAM-layer FUBs come out of reset. Controller idles in **POST_RESET** state.
4. Software writes `CTRL.init_start = 1`. `init_engine_fub` walks the per-memtype step table, driving `dfi_reset_n`, MR loads, ZQCAL, etc.
5. On completion, `init_engine_fub` asserts `irq_init_done` (one cycle) and `STATUS.init_done = 1`. Controller transitions to ACTIVE.
6. AXI traffic from the host is honored.

The minimum gap between `apb_presetn` and `mc_rst_n` deassertion is 16 `mc_clk` cycles (the synchronizer chain depth). Software does not need to wait — the controller stalls at POST_RESET until both resets are released.

---

6 Top-Level Integration (`ddr2_lpddr2_ctrl`)

Module: `ddr2_lpddr2_ctrl.sv` **Location:** `rtl/top/` **Category:** TOP (Structural only — no behavioral logic) **Status:** Skeleton

---

6.1 Purpose

`ddr2_lpddr2_ctrl` is the controller's top-level integration. It is **pure structural wiring** — every behavioral statement belongs in a FUB. The role of the top is:

1. Declare the external port list (see §1.2)
2. Instantiate every FUB
3. Wire FUB↔FUB interfaces
4. Fan-out per-rank / per-bank / per-phase signals where the FUB count is parametric

6.2 Instantiation Inventory

Instance Name	FUB	Count	Notes
`u_axi4_slave`	`axi4_slave_fub`	1	Single AXI4 slave port
`u_addr_mapper`	`addr_mapper_fub`	1	Combinational; no clock
`u_rd_cmd_cam`	`rd_cmd_cam_fub`	1	Read-side CAM
`u_wr_cmd_cam`	`wr_cmd_cam_fub`	1	Write-side CAM
`u_txn_queue`	`txn_queue_fub`	1	Unified pending queue
`u_scheduler`	`scheduler_fub`	1	
`u_page_predictor`	`page_predictor_fub`	0 or 1	Conditional on `PAGE_POLICY == HAPPY_HYBRID`
`u_bank_machine[r][b]`	`bank_machine_fub`	`NUM_RANKS × NUM_BANKS`	`generate` block, two-dimensional
`u_xbank_timers`	`xbank_timers_fub`	1	Internally per-rank tRRD/tFAW + global tCCD
`u_refresh_mgr`	`refresh_mgr_fub`	1	
`u_init_engine`	`init_engine_fub`	1	
`u_power_state`	`power_state_fub`	1	
`u_cmd_encoder`	`cmd_encoder_fub`	1	Parameterized by `MEMTYPE`
`u_gear_dfi`	`gear_dfi_fub`	1	
`u_odt_ctrl`	`odt_ctrl_fub`	1	
`u_wr_data_path`	`wr_data_path_fub`	1	
`u_rd_data_path`	`rd_data_path_fub`	1	

Instance Name	FUB	Count	Notes
u_csr_apb	csr_apb_fub	1	Only FUB in apb_pclk domain

6.3 Generate Blocks

The only parametric-fanout instances are the bank machines, fan-out arrays of per-rank CSRs, and per-rank ODT/CKE outputs:

```
generate
    for (genvar r = 0; r < NUM_RANKS; r++) begin : g_rank
        for (genvar b = 0; b < NUM_BANKS; b++) begin : g_bank
            bank_machine_fub #(.ROW_WIDTH(ROW_WIDTH)) u_bank_machine (
                .mc_clk          (mc_clk),
                .mc_rst_n        (mc_rst_n),
                .scheduler_if    (sched_to_bank[r][b]),
                .refresh_if      (refresh_to_bank[r][b]),
                .xbank_if        (xbank_to_bank[r][b]),
                .state_o         (bank_state[r][b]),
                .open_row_o      (bank_open_row[r][b]),
                .last_ref_age_o (bank_last_ref_age[r][b])
            );
        end
    end
endgenerate
```

The `for (genvar r)` outer loop is the per-rank dimension; the inner is the per-bank dimension. The aggregation of `bank_state[r][b]` into the scheduler input is a structural concatenation only — no behavioral aggregation in the top.

6.4 What the Top Does Not Do

- No combinational logic beyond signal renaming or pure concatenation
- No CSR field expansion (the CSR slave handles its own field encoding)
- No clock gating (clock-gate insertion is a synthesis-script concern, not RTL)
- No reset synchronization (handled per-domain in `power_state_fub` and `csr_apb_fub`)
- No `assign` statements except for tying-off optional DFI signals (DDR2 vs LPDDR2 strobe ties)

This intentional poverty of the top file makes it the obvious place for the structural-only synthesis sanity check: “every line in this file is either an instantiation or a port-mapping; if a line is anything else, it is a bug.”

7 AXI4 Slave (axi4_slave_fub)

Module: axi4_slave_fub.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** ddr2_lpddr2_ctrl
Status: Skeleton

7.1 Purpose

axi4_slave_fub is the AXI4 host-facing protocol engine. It accepts AW/W/AR transactions from the SoC, hands off the address to addr_mapper_fub for decode, and pushes the resulting transaction into the appropriate CAM (wr_cmd_cam_fub or rd_cmd_cam_fub). On B and R returns, it pulls completion signals from the CAMs and merges them with AXI-side metadata to produce protocol-compliant responses.

The FUB owns the AXI ID side table — the per-ID set of fields (awcache, awprot, awqos, awregion, bid echo, rid echo) that never need to cross into the DRAM-layer state.

7.2 External Interface (AXI side)

Per HAS §2.4 §1 — full AW/W/B/AR/R channel signal list. No additions in MAS.

7.3 Internal Buffers

Buffer	Depth	Width	Purpose
aw_buf	TXN_QUEUE_DEPTH/2 (default 8)	AW + IDW + 8(len) + 3(size) + 2(burst) + 4(cache) + 3(prot) + 4(qos) + 4(region)	Pending AW until CAM push
w_buf	TXN_QUEUE_DEPTH × max(burst_len)	DW + SW + 1(last) per beat	Write data staging
ar_buf	TXN_QUEUE_DEPTH/2 (default 8)	same AW fields	Pending AR until CAM push
b_response_fifo	small (4)	IDW + 2(resp)	Pending B responses

Buffer	Depth	Width	Purpose
r_response_fifo	small (4)	IDW + DW + 2(resp) + 1(last) per beat	Pending R responses
id_side_table	2^IDW	4(cache) + 3(prot) + 4(qos) + 4(region)	Per-ID AXI metadata, indexed by awid/arid

The buffers above are FUB-internal; they do **not** appear in the architecture-level transaction queue (txn_queue_fub). The transaction queue holds the DRAM-layer view of pending work; these buffers hold the AXI-layer view of in-flight bursts.

7.4 Data Flow

7.4.1 Write Path

1. AW intake: capture aw* fields; push to aw_buf. Compute outstanding-count and assert awready if aw_buf not full.
2. W intake: accept w* beats into w_buf. Each W beat carries wstrb and wlast. The W path does not wait for AW — W can lead AW or trail it within AXI ordering rules.
3. CAM push: when both aw_buf head and the matching w_buf head are present (or when a write-only / address-only burst's prerequisites are satisfied), push to wr_cmd_cam_fub with:
   - key = awid
   - data = (rank, bank, row, col, burst_len, w_buf_ptr, strb_ptr)
   - where (rank, bank, row, col) come from addr_mapper_fub (combinational, same cycle)
4. Scheduler issue: when scheduler picks this entry, the FUB hands w_data_path_fub the w_buf_ptr and strb_ptr for it to walk the data beats.
5. B response: when wr_cmd_cam_fub reports the entry complete (last W beat issued + tDFI write to DFI), the FUB pops the AXI ID, joins with id_side_table[id].axi_metadata, and pushes b_response_fifo.
6. B emit: standard B-channel handshake; pop b_response_fifo.

7.4.2 Read Path

1. AR intake: capture ar* fields; push to ar_buf. Assert arready if not full.
2. CAM push: pop ar_buf head; push to rd_cmd_cam_fub with:
   - key = arid

- `data = (rank, bank, row, col, burst_len, rid_counter, expected_beats)`
- where (rank, bank, row, col) come from `addr_mapper_fub`
- `rid_counter` is the AXI ID echo

3. Scheduler issue: when scheduler picks this entry, it commands the issue of RD / RDA. `rd_data_path_fub` watches for the corresponding DFI rddata beats.
4. R response: as each R beat completes, `rd_data_path_fub` strobes `rd_cmd_cam_fub` to decrement `expected_beats`; the CAM emits `r_beat_to_axi` with `rid`, beat data, `rlast` when the last beat arrives.
5. R emit: standard R-channel handshake; pop `r_response_fifo`.

7.5 Outstanding Burst Tracking

The FUB enforces:

Limit	Constraint
Total in-flight reads	≤ `rd_cmd_cam_fub` depth (default 16)
Total in-flight writes	≤ `wr_cmd_cam_fub` depth (default 16)
Per-ID in-flight reads	≤ `MAX_PER_ID_READS` parameter (default 4)
Per-ID in-flight writes	≤ `MAX_PER_ID_WRITES` parameter (default 4)

If `AXI_OOO_ACROSS_IDS == false`, the per-ID counts reduce to "one in-flight per ID" — the AXI completion is strictly in-order per ID, but the issue order at the scheduler is still OoO.

7.6 Timing Budget

Path	Budget
`awvalid` → `awready` (combinational)	≤ 0.7 of cycle
`aw_buf` push to `addr_mapper` valid	0 cycles (combinational)
`addr_mapper` valid to CAM push	1 cycle
`wr_cmd_cam` push to scheduler-visible	1 cycle
Worst path: `awvalid` → CAM-push register	2 cycles

The two-cycle AXI→CAM latency is the limit on AXI sustained issue rate; with default config it allows 1 burst per 2 cycles, well within the AXI burst rate at 200 MHz.

7.7 CSR Hooks

- `STATUS.axi_inflight_rd` (R only) — current count of in-flight read bursts
- `STATUS.axi_inflight_wr` (R only) — current count of in-flight write bursts
- `STATUS.axi_id_table_occ` (R only) — number of distinct AXI IDs currently in the side table
- `OBS_AXI_R_LATENCY_AVG`, `OBS_AXI_W_LATENCY_AVG` — driven from R/B-channel sampling in this FUB

7.8 TODO (week-of-MAS work)

- FSM diagram for the AW/W join and the AR intake (mermaid)
- B/R response-ordering corner cases (per HAS §3.1 OoO-across-IDs)
- Waveform examples (wavedrom) for back-pressure scenarios
- Detailed table of `id_side_table` field widths and reset values

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

8 Address XOR / Hash (`addr_mapper_fub`)

Module: `addr_mapper_fub.sv` **Location:** `rtl/fub/` **Category:** FUB (combinational; no clock)
Parent: `ddr2_lpddr2_ctrl` **Status:** Skeleton

8.1 Purpose

`addr_mapper_fub` decodes a flat `AXI_ADDR_WIDTH`-bit address into the DRAM-layer `(rank, bank, row, column)` tuple per the currently-active address-mapping scheme. The XOR / hash happens **here** — upstream of the read/write CAMs — so that the CAMs store the decoded tuple and not the raw AXI address.

The FUB is **pure combinational** — there is no clock, no flop, no FSM. The output is valid the same cycle as the input. The path is one of the timing-critical signatures in the design and is treated as a multi-cycle path only if synthesis cannot meet timing.

8.2 Mirror to the Python AddressMapping Class

This RTL is the mirror of the AddressMapping Python class in the DV repository (CocoTBFramework.components.dfi.address_mapping). Both produce **bit-for-bit identical** decoded outputs for the same input. The mirror is enforced by:

- Same scheme list (ROW_MAJOR, BANK_INTERLEAVE, XOR_HASH)
- Same field-bit positions per scheme
- Same XOR_HASH seed and tap matrix

The shared decode function eliminates a common class of bugs where simulation passes (because the BFM decodes one way) but RTL fails (because the controller decodes a different way).

8.3 Interface

Signal	Direction	Width	Description
axi_addr_i	input	AW	Flat AXI byte address
scheme_active_i	input	2	Runtime scheme select from ADDR_MAP_TUNING.scheme_or
xor_seed_i	input	8	Runtime seed for XOR_HASH (from CSR)
bg_field_pos_i	input	3	Bank-group field bit position (unused in DDR2/LPDDR2; reserved)
rank_o	output	$clog2(NR)	Decoded rank
bank_o	output	BA	Decoded bank
row_o	output	ROW_WIDTH	Decoded row
col_o	output	COL_WIDTH	Decoded column (byte-aligned within burst)

8.4 Scheme Decode

8.4.1 Scheme 1: ROW_MAJOR

The simplest mapping; one contiguous row per rank/bank stride. Bit layout:

```
| ... reserved ... | rank | row [ROW_WIDTH-1:0] | bank [BA-1:0] | col [COL_WIDTH-1:0] | byte_in_dq |
```

The byte-in-DQ field is the controller's column-byte offset and is consumed by wr_data_path_fub for wstrb masking; it is **not** part of the decoded col_o (which is column-word aligned).

ROW_MAJOR is best for sequential streaming workloads where a long burst stays in one row.

8.4.2 Scheme 2: BANK_INTERLEAVE

```
| ... reserved ... | rank | row [ROW_WIDTH-1:0] | col_hi [...] | bank
[BA-1:0] | col_lo [...] | byte_in_dq |
```

Bank bits are placed **between** column-low and column-high bits, so consecutive cache lines round-robin across banks. This is the recommended default for general-purpose workloads with mixed access patterns; it produces higher bank-parallelism in the scheduler.

8.4.3 Scheme 3: XOR_HASH

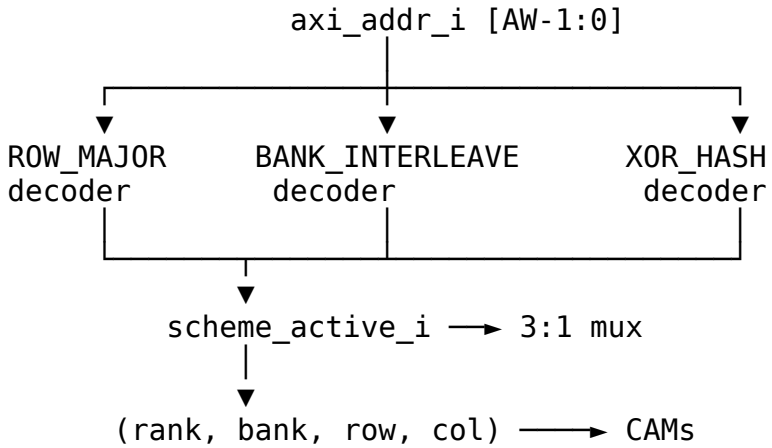
Same layout as BANK_INTERLEAVE, but the bank field and (a subset of) the column-high bits are XOR-hashed with row bits:

```
bank_hashed[i] = bank_raw[i] XOR row_raw[i] XOR row_raw[i + BA] XOR
xor_seed[i]
```

The XOR_HASH scheme defeats pathological row-conflict patterns (e.g., a power-of-two stride that lands on the same bank in BANK_INTERLEAVE). The seed `xor_seed_i` is runtime-modifiable so a bring-up engineer can change the hash without rebuilding.

8.5 Runtime Scheme Mux

The output is multiplexed across the synthesized schemes per `scheme_active_i`. Only schemes in `ADDR_MAP_SCHEMES_SYNTH` are routed to the mux; un-synthesized schemes tie off and writing them via CSR returns `pslverr`.



8.6 Timing Budget

The decoder is purely combinational. The longest path in XOR_HASH is the XOR of (a) several address bits, (b) the seed, and (c) the row bits — about 5–6 levels of XOR gates plus a 3:1 mux. At 500 MHz this is comfortably within 0.5 ns of MC clock cycle time.

Path	Estimate (gate-level)
`axi_addr_i` → `bank_o` (ROW_MAJOR)	2 levels of MUX
`axi_addr_i` → `bank_o` (BANK_INTERLEAVE)	2 levels of MUX
`axi_addr_i` → `bank_o` (XOR_HASH)	5 levels XOR + 2 MUX
`axi_addr_i` → `row_o`	1 level MUX

If XOR_HASH does not meet timing at the target frequency, the FUB can be re-fitted with a single pipeline stage (the seed-XOR can absorb a flop without changing the per-tuple invariant — the seed only changes on quiet points anyway).

8.7 Verification

The RTL decoder and the Python `AddressMapping` class are co-validated by a small cocotb test that picks 10000 random addresses, asks each side to decode, and asserts bit-equality on the (rank, bank, row, col) tuple. This is the primary regression for `addr_mapper_fub`.

8.8 TODO

- Tap matrix for XOR_HASH — the exact bit mapping
- Synthesis hint for the XOR_HASH critical path
- Edge cases for `AW < (ROW_WIDTH + COL_WIDTH + BA + clog2(NR))` (under-decoded address — should it tie-off or error?)

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

9 Read Command CAM (`rd_cmd_cam_fub`)

Module: `rd_cmd_cam_fub.sv` **Location:** `rtl/fub/` **Category:** FUB **Parent:** `ddr2_lpddr2_ctrl` **Status:** Skeleton

---

9.1 Purpose

`rd_cmd_cam_fub` is the **read-side content-addressable storage** of in-flight read commands. It holds the DRAM-layer view (rank, bank, row, col) of every read burst that has been pushed by `axi4_slave_fub` and not yet fully returned on the AXI R-channel.

The CAM is keyed by **AXI ID**, allowing the scheduler to find all reads to a given (rank, bank) without iterating the queue and allowing `rd_data_path_fub` to find the right entry to decrement when a read beat returns.

The read CAM and write CAM are intentionally separate FUBs — see §1.1 for the rationale.

9.2 Parameters

Parameter	Default	Purpose
`RD_CAM_DEPTH`	16	Number of in-flight read slots
`IDW`	`AXI_ID_WIDTH`	Key width
`BURST_LEN_WIDTH`	8	Burst-length field width

9.3 Storage Per Slot

Field	Width	Description
`valid`	1	Slot occupied
`axi_id`	`IDW`	Key — the originating AXI ID
`rank`	`$clog2(NR)`	Decoded rank
`bank`	`BA`	Decoded bank
`row`	`ROW_WIDTH`	Decoded row
`col_start`	`COL_WIDTH`	Starting column
`burst_len`	`BURST_LEN_WIDTH`	Total beats in burst
`beats_returned`	`BURST_LEN_WIDTH`	Beats already returned on R
`issued`	1	Scheduler has picked this entry
`last_beat_at`	small	Cycle of last beat return (for latency telemetry)

Total per-slot width (default config): ~46 bits.

Total storage (16 entries × 46 b): ~92 bytes — small enough to fit comfortably in distributed flops with no SRAM macro.

9.4 Interfaces

9.4.1 Push (from axi4_slave_fub)

Signal	Direction	Width	Description
push_valid	input	1	New read burst arriving
push_ready	output	1	CAM has a free slot
push_id	input	IDW	AXI ID
push_rank	input	$clog2(NR)	from addr_mapper
push_bank	input	BA	
push_row	input	ROW_WIDTH	
push_col	input	COL_WIDTH	
push_len	input	BURST_LEN_WIDTH	

9.4.2 Scheduler Query

The scheduler queries the CAM to: - Find all unissued entries matching a target (rank, bank) — for next-issue selection - Find row-hit candidates (entries whose row == open_row[rank][bank])

Signal	Direction	Description
q_rank_i	input	Query rank
q_bank_i	input	Query bank
q_row_i	input	Query row (for row-hit check)
match_unissued_o	output	One-hot vector of slots matching (rank, bank) and not issued
match_rowhit_o	output	One-hot vector of slots matching (rank, bank, row) and not issued
match_data_o	output	The selected slot's payload (rank, bank, row, col, len)

9.4.3 Issue Notification

When the scheduler picks an entry from this CAM:

Signal	Direction	Description
issued_slot_i	input	Slot index to mark as issued
issued_we_i	input	Mark-issued strobe

9.4.4 Beat Return (from rd_data_path_fub)

Signal	Direction	Description
beat_id_i	input	AXI ID of returning beat
beat_we_i	input	Beat-arrived strobe
entry_complete_o	output	One-hot vector of slots that just completed (last beat of their burst)

On entry-complete the slot's valid is cleared in the next cycle.

9.5 Lookup Mechanism

The CAM uses a **parallel match-line** structure: each slot has a small comparator for {rank, bank} that drives a per-slot match wire. The scheduler reads the full match-vector and picks among matches using a priority encoder (age-priority — oldest slot wins ties).

For the typical RD_CAM_DEPTH of 16 with BA=3, $clog2(NR)=1..2 (1 for single-rank, 2 for quad-rank), this is 16 4–5-bit equality comparators in parallel — trivial silicon.

9.6 Per-ID Beat Counting

Because reads return in-order *per AXI ID* (an AXI ordering rule), the read CAM does not need a per-beat slot table. A slot's beats_returned increments on each beat that returns with that ID, and when beats_returned == burst_len the slot completes and clears.

If AXI_OOO_ACROSS_IDS == false (rare), the SoC has asked for in-order return across all IDs, and the read CAM enforces this by issuing a head-of-queue priority hint to the scheduler — entries are still cleared in arrival order. The OoO-across-IDs default is true; per-ID in-order is the AXI default and is preserved.

9.7 Timing Budget

Path	Budget
push_valid → slot occupy (1 cycle)	1 cycle
q_rank_i / q_bank_i → match_*_o	combinational
beat_id_i →	1 cycle

Path	Budget
`beats_returned[slot]+1`	
`entry_complete_o` → slot clear	1 cycle

The combinational match path is on the scheduler's critical loop and is shared with `wr_cmd_cam_fub` and the bank machines' state matrix. See `ch02_blocks/07_scheduler.md` for the full critical-path breakdown.

9.8 CSR Hooks

- `STATUS.rd_cam_occ` — current occupancy
- `OBS_RD_CAM_HIGH_WATER` — peak occupancy observed since last read

9.9 TODO

- Mermaid for the parallel match-line topology
- Wavedrom: push → scheduler-match → issue → beats return → clear
- Per-ID head-of-queue ordering corner case spec

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

10 Write Command CAM (`wr_cmd_cam_fub`)

Module: `wr_cmd_cam_fub.sv` **Location:** `rtl/fub/` **Category:** FUB **Parent:** `ddr2_lpddr2_ctrl` **Status:** Skeleton

---

10.1 Purpose

`wr_cmd_cam_fub` is the **write-side content-addressable storage** of in-flight write commands. It mirrors the read CAM's role for write traffic but carries different per-slot metadata:

- Pointer into the AXI-side write-data buffer (`w_buf_ptr`)
- Pointer into the AXI-side write-strobe buffer (`strb_ptr`)
- Per-slot beats-issued counter (so the FUB knows when the last W beat has been pushed to DFI write data)

Separation from the read CAM lets the write path sit shallow — writes complete on B response, which is a single AXI event, so the write CAM does not need to track per-beat return.

10.2 Parameters

Parameter	Default	Purpose
WR_CAM_DEPTH	16	Number of in-flight write slots
IDW	AXI_ID_WIDTH	Key width
W_BUF_PTR_WIDTH	$clog2(W_BUF_DEPTH)	Pointer into AXI W buffer
BURST_LEN_WIDTH	8	Burst-length field width

10.3 Storage Per Slot

Field	Width	Description
valid	1	Slot occupied
axi_id	IDW	Key — the originating AXI ID
rank	$clog2(NR)	Decoded rank
bank	BA	Decoded bank
row	ROW_WIDTH	Decoded row
col_start	COL_WIDTH	Starting column
burst_len	BURST_LEN_WIDTH	Total beats
beats_issued	BURST_LEN_WIDTH	Beats already pushed to DFI write data
w_buf_ptr	W_BUF_PTR_WIDTH	Pointer into the AXI W-buffer for next beat
strb_ptr	W_BUF_PTR_WIDTH	Pointer into the per-beat strobe buffer
issued	1	Scheduler has picked this entry
b_pending	1	Awaiting B response from DFI write completion

Total per-slot width: ~58 bits in the default config. Total storage (16 × 58): ~116 bytes, distributed flops.

10.4 Interfaces

10.4.1 Push (from axi4_slave_fub)

Same shape as the read CAM push, plus `w_buf_ptr` and `strb_ptr` (which the AXI slave allocates at push time).

10.4.2 Scheduler Query

Same shape as the read CAM. The scheduler matches by (rank, bank) and selects with row-hit priority.

10.4.3 Issue Notification

When the scheduler picks a write, the CAM: 1. Marks `issued = 1` 2. Begins handing per-beat data to `wr_data_path_fub` via the `beat_data_pull` interface (below)

10.4.4 Beat-Pull (to wr_data_path_fub)

Signal	Direction	Description
`beat_pull_o`	output	Strobe — pull next beat for this slot
`beat_pull_slot_o`	output	Slot index whose beat is being pulled
`beat_pull_ptr_o`	output	`w_buf_ptr + beats_issued`
`beat_pull_strb_o`	output	`strb_ptr + beats_issued`
`beat_pull_last_o`	output	1 when this is the burst's final beat

`wr_data_path_fub` walks the AXI W buffer at the supplied pointer and pushes to DFI wrdata. On the last beat, it asserts `b_pending_set` to the CAM.

10.4.5 Completion (from DFI write window expiration)

Signal	Direction	Description
`b_pending_set_i`	input	Last W beat pushed; await tCWL+tWR window
`b_complete_i`	input	Write window expired (from `xbank_timers`)
`entry_complete_o`	output	Burst is fully complete; signal `axi4_slave_fub` to push B response

10.5 Difference From Read CAM

Aspect	Read CAM	Write CAM
Lifecycle	Push → issue → beats return → clear	Push → issue → beats issued → write window expires → B →

Aspect	Read CAM	Write CAM
		clear
Beat tracking	Counts beats *returned* from DFI	Counts beats *issued* to DFI; also tracks the post-issue write window
AXI ID echo	`rid` returned with each R beat	`bid` returned once at B-response time
Pointer fields	none (just a beat counter)	`w_buf_ptr`, `strb_ptr` into AXI-side buffers
Completion driver	`rd_data_path` (last beat of R)	`xbank_timers` (write window expiration) → `wr_data_path`

The asymmetry justifies the separate FUBs. Trying to share a CAM would either inflate the read slot to carry write-only fields, or vice versa.

10.6 Timing Budget

Same shape as the read CAM. The combinational match-line path is shared with `rd_cmd_cam_fub` in the scheduler's match aggregation.

10.7 CSR Hooks

- `STATUS.wr_cam_occ` — current occupancy
- `STATUS.wr_b_pending` — count of slots awaiting B response
- `OBS_WR_CAM_HIGH_WATER` — peak occupancy

10.8 TODO

- Mermaid for the issued → beat-pull → b_pending → b_complete lifecycle
- Wavedrom showing a multi-beat write burst from CAM push to B response
- Backpressure path when `wr_data_path` can't keep up with `beat_pull` (rare; DFI wrdata is wider than AXI W)

11 Transaction Queue (txn_queue_fub)

Module: txn_queue_fub.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** ddr2_lpddr2_ctrl
Status: Draft v0.1

> Architectural context: HAS §3.2 txn_queue. This block-level MAS section is the implementation detail: entry format, port arrangement, insert/clear paths, row_hit_cached coherency, backpressure, storage choice.

11.1 Purpose

txn_queue_fub is the **scheduling view** of in-flight DRAM transactions. It is the data structure the scheduler reads every MC clock to pick the next command. The queue is not the source of truth for AXI metadata or beat tracking — that lives in rd_cmd_cam_fub and wr_cmd_cam_fub. The queue carries *only* the fields the scheduler's priority function needs, plus a reverse pointer back to the CAM slot.

The split exists because the scheduler's read path is the hottest in the design: every cycle it touches every entry in parallel. Making the entry narrow (~32 bits, see below) keeps the parallel-comparator fan-in manageable. The CAMs carry the wider per-burst metadata (w_buf_ptr, strb_ptr, beats_returned, etc.) which is only touched on insert, on scheduler pick, and on per-beat completion — never in the per-cycle scheduling loop.

11.2 Relationship to the CAMs

Each entry in txn_queue reflects exactly one slot in rd_cmd_cam or wr_cmd_cam. The cam_slot_idx field is the reverse pointer:

```
txn_queue[q].cam_slot_idx ⟶ (is_write ? wr_cmd_cam[cam_slot_idx] :
rd_cmd_cam[cam_slot_idx])
```

When the scheduler picks txn_queue[q], it issues an "mark issued" strobe to the corresponding CAM using cam_slot_idx (see scheduler_fub, §2.7). When the CAM signals entry-complete (read: last beat returned; write: B-channel quiet point reached), it strobes back to clear txn_queue[q].

The queue and CAMs are sized independently:

Structure	Default depth	Reason
txn_queue	16	Scheduling pool — depth bounded by parallel-

Structure	Default depth	Reason
		comparator cost
`rd_cmd_cam`	16	One slot per pending read
`wr_cmd_cam`	16	One slot per pending write

In the default 16+16+16 configuration, the queue is the bottleneck — at most 16 transactions can be queued for scheduling at any time, regardless of how many slots are free in each CAM individually. The CAMs are not allowed to oversubscribe the queue (the `axi4_slave_fub` push path stalls on `txn_queue` full, not on CAM-full). The two CAMs *can* be sized larger than the queue if the read-path and write-path have very different burst-completion latencies — but that's not v1.

11.3 Synthesis Parameters

Parameter	Source	Effect
`TXN_QUEUE_DEPTH`	top (default 16)	Number of parallel-readable entries
`AGE_MAX`	top (default 256)	Age counter saturation; per-entry `clog2(AGE_MAX) = 8` bits
`ROW_WIDTH`	top	Width of the `row` field
`NUM_RANKS`	top	Width of the `rank` field
`NUM_BANKS`	top	Width of the `bank` field

11.4 Entry Format

Each slot carries the **scheduling view** — the minimum metadata the scheduler's priority function needs.

Field	Width	Description
`valid`	1	Slot occupied
`state`	2	`{FREE, PENDING, ISSUED, COMPLETING}` (see lifecycle below)
`is_write`	1	Picks rd vs wr CAM for the reverse pointer
`rank`	`$clog2(NUM_RANKS)`	Target rank (1-2 bits)
`bank`	`$clog2(NUM_BANKS)`	Target bank (3 bits for NB=8)

Field	Width	Description
row	ROW_WIDTH	Target row (14 bits default)
row_hit_cached	1	Cached at insert; refreshed on bank-state change
age	$clog2(AGE_MAX)	Saturating age counter (8 bits default)
cam_slot_idx	$clog2(CAM_DEPTH)	Reverse pointer to rd_cmd_cam or wr_cmd_cam slot (4 bits)
Total per slot	**~32 bits**	(default: NR=1, NB=8, ROW=14, AGE_MAX=256, CAM=16)

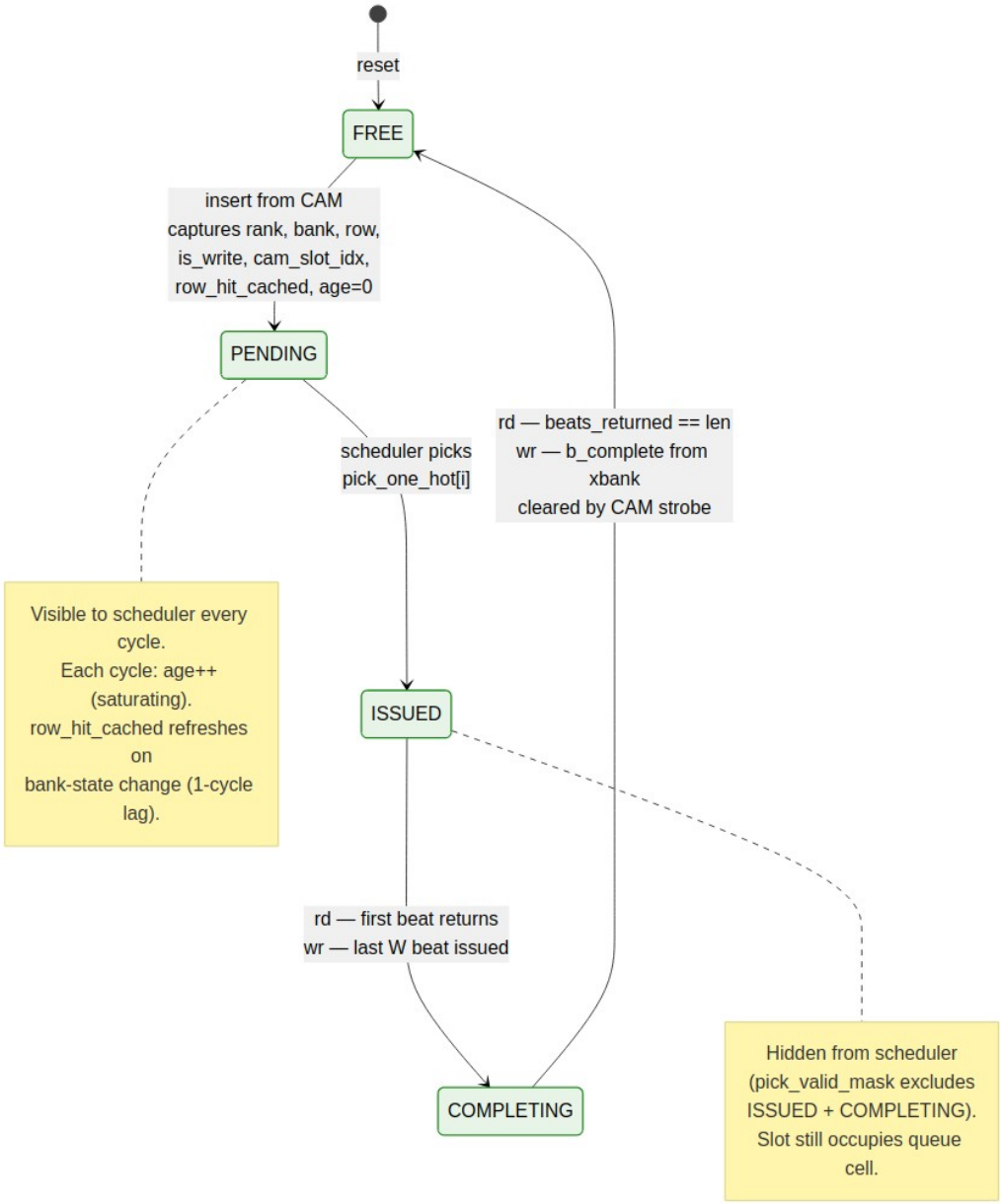
Total queue storage at depth 16: **~64 bytes** (16 × 32 b). This is distributed flops — no SRAM macro. Even at depth 64 it's 256 bytes, still trivially distributed.

Fields *not* in the queue entry (these stay in the CAM):

- col_start, burst_len — only consulted at issue time (Stage 4 of scheduler), not every cycle
- axi_id — opaque to the scheduler; only the CAM and axi4_slave_fub need it
- w_buf_ptr, strb_ptr, beats_returned, beats_issued — beat-level tracking, not scheduling

At issue time, the scheduler reads col_start and burst_len via the cam_slot_idx reverse pointer — a single read from the CAM's lookup port. This is one cycle slower than carrying them in the queue, but it keeps the queue narrow.

11.5 Entry Lifecycle



txn_queue Entry State Diagram

Source: 06_txn_queue_entry_state.mmd

Four states tracked per slot:

State	Visible to scheduler?	Counts toward queue depth?	Notes
FREE	No	No	Empty slot, available for insert
PENDING	Yes (eligible)	Yes	Scheduler is searching for this entry
ISSUED	No (masked out)	Yes	Picked by scheduler; column command in flight
COMPLETING	No (masked out)	Yes	Last beat / write-window draining

The scheduler's `pick_valid_mask` (see §2.7 Stage 2) explicitly excludes `ISSUED` and `COMPLETING` so the same entry can never be picked twice. The mask is `(state == PENDING)`.

`ISSUED → COMPLETING` is the same edge in both rd and wr cases but triggered by different signals:

- **Read**: first R beat returns from DFI → `rd_data_path_fub` strobes `entry_transition_completing` via the CAM.
- **Write**: last W beat is pushed to DFI write data → `wr_data_path_fub` strobes the same edge.

The `COMPLETING → FREE` edge is driven by the CAM's `entry_complete_o` strobe (see §2.4 and §2.5). The queue does not own completion-detection logic; it follows the CAM.

11.6 Ports

11.6.1 Insert (from CAMs)

Signal	Direction	Width	Description
ins_valid_i	input	1	New entry to push
ins_ready_o	output	1	Queue has a free slot
ins_is_write_i	input	1	
ins_rank_i	input	$clog2(NR)	from addr_mapper
ins_bank_i	input	$clog2(NB)	
ins_row_i	input	ROW_WIDTH	
ins_cam_slot_i	input	$clog2(CAM_DEPTH)	Reverse pointer

row_hit_cached is **not** an insert port — it's computed inside this FUB at insert time by querying bank_state_i[ins_rank][ins_bank].open_row (see "Insert Path" below). This keeps the insert-time row-hit computation local to the queue.

11.6.2 Parallel Read (to scheduler — every cycle)

Signal	Direction	Width	Description
q_entries_o[TXN_QUEUE_DEPTH-1:0]	output	per entry-format table	Live snapshot of all slots, parallel

This is a wide bus — at the default TXN_QUEUE_DEPTH = 16 and ~32 bits per entry, it's a 512-bit bus. The scheduler reads it combinationally; there's no read-port arbitration because there's only one reader.

11.6.3 Bank-State Watchpoint (from bank machines)

Signal	Direction	Width	Description
bank_state_i[NR][NB]	input	per bank_machine	Live bank state (used at insert for row_hit_cached)
bank_open_row_changed_i[NR][NB]	input	NR×NB	One-cycle strobe when a bank's open row changes
bank_new_open_row_i[NR][NB]	input	NR×NB × ROW_WIDTH	New open-row value, valid the cycle after the strobe

The _changed + _new_open_row pair is what drives the per-entry row_hit_cached refresh path (see "Row-Hit Cache Coherency" below).

11.6.4 Scheduler Pick (from scheduler)

Signal	Direction	Width	Description
pick_one_hot_i	input	TXN_QUEUE_DEPTH	One-hot vector of which slot the scheduler picked
pick_strobe_i	input	1	Pick happened this cycle

On pick_strobe_i, the indicated slot transitions PENDING → ISSUED.

11.6.5 CAM Completion (from CAMs)

Signal	Direction	Width	Description
entry_transition_i	input	2	01 = ISSUED → COMPLETING, 10 = COMPLETING → FREE
entry_transition_slot_i	input	$clog2(TXN_QUEUE_DEPTH)	Slot index that's transitioning
entry_transition_strobe_i	input	1	Transition strobe

11.6.6 Backpressure Output

Signal	Direction	Width	Description
q_high_water_o	output	1	High when occupancy ≥ SCHED_TUNING.txn_queue_high_water
q_full_o	output	1	All slots are non-FREE
q_occupancy_o	output	$clog2(DEPTH+1)	Current occupancy (number of slots not in FREE state)

q_high_water_o drives the AXI backpressure path in axi4_slave_fub — awready / arready deasserts when the queue is near full. q_full_o is the hard backpressure (no inserts accepted).

11.7 Insert Path

When axi4_slave_fub has decoded a new burst and pushed it to either CAM, the CAM in turn pushes to this queue:

```
cycle T:   CAM push from axi4_slave  →  ins_valid asserted, ins_ready
asserted
cycle T:   queue selects free_slot via priority encoder over (state ==
FREE)
cycle T:   row_hit_cached_init = (bank_state[ins_rank][ins_bank].state
== ACTIVE
                                  AND bank_state[ins_rank]
[ins_bank].open_row == ins_row)
cycle T+1: free_slot.{is_write, rank, bank, row, row_hit_cached,
```

```
cam_slot_idx} latched
cycle T+1: free_slot.state = PENDING, age = 0
```

The insert is a **two-cycle latency** from CAM push to first scheduler visibility, but the queue can accept one new insert per cycle (the comparator network for `free_slot` finds the next available slot combinationally; the latch is registered).

`row_hit_cached_init` is the only nontrivial computation at insert time. It's a single comparison: does the target bank's open row match the inserted row? If the bank is IDLE, it's automatically 0 (no row is open). If the bank is in the middle of a row change (ACTIVATING or PRECHARGING), it's also 0 — the cached value will be refreshed in the next cycle when the bank settles via the watchpoint path below.

11.8 Row-Hit Cache Coherency

The `row_hit_cached` field is computed at insert (above) and **refreshed** any time a bank's open row changes.

The refresh path:

```
cycle T:   bank_machine[r,b] transitions ACTIVE → ACTIVE-on-new-row
           (via PRE then ACT)  OR  IDLE → ACTIVE (via ACT)
cycle T:   bank_machine[r,b] strobes bank_open_row_changed[r][b] = 1
           and presents bank_new_open_row[r][b] = R_new
cycle T+1: txn_queue refresh path:
           for each slot i:
             if entries[i].state in {PENDING}
                AND entries[i].rank == r AND entries[i].bank == b:
               entries[i].row_hit_cached = (entries[i].row == R_new)
```

The refresh is broadcast-parallel: all `TXN_QUEUE_DEPTH` entries are checked in parallel (their `(rank, bank)` matches are computed; the matching set has its `row_hit_cached` recomputed). The refresh fires for `ISSUED` and `COMPLETING` entries too, but those are masked out by the scheduler anyway — refreshing them is harmless and cheaper than gating.

Why a one-cycle lag is acceptable. In the cycle the bank state changes, the scheduler may still observe stale `row_hit_cached`. The worst outcome is the scheduler picks an entry as a "row hit" that isn't, or vice versa. The bank_machine catches this — `accepts_rd` / `accepts_wr` are gated on the current open_row, so the scheduler can't issue an *incorrect* column command. It would issue a bare RD/WR when an RDA/WRA was warranted (or vice versa), costing at most one extra PRE later. This is the trade documented in scheduler §2.7.

The synchronous broadcast (rather than asynchronous combinational re-eval) keeps the scheduler's Stage-1 path off the bank_machine state machines. If we tried to make this combinational, the Stage-1 critical path would route through every bank machine's open_row

register, every queue entry's row comparator, and every priority mask — easily 8-10 LUT levels, ruining the timing budget.

11.9 Age Counter Saturation

Every cycle, every `PENDING` entry's `age` increments by 1. The counter saturates at `AGE_MAX - 1`. Concretely with `AGE_MAX = 256`:

- New entry: age = 0
- After 1 cycle: age = 1
- After 255 cycles: age = 255
- After 256+ cycles: age = 255 (saturated)

The scheduler's age priority encoder treats saturated entries equally — once an entry has been waiting "long enough," it has maximum priority and ties break by slot index. In normal traffic, ages rarely saturate; in pathological cases (a row-conflict victim behind a stream of row-hits) saturation provides the anti-starvation guarantee.

The runtime override `SCHED_TUNING.age_max_runtime` clips the effective saturation point lower than the build-time `AGE_MAX` — useful for characterization sweeps to shorten the saturation timescale without rebuilding.

Increment gating. Age does *not* increment when `state` is `ISSUED` or `COMPLETING`. Those entries are out of the scheduling pool; aging them further would skew the priority for any subsequent entries that get re-queued (which doesn't currently happen in v1, but the gate is cheap insurance).

11.10 Storage Choice

At the default `TXN_QUEUE_DEPTH = 16`, total storage is ~64 bytes. Distributed flops are the right answer — an SRAM macro would have higher access latency and the macro overhead is significant for so little data.

The crossover point where SRAM becomes attractive is around `TXN_QUEUE_DEPTH ≥ 64` (256 bytes), and even then only if the scheduler can be re-architected to issue from a partial snapshot rather than the full queue. Currently, the scheduler reads all entries in parallel — SRAM ports can't deliver that. So in v1 we cap at distributed flops; the parameter range is `4 … 64`.

For depth = 64, the entry bus is 64 × 32 = 2048 bits — wide but routable on 7-series. Stage-1 priority comparators scale linearly with depth (16 → 64 is 4× the comparators), and the age priority encoder tournament grows by $\log_2(4) = 2$ levels. The critical-path budget in §2.7 absorbs this at the 200 MHz target; at 500 MHz the scheduler will need flop insertion as noted.

11.11 CSR Hooks

CSR field	Source signal	Use case
STATUS.txn_queue_occ (R)	q_occupancy_o	Live occupancy
OBS_TXN_QUEUE_DEPTH_MAX (R)	High-water-mark register	Peak occupancy since last read (HAS §6.3)
OBS_TXN_QUEUE_DEPTH_AVG (R)	Rolling time-average	Time-averaged occupancy
STATUS.txn_queue_full_pulses (R)	Counter — number of times q_full_o asserted	Hard-backpressure event counter
SCHED_TUNING.txn_queue_high_water (R/W)	Threshold for q_high_water_o	Software-tunable soft-backpressure point

The _high_water threshold defaults to TXN_QUEUE_DEPTH - 2 so AXI has a few-burst headroom before hard backpressure. Software can tune it lower for tighter latency control.

11.12 Verification Notes (cocotb test plan)

Scenario	What it proves
Single insert, single pick, single clear	Smoke: FREE → PENDING → ISSUED → COMPLETING → FREE
Insert at full speed (1 per cycle) until full	Insert throughput; q_full_o asserts on the 17th
Bank-state change while N entries target that (rank, bank)	Broadcast row-hit refresh works on all matching slots
Bank-state change *during* a pick on the same slot	Pick is honored; refresh applies to next cycle's view
Age saturation across all queue slots	All saturate at AGE_MAX - 1; tie-break by slot index
age_max_runtime = 16, age saturates earlier	Runtime clip works

Scenario	What it proves
than `AGE_MAX`	
Insert + pick + complete in the same cycle (different slots)	The four ports are non-conflicting
`txn_queue_high_water = 8`; AXI backpressure asserts at occupancy 8	Soft backpressure threshold
`q_full_o` blocks CAM push; CAM stalls AXI	Hard backpressure path
Insert with `is_write = 0` and `is_write = 1` interleaved	Reverse-pointer routing to correct CAM
`entry_transition` with `slot` out-of-range (illegal)	Assertion fires; entry untouched
Reset during mid-life: all slots clear, no spurious completions emitted	Reset behavior

11.13 Open Questions / Future Work

- Should the queue allow **re-queue** of a `COMPLETING` entry that fails (e.g., a DFI rddata timeout)? Currently the CAM owns failure-recovery; if it decides to retry, it would have to push a new queue entry. A "re-queue without losing age" path would be cheaper but adds complexity. Defer to v2.
- Should `cam_slot_idx` be **wider** than `$clog2(CAM_DEPTH)` to allow over-sized CAMs? Currently sized to match the smaller of the two CAM depths. If the rd-CAM grows to 32 and wr-CAM stays at 16, the field width adapts at elaboration; this is fine but the field is technically over-provisioned for write entries. Not worth fixing in v1.
- The broadcast `row_hit_cached` refresh fires for `ISSUED` / `COMPLETING` entries too. These are masked at the scheduler anyway. The gate would save a few comparator switches but no flops — drop the gate optimization. Document the decision and move on.

12 FR-FCFS Scheduler (scheduler_fub)

Module: scheduler_fub.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** ddr2_lpddr2_ctrl
Status: Draft v0.1

> Architectural context: HAS §3.2 (scheduler, priority function, lookahead, force_inorder, refresh-priority). This block-level MAS section is the implementation view: signals, internal datapath, pipeline staging, critical paths, and verification scenarios.

12.1 Purpose

scheduler_fub is the central command issue engine. Every MC clock cycle it:

1. Snapshots txn_queue and all bank_machine states
2. Computes a per-entry readiness vector (which queue entries *could* issue right now)
3. Applies priority masking (init, refresh, force-inorder, row-hit-first, age tie-break)
4. Picks at most one winner via parallel priority encoders
5. Generates the DRAM command (issue_op, operands) and the CAM "mark issued" strobe

Issue rate is **one command per MC clock**. Multi-cycle commands (LPDDR2's 2-cycle CA encodings) are absorbed inside cmd_encoder_fub and gear_dfi_fub; the scheduler still sees one issue per cycle.

The block is the **single hardest synthesis-timing FUB** in the design — the parallel match-line into per-entry readiness, then into priority encoders, is what sets the MC clock frequency ceiling. Everything else has slack.

12.2 Synthesis Parameters

Parameter	Source	Effect on this FUB
SCHEDULER_MODE	top	"000" synthesizes the full FR-FCFS comparator network; "INORDER" synthesizes only the first-ready FIFO path (smaller area, no row-hit reordering).

Parameter	Source	Effect on this FUB
TXN_QUEUE_DEPTH	top (default 16)	Number of parallel readiness comparators
LOOKAHEAD_DEPTH_MAX	top (default 4)	Width of the lookahead window peek into txn_queue
PAGE_POLICY	top	Picks which Stage-4 auto-precharge decoder is synthesized
NUM_RANKS, NUM_BANKS	top	Width of the bank-state input matrix
AGE_MAX	top (default 256)	Width of per-entry age counter (clog2(AGE_MAX))

The "INORDER" synthesis variant is a different decoder tree, not a runtime gate of the OoO tree. The runtime SCHED_TUNING.force_inorder bit is honored *only* when SCHEDULER_MODE == "OOO" was synthesized (the bit becomes a tied observation in INORDER builds, per HAS §3.2).

12.3 Interface

12.3.1 Inputs

Signal	Direction	Width	Description
mc_clk, mc_rst_n	input	1, 1	MC clock + async-assert / sync-deassert reset
q_entries_i[TXN_QUEUE_DEPTH-1:0]	input	per HAS §3.2	Live queue snapshot: per entry {valid, is_write, rank, bank, row, col, burst_len, row_hit_cached, age, state, cam_slot_idx}
bank_state_i[NUM_RANKS][NUM_BANKS]	input	per bank_machine_fub	Per-(rank, bank): state, open_row, accepts_act/rd/wr/pre/ref
xbank_blocks_i	input	struct	blocks_act_per_rank[NR], blocks_xrank_rd, blocks_xrank_wr
refresh_wants_i	input	1	One-bit “refresh due now” from refresh_mgr_fub

Signal	Direction	Width	Description
refresh_done_for_i[NUM_RANKS][NUM_BANKS]	input	NR×NB	Per-(rank, bank) “refresh handshake granted” from refresh_mgr_fub
init_in_progress_i	input	1	From init_engine_fub — blocks all normal traffic
predict_hit_i[NUM_RANKS][NUM_BANKS]	input	NR×NB	HAPPY predictor output (tied 0 when PAGE_POLICY != HAPPY_HYBRID)
cfg_force_inorder_i	input	1	SCHED_TUNING.force_inorder runtime override
cfg_lookahead_active_i	input	4	SCHED_TUNING.lookahead_active
cfg_page_policy_or_i	input	2	REFRESH_TUNING.page_policy_or
cfg_happy_enable_i	input	1	SCHED_TUNING.happy_enable

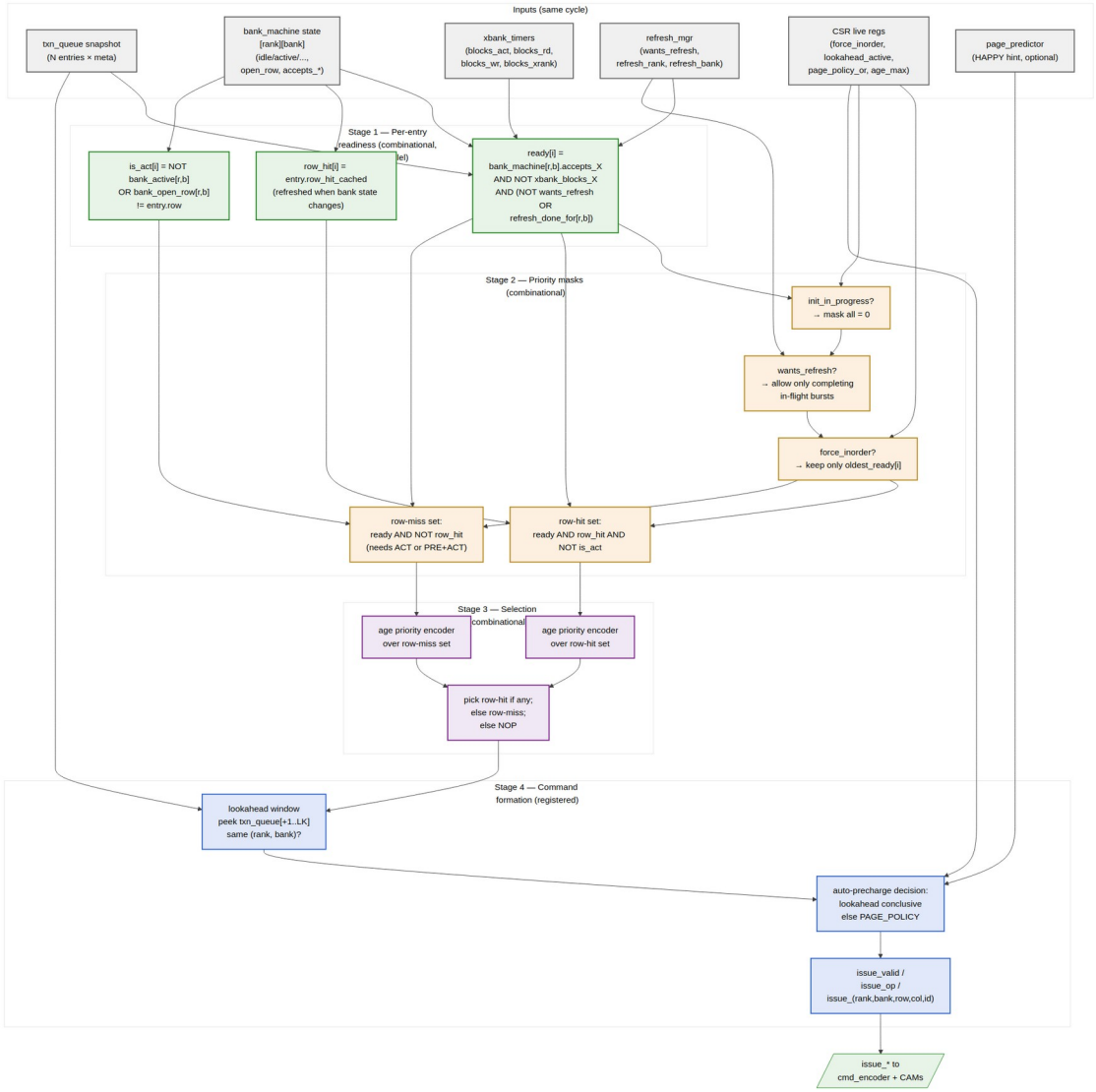
12.3.2 Outputs

Signal	Direction	Width	Description
issue_valid_o	output	1	A command is being issued this cycle
issue_op_o	output	4	One of {NOP, ACT, RD, RDA, WR, WRA, PRE, PREA, REF, REFPB, MRS, ZQCS, ZQCL}
issue_rank_o	output	$clog2(NR)	Target rank
issue_bank_o	output	$clog2(NB)	Target bank
issue_row_o	output	ROW_WIDTH	Target row (for ACT) or open-row pass-through (for RD/WR)
issue_col_o	output	COL_WIDTH	Target column (for RD/WR)

Signal	Direction	Width	Description
issue_axi_id_o	output	AXI_ID_WIDTH	Originating AXI ID (passed to the CAM for completion routing)
issue_cam_slot_o	output	clog2(CAM_DEPTH)	Index into rd_cmd_cam or wr_cmd_cam to mark “issued”
issue_is_write_o	output	1	Picks wr_cmd_cam vs rd_cmd_cam for the mark-issued strobe
predict_query_o	output	struct	(rank, bank, row) query to page_predictor_fub for stage-4 auto-pre
dbg_inorder_active_o	output	1	(debug) High when current cycle picked the in-order path
dbg_refresh_blocking_o	output	1	(debug) High when wants_refresh reduced the candidate pool

12.4 Decision Pipeline

The decision is a single MC-clock-cycle combinational pipeline broken into four conceptual stages, finishing in a registered output. Stages 1–3 are combinational; Stage 4 latches the final command into output flops.



Scheduler Priority Pipeline

Source: 05_scheduler_priority_pipeline.mmd

12.4.1 Stage 1 — Per-Entry Readiness (combinational, N parallel comparators)

For each of `TXN_QUEUE_DEPTH` entries, in parallel:

```
ready[i] = q_entries[i].valid
         AND  (q_entries[i].state == PENDING)
         AND  bank_machine[r,b].accepts_X
         AND  NOT xbank_blocks_X
         AND  ( NOT refresh_wants
                OR  bank_machine[r,b].in_refresh_done_set )
```

```
is_row_hit[i] = q_entries[i].row_hit_cached
                ( cached at queue insert; refreshed when
bank_machine[r,b]
                  state changes — see "Cache Coherency" below )

needs_act[i]  = NOT bank_active[r,b]
                OR  bank_open_row[r,b] != q_entries[i].row
```

Where X is `rd` if `is_write == 0`, `wr` if `is_write == 1`. `r = q_entries[i].rank`, `b = q_entries[i].bank`.

`ready[i]` answers "could I issue *some* command for this entry this cycle without violating any per-bank or cross-bank timer?" `is_row_hit[i]` answers "if I do, is it a column command on the already-open row?" `needs_act[i]` answers "is the preliminary command actually ACT (or PRE+ACT) rather than RD/WR?"

12.4.2 Stage 2 — Priority Masking (combinational)

Five masks, computed in parallel from the Stage-1 vector and the config inputs:

Mask	Definition	Effect
`init_mask`	`init_in_progress ? 0 : ready[i]`	Block all normal entries during init
`refresh_mask`	`refresh_wants ? (ready[i] AND bank_in_refresh_done_set) : ready[i]`	Allow only entries whose target rank+bank has already been granted the refresh handshake (i.e., already transitioned out of REFRESHING, or never entered it)
`inorder_mask`	`force_inorder ? oldest_ready_one_hot : ready[i]`	Collapse to first-ready FIFO
`row_hit_mask`	`ready[i] AND is_row_hit[i] AND NOT needs_act[i]`	Row-hit candidates (column-only)
`row_miss_mask`	`ready[i] AND (NOT is_row_hit[i] OR needs_act[i])`	Row-miss candidates (need ACT or PRE+ACT)

The composed candidate vectors are:

```
cands_row_hit  = init_mask AND refresh_mask AND inorder_mask AND
row_hit_mask
cands_row_miss = init_mask AND refresh_mask AND inorder_mask AND
row_miss_mask
```

12.4.3 Stage 3 — Selection (combinational priority encoders)

Two parallel age-priority encoders, then a 2:1 mux:

```
winner_hit  = age_pe(cands_row_hit)    // index of oldest entry in
cands_row_hit, or none
winner_miss = age_pe(cands_row_miss)   // index of oldest entry in
cands_row_miss, or none

picked = winner_hit  IS_VALID ? winner_hit
       : winner_miss IS_VALID ? winner_miss
                              : NONE   // issue NOP
```

The age priority encoder is the slowest path in this stage — it's an `N`-entry tournament on `clog2(AGE_MAX) = 8`-bit ages. For `N = 16` (the default), that's 4 levels of pairwise compare-and-select. Synthesizable in two LUT layers per level on 7-series; ~30 LUT-delays in the typical config.

12.4.4 Stage 4 — Command Formation (registered)

Once an entry is picked, Stage 4 turns it into a DRAM command. This stage **may pre-flop intermediate signals** for timing closure on the highest clock targets (see "Pipeline Staging" below).

1. **Lookahead peek** — examine the next `cfg_lookahead_active` queue entries *after* the picked one to see if any target the same (rank, bank).
2. **Auto-precharge decision** — per HAS §3.2 (lookahead first, fallback to `PAGE_POLICY`):
   - Same-bank entry found AND its row matches the open row → **keep open** → bare RD / WR
   - Same-bank entry found AND its row differs → **close** → RDA / WRA
   - No same-bank entry within window → consult `cfg_page_policy_or`:
     - OPEN → bare RD / WR
     - CLOSE → RDA / WRA
     - HAPPY_HYBRID → query `predict_hit_i[r][b]`; if predicted hit → bare; else RDA/WRA
3. **Op encoding** — generate the 4-bit `issue_op_o` from `(is_write, needs_act, auto_pre)`:
   - `needs_act` and bank state == IDLE → `ACT`
   - `needs_act` and bank state == ACTIVE (different row) → `PRE` (the column command follows next cycle after tRP)
   - `NOT needs_act` and `is_write` → `WR` or `WRA`
   - `NOT needs_act` and `NOT is_write` → `RD` or `RDA`

4. **Latch** outputs into the issue flops.

12.4.5 Refresh Window Behavior

When `refresh_wants_i` is high, `refresh_mgr_fub` is asserting `refresh_req` to one or more bank machines. Those bank machines drive `refresh_done_for_i[r][b] = 1` only after reaching IDLE and handing over the grant. While `refresh_done_for_i[r][b] == 0`, that bank's entries are masked out — no new column commands to a bank that's about to refresh.

Bank machines for *other* ranks (in the multi-rank REFab round-robin) or *other* banks (in REFpb) remain available, so the scheduler continues to drain those during the refresh window. This is what makes per-rank REFab dispatch beneficial — the non-refreshing ranks keep their throughput.

When all required refreshes complete and `refresh_wants_i` drops, the masks lift and normal FR-FCFS resumes the next cycle.

12.5 Cache Coherency: `row_hit_cached`

`q_entries[i].row_hit_cached` is computed at queue insertion (by `axi4_slave_fub` via a one-cycle query to `bank_machine[r,b].open_row`) and stored in the queue entry. This avoids re-querying every cycle. But the bank state can change between insertion and selection:

- When `bank_machine[r,b]` transitions ACTIVE → ACTIVE on a different row (via PRE + ACT to a new row), all `q_entries` with `(rank, bank) == (r, b)` need their `row_hit_cached` recomputed against the new `open_row`.
- When `bank_machine[r,b]` transitions IDLE → ACTIVATING, the queue entries get `row_hit_cached = 1` for entries whose row matches the in-flight ACT.

This is implemented as a **broadcast update**: on every bank state transition that affects the open row, the bank machine emits `open_row_changed[r][b]` + the new `open_row`. The queue has a per-entry update path that, in parallel, recomputes `row_hit_cached`. This is a one-cycle update — the recomputed value is visible to the scheduler in the cycle *after* the bank transition.

The one-cycle lag means the scheduler may make an "outdated row-hit" decision once per row change. The downside is one mis-categorized RD/WR (issued as bare when RDA was warranted, or vice versa); the worst case is one extra PRE later. This is a deliberate trade — the alternative is a combinational broadcast from bank state through every queue entry's row-hit comparator, which would inflate the Stage-1 critical path.

12.6 Pipeline Staging

The default build is a **single-cycle combinational** scheduler — Stages 1–3 are combinational; Stage 4's auto-precharge decoder is combinational; only the final issue-flop is registered. On 7-series FPGA at 200 MHz this closes timing comfortably. For 14 nm targets at 500+ MHz, two timing-closure escape hatches:

1. **Stage 3 → Stage 4 flop insertion** — register `picked` between stages. Adds one cycle of issue latency. Throughput is unchanged because the pipeline is single-issue anyway; the only visible difference is that a request inserted at cycle T can first issue at cycle T+3 instead of T+2.

2. **Stage 1 partition by bank** — partition the readiness comparator network by (rank, bank) so each partition produces a per-bank ready vector independently, then aggregate. Useful when `NUM_RANKS × NUM_BANKS` is small (the default 1×8 = 8 partitions) and routing congestion dominates.

These are not on by default; they're synthesis-script switches, not RTL parameters.

12.7 Strict In-Order Mode (collapse)

When `cfg_force_inorder_i = 1` (and `SCHEDULER_MODE == "OOO"` is synthesized):

- `inorder_mask` collapses to `oldest_ready_one_hot` — a one-hot vector that picks the *oldest valid ready entry* by age.
- This drops the row-hit prioritization (no row-hit vs row-miss split — both go through `cands_row_miss` since age order rules).
- Refresh priority is preserved (JEDEC requirement).
- Lookahead and HAPPY remain on at elaboration but can be independently disabled at runtime (`cfg_lookahead_active = 0`, `cfg_happy_enable = 0`) for a pure first-ready FIFO.

When `SCHEDULER_MODE == "INORDER"` is synthesized:

- The Stage-2 / Stage-3 priority encoders are different RTL: the row-hit/row-miss split is omitted; only the oldest-ready selector is built.
- `cfg_force_inorder_i` becomes a tied observation bit at `STATUS.force_inorder_obs = 1`.

12.8 Critical-Path Analysis (rough budget at 500 MHz target, 2 ns cycle)

Path	Levels (approx)	Budget
`bank_state_i.accepts_X` → `ready[i]` → row-hit mask → `cands_row_hit[i]`	4 LUT levels	0.7 ns
`cands_row_hit` → age priority encoder → `winner_hit` (16-entry tournament)	4 levels × 2 LUT each = 8	1.2 ns
`winner_hit / winner_miss` → 2:1 mux → `picked`	1 LUT level	0.2 ns
Routing / setup margin		0.3 ns
Total Stage 1–3		**2.4 ns**

So at 500 MHz the path **misses by ~400 ps** and needs Stage 3 → Stage 4 flop insertion (the first escape hatch above). At 200 MHz target (5 ns cycle, embedded SoC default), the path closes with ~2.6 ns of slack — no escape hatch needed.

The 500 MHz analysis is FPGA-pessimistic; ASIC targets close 500 MHz on this path comfortably with standard pipelining.

12.9 CSR Hooks

The scheduler drives several CSR observability fields:

CSR field	Source signal	Use case
`STATUS.issue_op_obs` (R)	Last issued `issue_op_o`	Bring-up debug: what was the last DRAM command
`STATUS.scheduler_idle_pct` (R)	Rolling counter — fraction of cycles `issue_valid_o = 0`	Bandwidth-headroom telemetry
`OBS_ROW_HIT_RANK<R>_BANK<N>` (R)	Incremented when row-hit path wins for that bank	Per-bank row-hit rate (HAS §6.3)
`OBS_INORDER_FALLBACKS` (R)	Incremented when `force_inorder` ate a row-hit	Tells software how much OoO is currently giving up

CSR field	Source signal	Use case
	win	
`OBS_LOOKAHEAD_HITS` (R)	Incremented when Stage-4 lookahead was conclusive	Tunes `LOOKAHEAD_DEPTH_MAX` characterization sweep
`STATUS.force_inorder_obs` (R)	Echo of effective force-inorder state	Software-readable confirmation

12.10 Verification Notes (cocotb test plan)

Scenario	What it proves
Single-write, single-bank, single-rank	Smoke test: ACT → WR → b_complete cycle
Single-read, single-bank, single-rank	Smoke test: ACT → RD → beat return
Two reads, same bank, same row, oldest first	Row-hit reordering not invoked (already in order)
Two reads, same bank, *different* rows, oldest is row-conflict	OoO: younger row-hit wins over older row-miss
Same as above with `force_inorder = 1`	OoO disabled: older row-miss wins despite younger row-hit
Cross-bank: rd to bank 0 and rd to bank 1, bank 0 row-miss, bank 1 row-hit	Bank parallelism: bank-1 issue fires first
Cross-rank (NUM_RANKS=2): rd to rank 0 bank 3 and rd to rank 1 bank 3	Per-rank bank parallelism
Refresh during in-flight burst	Refresh window correctly drains and resumes
Refresh during multi-rank traffic (REFab on rank 0, traffic on rank 1)	Per-rank REFab dispatch: rank-1 traffic continues
Age-saturation under sustained row-conflict from younger IDs	Older row-conflict eventually wins via age priority
`LOOKAHEAD_DEPTH_MAX = 4`, lookahead concludes auto-precharge correctly	Lookahead path
`LOOKAHEAD_DEPTH_MAX = 0`, HAPPY	Fallback path through page

Scenario	What it proves
predictor used	predictor
`PAGE_POLICY = CLOSE`, every column command is auto-pre	CLOSE policy
Init-in-progress blocks normal traffic	`init_mask` gating
Queue full + back-pressure to AXI	`axi4_slave` does not accept new bursts

12.11 Open Questions / Future Work

- Should QoS (`awqos` / `arqos`) be promoted from the v1 "no behavior" pass-through to a real priority boost in the FR-FCFS function? Currently HAS §3.2 leaves QoS as a side-band hint for v2. The hook would be a `qos_boost_mask` between the row-hit and row-miss stages.
- The cross-rank read-to-write turnaround (`tRTRS` + write window) lives in `xbank_timers_fub` today; the scheduler consumes the gate but does not pre-plan around it. A smarter scheduler could **batch by rank** (issue all ready reads on rank 0, then all on rank 1) to amortize `tRTRS`. This is a v2 feature; v1 takes the simpler per-cycle gating.
- Whether `OBS_INORDER_FALLBACKS` is useful for characterization is not yet validated; the counter is cheap, but if it's never read by the bring-up team it can be dropped in v2.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

13 HAPPY Page Predictor (`page_predictor_fub`)

Module: `page_predictor_fub.sv` **Location:** `rtl/fub/` **Category:** FUB (conditional — synthesized only when `PAGE_POLICY == HAPPY_HYBRID`) **Parent:** `ddr2_lpddr2_ctrl` **Status:** Draft v0.1

> Architectural context: HAS §3.2 `page_predictor`. The algorithm view (query + update flow) is in `ddr2_lpddr2_has/assets/mermaid/09_happy_predictor.png` — refer to

it for the algorithm. This block-level MAS section is the implementation view: hash inputs, table organization, multi-rank handling, warmup / hysteresis, storage choice, scheduler timing.

13.1 Purpose

`page_predictor_fub` is the HAPPY hybrid page-conflict predictor (Ghasempour et al., 2015). It answers one question for the scheduler:

> "For a request I am about to issue to (rank, bank), will the *next* access to this (rank, bank) hit the row I am about to leave open, or conflict with it?"

The answer drives the Stage-4 auto-precharge fallback in `scheduler_fub` (§2.7) when the lookahead window is inconclusive. A "next will hit" prediction → issue bare RD/WR (keep row open). A "next will conflict" prediction → issue RDA/WRA (close row in flight to save tRP later).

The FUB is **synthesized only when `PAGE_POLICY == HAPPY_HYBRID`** at elaboration. Builds with `PAGE_POLICY == OPEN` or `PAGE_POLICY == CLOSE` do not instantiate this FUB; the scheduler ties its `predict_hit_i` input to 1 (default-to-open) or 0 (default-to-close) respectively.

A runtime kill-switch (`SCHED_TUNING.happy_enable`) lets software A/B-test the predictor's contribution without rebuilding. When `happy_enable == 0`, the FUB stays in silicon but its output forces `predict_hit = 1` (default open).

13.2 Conditional Instantiation

```
generate
    if (PAGE_POLICY == "HAPPY_HYBRID") begin : g_happy
        page_predictor_fub #(
            .PAGE_PREDICTOR_TABLE_BITS (PAGE_PREDICTOR_TABLE_BITS),
            .NUM_RANKS                 (NUM_RANKS),
            .NUM_BANKS                 (NUM_BANKS),
            .ROW_WIDTH                 (ROW_WIDTH)
        ) u_page_predictor ( ... );

        assign predict_hit_to_sched = u_page_predictor.predict_hit_o;
    end
    else if (PAGE_POLICY == "OPEN") begin : g_open
        assign predict_hit_to_sched = '1;        // always predict
"hit" → bare RD/WR
    end
    else begin : g_close // CLOSE
```

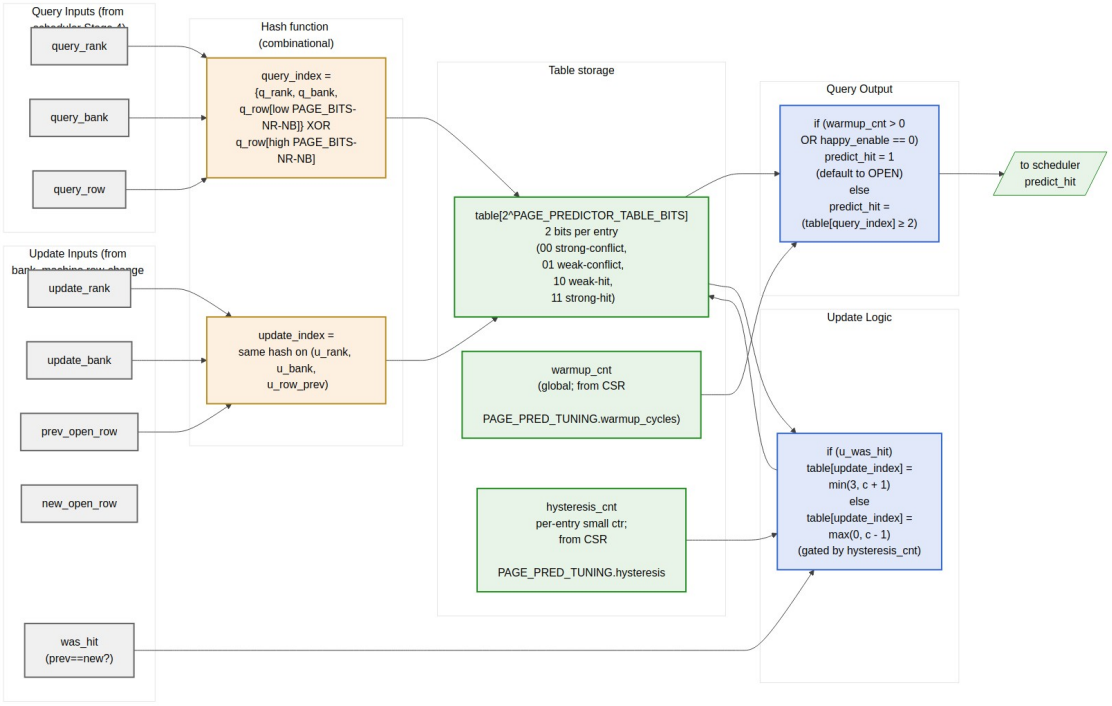
```
        assign predict_hit_to_sched = '0;        // always predict
"conflict" → RDA/WRA
    end
endgenerate
```

The scheduler's interface is identical in all three cases; the predictor's presence is invisible at the scheduler boundary.

13.3 Synthesis Parameters

Parameter	Source	Effect
PAGE_PREDICTOR_TABLE_BITS	top (default 12)	Table size = 2^PAGE_PREDICTOR_TABLE_BITS entries (default 4 K entries)
NUM_RANKS	top	Folded into the hash index
NUM_BANKS	top	Folded into the hash index
ROW_WIDTH	top	Source for the row-bit hash inputs
HYSTERESIS_BITS	derived	Width of per-entry hysteresis counter (small, e.g., 3 bits)
WARMUP_WIDTH	derived	Width of global warmup counter (16-bit per CSR field)

13.4 Block Implementation View



Page-Predictor Implementation — hash, table, warmup, hysteresis, query/update

Source: 09_page_predictor_implementation.mmd

The HAS-side algorithm view (`ddr2_lpddr2_has/assets/mermaid/09_happy_predictor.png`) shows the query/update *flow*; the diagram above shows the FUB's *physical organization* — input ports, hash, table, gate logic.

13.5 Hash Function

The table is indexed by a hash of (rank, bank, row). Multi-rank is folded into the same table — there is no per-rank replication. The hash is the **concatenation of identity bits with a XOR-folded row hash**:

```
hash_inputs    = {rank, bank, row}        // total width: clog2(NR) +
clog2(NB) + ROW_WIDTH
table_idx_width = PAGE_PREDICTOR_TABLE_BITS

// Identity portion: low (clog2(NR) + clog2(NB)) bits of the index
id_portion = {rank, bank}                  // width clog2(NR) +
clog2(NB)
```

```
// Hash portion: remaining (table_idx_width - id_portion_width) bits
of the index
hash_portion = row[low_HASH_BITS]
               XOR row[mid_HASH_BITS]
               XOR row[high_HASH_BITS]
               // (each slice is HASH_BITS = table_idx_width -
id_portion_width wide)

table_idx = {id_portion, hash_portion}
```

Concrete example at default config (`NR=1, NB=8, ROW_WIDTH=14, PAGE_PREDICTOR_TABLE_BITS=12`):

- `id_portion` = `{0-bit rank, 3-bit bank}` = 3 bits
- `hash_portion` = 9 bits = `row[8:0] XOR row[13:5]` (XOR of two row slices)
- `table_idx` = `{3-bit bank, 9-bit row_hash}` — 4 K entries, 8 banks each get a 512-entry slice of the table

For multi-rank (`NR=4`, others same):

- `id_portion` = `{2-bit rank, 3-bit bank}` = 5 bits
- `hash_portion` = 7 bits = `row[6:0] XOR row[13:7]`
- `table_idx` = `{2-bit rank, 3-bit bank, 7-bit row_hash}` — 4 K entries, 32 (rank, bank) tuples each get a 128-entry slice

Why fold rank into identity rather than hash. Folding rank into the identity portion guarantees that two different ranks never collide for the same row hash. The trade-off is that the per-(rank, bank) slice shrinks (128 entries each at NR=4 vs 512 at NR=1). At default 4 K-entry table, this is fine — even 128 entries per slice is more than enough to track the working set of a typical bank.

If `PAGE_PREDICTOR_TABLE_BITS` is small enough that the identity portion eats the whole index (e.g., `PAGE_PREDICTOR_TABLE_BITS = 5` at `NR=4, NB=8`), there is no `hash_portion` and the predictor degrades to a per-(rank, bank) single-counter. An elaboration assertion fires in that case requiring `PAGE_PREDICTOR_TABLE_BITS >= id_portion_width + 4`.

13.6 Table Storage

Each entry is a 2-bit saturating counter:

- `00` = strong conflict-likely
- `01` = weak conflict-likely
- `10` = weak hit-likely (reset value)
- `11` = strong hit-likely

A `predict_hit` query returns 1 when the entry value is `>= 2` (hit-likely side of the saturating range).

Total storage = `2^PAGE_PREDICTOR_TABLE_BITS × 2 bits`. Concrete:

PAGE_PREDICTOR_TABLE_BITS	Entries	Storage	Implementation
8	256	512 bit	Distributed flops
12 (default)	4 K	8 Kbit (1 KB)	Single BRAM18 (FPGA) / small RF (ASIC)
16	64 K	128 Kbit (16 KB)	Multiple BRAM tiles or SRAM macro

The storage choice is synthesis-script driven: a SystemVerilog `(* ram_style = "block" *)` attribute on the table array hints BRAM for sizes ≥ 8 (the BRAM18 sweet spot at 512×36-bit = 512 entries × 4 bits, which fits two predictor lookups per BRAM word).

At the default 12-bit (4 K entries), a single 36 Kbit BRAM18 holds the entire table with room to spare for hysteresis counters. The query path is one BRAM-read cycle latency (registered output) — important for the scheduler's Stage-4 timing.

BRAM port arrangement. The table has one read port (the query path) and one read-modify-write port (the update path). On 7-series BRAM, this is naturally a true dual-port configuration. Update and query can happen in parallel.

13.7 Warmup Counter

A global warmup counter delays the predictor's first trustworthy output. At reset:

```
warmup_cnt = PAGE_PRED_TUNING.warmup_cycles    // 16-bit, default 4096
```

While `warmup_cnt > 0`:

- Every cycle decrement `warmup_cnt`
- Query output forces `predict_hit_o = 1` (default open)

When `warmup_cnt == 0`:

- Query output is `(table[table_idx] >= 2)`

The warmup exists because the table starts at "weakly hit-likely" everywhere. Without it, the predictor would output uniform "hit" for the first few hundred accesses anyway — but the warmup makes that explicit and tunable. Software can shorten the warmup for short-running workloads or extend it for cold-start benchmarks.

13.8 Hysteresis Per Entry

To avoid the predictor flipping on every single misprediction, each table entry has a small **hysteresis counter** that gates updates:

```
struct entry_t {
    logic [1:0] saturating_counter;
    logic [HYSTERESIS_BITS-1:0] hyst_cnt;
};

// On update with was_hit:
//   if (hyst_cnt > 0):
//     hyst_cnt = hyst_cnt - 1
//     (no change to saturating_counter)
//   else:
//     update saturating_counter (inc on hit, dec on conflict)
//     hyst_cnt = PAGE_PRED_TUNING.hysteresis
```

At reset `hyst_cnt = 0` (full sensitivity). The CSR-supplied hysteresis value (default 0) is added per update. With `PAGE_PRED_TUNING.hysteresis = 3`, the predictor only updates every 4th conflicting observation — useful for workloads with high inherent variance.

The hysteresis flops are *per-entry* — at default 12-bit table with 3-bit hysteresis, total hysteresis storage is `4096 × 3 = 12 Kbit`, which is significant. The hysteresis_bits parameter defaults to 0 (disabling hysteresis); enabling it is opt-in via `HYSTERESIS_BITS` at elaboration. When `HYSTERESIS_BITS == 0` the FUB synthesizes no hysteresis flops and the update fires every cycle.

13.9 Update Path

The predictor is *trained* by `bank_machine_fub`'s open-row change broadcast (§2.9). Each time a bank's open row changes (via PRE+ACT or auto-precharge → ACT), the predictor learns:

```
was_hit = (new_open_row == prev_open_row_when_request_was_made)
        // i.e., did the next access actually hit the row we predicted
on?
```

This requires the predictor to remember the row context of each prediction it made. The simplest implementation: a small per-bank shadow register that captures the row each request was predicted on. On open-row change broadcast, the shadow register's row is the "prev" row that was predicted on, and the broadcast's new_open_row is what the actual row turned out to be.

The shadow register is `ROW_WIDTH × NR × NB` flops total — for default config (14 × 1 × 8) = 112 flops. For multi-rank (14 × 4 × 8) = 448 flops.

Why this works. The HAPPY paper trains on actual observed outcomes; we observe outcomes by watching bank state transitions. A row-hit observation occurs when the bank stays ACTIVE on the same row across multiple column commands; a row-conflict occurs when the bank transitions PRE → ACT (new row).

13.10 Query Path Timing

The scheduler's Stage-4 (per §2.7) queries the predictor when:

- An entry has been picked for issue
- Lookahead is inconclusive (no same-bank pending in the lookahead window)
- `PAGE_POLICY == HAPPY_HYBRID`

The query is single-cycle:

```
cycle T:   scheduler asserts query_valid_o, query_rank/bank/row
cycle T:   predictor computes hash, indexes table (BRAM read)
cycle T+1: predict_hit_o is valid
cycle T+1: scheduler latches the auto-precharge decision
```

This puts the predictor in the scheduler's Stage-4 register-to-register path. The BRAM read access dominates: at FPGA targets ~1.5 ns. Combined with the scheduler's Stage-4 mux, the total Stage-4 path is ~2.2 ns — fits the 200 MHz target (5 ns) with slack; the 500 MHz target requires an extra register stage on the predict_hit output.

If a query and an update target the same table entry in the same cycle, the BRAM's true-dual-port write-first behavior makes the read return the newly-written value. This is fine for correctness — the more-recent training applies.

13.11 Interface

13.11.1 Query (from scheduler)

Signal	Direction	Width	Description
`query_valid_i`	input	1	Scheduler is requesting a prediction
`query_rank_i`	input	`$clog2(NR)`	Query rank
`query_bank_i`	input	`$clog2(NB)`	Query bank
`query_row_i`	input	`ROW_WIDTH`	Row of the request being predicted

Signal	Direction	Width	Description
predict_hit_o	output	1	1-cycle-later: 1 = predicted hit, 0 = predicted conflict

13.11.2 Update (from bank machine broadcast)

Signal	Direction	Width	Description
bank_orw_changed_i	input	NR×NB	One-cycle strobe per (rank, bank) — from §2.9 broadcast
bank_new_open_row_i	input	NR×NB × ROW_WIDTH	New open row value

The predictor internally maintains the shadow `prev_open_row[NR][NB]` register described above; the strobe + new row let it compute `was_hit = (bank_new_open_row_i == prev_open_row_shadow[r][b])`.

13.11.3 CSR live inputs

Signal	Direction	Width	Source
cfg_warmup_i	input	16	PAGE_PRED_TUNING.warmup_cycles
cfg_hysteresis_i	input	8	PAGE_PRED_TUNING.hysteresis
cfg_happy_enable_i	input	1	SCHED_TUNING.happy_enable

13.11.4 Telemetry outputs

Signal	Direction	Width	Description
dbg_table_warm_pct_o	output	8	Fraction of table entries that have updated at least once
dbg_accuracy_per_bank_o[NR][NB]	output	8	Rolling prediction accuracy per (rank, bank)
dbg_warmup_active_o	output	1	warmup_cnt > 0

The accuracy telemetry is the **single most useful debug signal** in the FUB. The bring-up team will watch it to decide whether HAPPY is helping, hurting, or break-even on the workload of the day.

13.12 CSR Hooks

CSR field	Source	Use case
OBS_PAGE_PRED_ACCURACY (R)	Aggregate accuracy across all banks	Headline predictor health
OBS_PAGE_PRED_ACCURACY_R<R>_B<N> (R)	Per-(rank, bank) accuracy (sparse, mirrors OBS_BANK_OPEN_ROW_*)	Per-bank predictor health
STATUS.page_pred_warmup_active (R)	dbg_warmup_active_o	Is the predictor producing real predictions yet?
PAGE_PRED_TUNING.warmup_cycles (R/W)	cfg_warmup_i	Tunable warmup
PAGE_PRED_TUNING.hysteresis (R/W)	cfg_hysteresis_i	Tunable hysteresis

13.13 Verification Notes (cocotb test plan)

Scenario	What it proves
Cold reset; all queries return predict_hit = 1 (warmup phase)	Warmup gate works
warmup_cycles = 0; first query after reset goes through the table	Warmup can be disabled
warmup_cycles = 4096; query returns gate-default until cycle 4096	Warmup countdown
Train 100 row-hits to (rank 0, bank 3); predict_hit on a fresh query to same bank	Counter trained to strong-hit
Train 100 row-conflicts to (rank 0, bank 3); predict_hit returns 0	Counter trained to strong-conflict
hysteresis = 3; counter updates only every 4th observation	Hysteresis gate
Multi-rank (NR=2): training on (0, b) does not affect (1, b)	Rank-isolation via identity bits
Two different rows on same bank with identical low-bits collision	Hash sensitivity to row mid/high bits

Scenario	What it proves
`happy_enable = 0` runtime; queries return 1 regardless of table state	Runtime kill-switch
`PAGE_POLICY != HAPPY_HYBRID` build: FUB not instantiated	Conditional gen check
Concurrent query + update to same hash bucket	BRAM true-dual-port write-first read returns new value
Predictor accuracy CSR telemetry tracks ground truth	Accuracy counter correctness

13.14 Open Questions / Future Work

- **Per-bank vs unified table.** A unified table (current design) shares storage across all (rank, bank) pairs. An alternative is a per-bank smaller table — saves cross-bank interference at the cost of less aggregate storage. The unified design's identity-bits scheme already provides per-(rank, bank) isolation at the index level; per-bank tables would only help if the row hash collisions are concentrated. Punt; revisit if characterization shows clustering.
- **Update gating during refresh.** When a bank's open-row changes due to refresh (REFRESHING → IDLE → ACT), the predictor sees a row-change broadcast that isn't really a "prediction outcome." Currently the update path doesn't distinguish. A flag bit on the broadcast (`is_refresh_induced`) would let the predictor skip these updates. Cheap to add; not in v1.
- **Two-level predictor.** HAPPY in the paper is a two-level predictor (history table + decision table). The v1 implementation is the one-level decision-table only — simpler and the paper's results show one-level captures most of the benefit. The two-level variant could be added as `PAGE_PREDICTOR_LEVELS = 2` parameter if characterization shows v1 misses the workload's true pattern.
- **Predictor accuracy across runtime mode changes.** When `force_inorder` toggles or `lookahead_active` changes, the predictor's training set distribution shifts. Should the accuracy counters reset on these mode changes? Probably yes — adds one extra clear strobe. Punt to v2 unless bring-up calls it out.

14 Bank Machine (bank_machine_fub)

Module: bank_machine_fub.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** ddr2_lpddr2_ctrl (instantiated NUM_RANKS × NUM_BANKS times) **Status:** Draft v0.1

> Architectural context: HAS §3.3. The FSM state diagram is in ddr2_lpddr2_has/assets/mermaid/03_bank_machine_fsm.png and is the canonical state reference — this section is the implementation view (port interface, counter pool, broadcast updates, multi-rank instantiation, timing).

14.1 Purpose

bank_machine_fub is the per-(rank, bank) state machine that enforces JEDEC per-bank timing constraints. One FSM instance per (rank, bank) — so a 1-rank 8-bank build has 8 instances, a 2-rank 8-bank build has 16, a 4-rank 8-bank build has 32. Each instance tracks:

- Open-row state and the row identifier
- All per-bank timing counters (tRCD, tRAS, tRP, tRC, tWR, tRFCpb, plus per-RD CL and per-WR CWL windows)
- The "last refreshed" age used by DARP per-bank refresh selection
- The refresh handshake state to refresh_mgr_fub

The instance fans its outputs into:

- A per-(rank, bank) row of the scheduler's bank-state matrix
- The txn_queue's row-hit-cache refresh broadcast (open_row_changed[r][b] + new_open_row[r][b])
- The refresh manager's refresh_gnt[r][b] array and the last_ref_age[r][b] DARP selection input

The bank machine is the most-replicated FUB in the design. Per-instance area is small (~150–300 LUTs, see below), but the replication factor (NR × NB, up to 32) makes total bank-machine area the second-largest after the scheduler.

14.2 Synthesis Parameters

Parameter	Source	Effect on this FUB
ROW_WIDTH	top	Width of open_row register and new_open_row broadcast
COL_WIDTH	top	Width of column passthrough (consumed by cmd_encoder, not stored)
MEMTYPE	top	Controls whether the REFRESHING state uses tRFCab or tRFCpb (LPDDR2 only has REFpb)
T_*_WIDTH_MAX	derived	Width of each timing-counter register (set by max CSR-loaded tREFI, tRC, tRFCpb, etc.)
LAST_REF_AGE_WIDTH	derived	Width of last_ref_age counter; sized to span 8 × tREFI_cycles without saturation

The bank machine takes **no rank/bank identifier as a parameter** — every instance is identical. Identity comes from the position in the top-level generate block and is passed in via wire-level bank_id_i and rank_id_i ports (used only for assertion messages and CSR routing; the FSM logic is identity-agnostic).

14.3 Instantiation Pattern

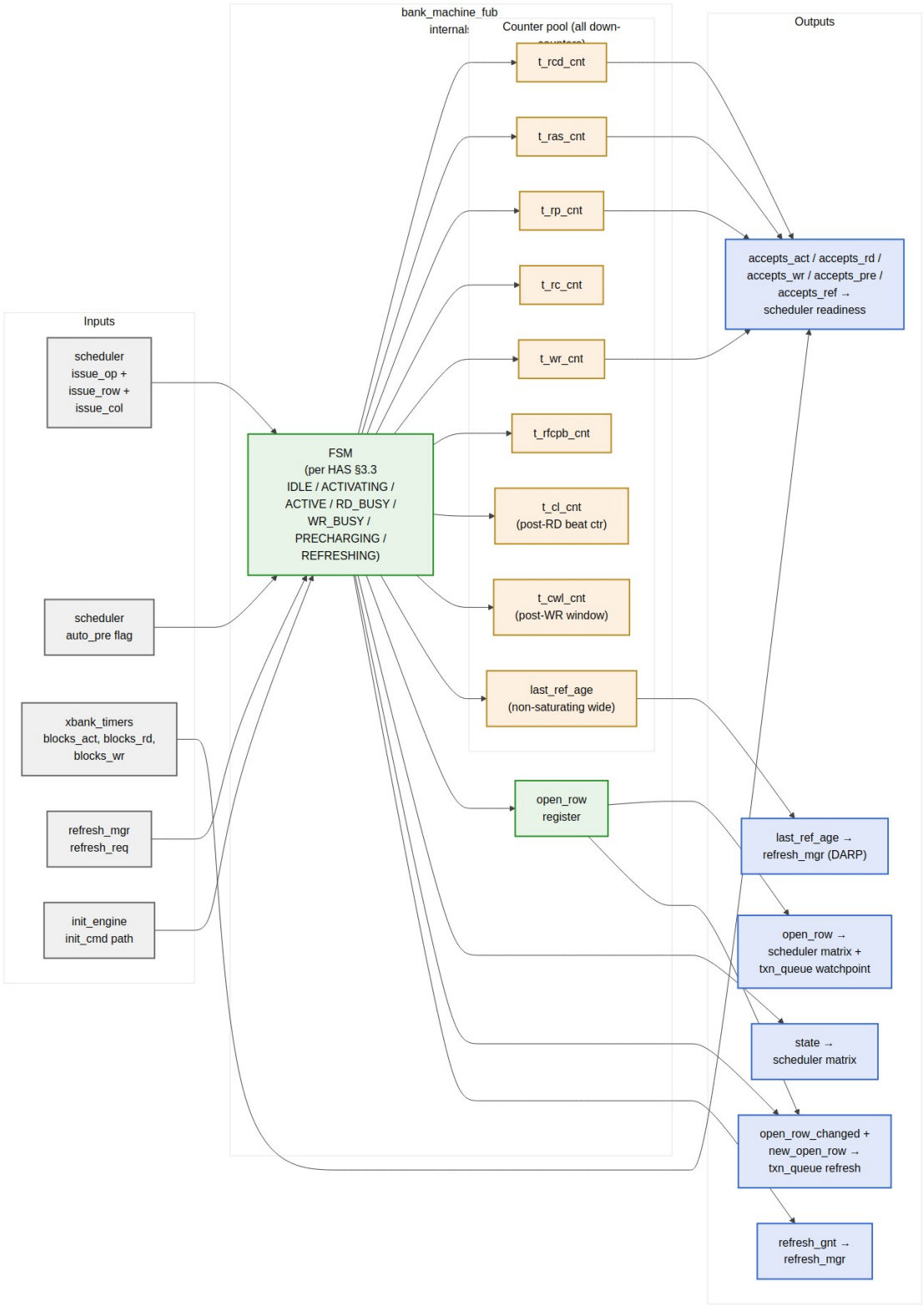
Recap from §2.1 top-integration:

```
generate
    for (genvar r = 0; r < NUM_RANKS; r++) begin : g_rank
        for (genvar b = 0; b < NUM_BANKS; b++) begin : g_bank
            bank_machine_fub #(.ROW_WIDTH(ROW_WIDTH)) u_bank_machine (
                .mc_clk             (mc_clk),
                .mc_rst_n           (mc_rst_n),
                .rank_id_i          (r[$clog2(NR)-1:0]),
                .bank_id_i          (b[$clog2(NB)-1:0]),
                .sched_issue_if     (sched_to_bank[r][b]),
                .xbank_blocks_i     (xbank_to_bank[r][b]),
                .refresh_req_i      (refresh_req[r][b]),
                .refresh_gnt_o      (refresh_gnt[r][b]),
                .init_cmd_if        (init_to_bank[r][b]),
                .state_o            (bank_state[r][b]),
                .open_row_o         (bank_open_row[r][b]),
                .accepts_o          (bank_accepts[r][b]),
                .open_row_changed_o (orw_changed[r][b]),
                .new_open_row_o     (new_orw[r][b]),
                .last_ref_age_o     (bank_last_ref_age[r][b])
```

```
            );
        end
    end
endgenerate
```

The outputs are 2-D-indexed arrays at the top level. The scheduler and txn_queue consume them by name; no aggregation logic in the top — pure structural wiring.

14.4 Block Internals



Bank-Machine Internals — port boundary, counter pool, FSM, broadcasts

Source: 07_bank_machine_internals.mmd

The diagram above is the implementation view. The HAS state-diagram view is at `ddr2_lpddr2_has/assets/mermaid/03_bank_machine_fsm.png` — refer to it for state transitions; this section is the port + counter detail.

14.5 FSM States and Transition Triggers

Recap of the seven states (full state diagram in HAS):

State	Held while	Exits via
`IDLE`	No row open; bank free	ACT or REFpb issued by scheduler
`ACTIVATING`	`t_rcd_cnt > 0`	`t_rcd_cnt == 0` → `ACTIVE`
`ACTIVE`	Row open; awaiting column command or PRE	RD/RDA, WR/WRA, or PRE issued by scheduler
`RD_BUSY`	`t_cl_cnt > 0` (CL + BL/2 beats remaining)	Last beat returned → `ACTIVE` (bare RD) or `PRECHARGING` (RDA)
`WR_BUSY`	`t_cwl_cnt > 0` (CWL + BL/2 + tWR window)	Window expired → `ACTIVE` (bare WR) or `PRECHARGING` (WRA)
`PRECHARGING`	`t_rp_cnt > 0`	`t_rp_cnt == 0` → `IDLE`
`REFRESHING`	`t_rfc_cnt > 0` (tRFCpb for LPDDR2, tRFCab for DDR2 REFab)	`t_rfc_cnt == 0` → `IDLE`

Auto-precharge handling. RDA / WRA do not require a separate state — they are encoded as an "auto-pre pending" flag set when the FSM enters `RD_BUSY` / `WR_BUSY`. When the busy window expires, the flag steers the next-state to `PRECHARGING` instead of `ACTIVE`. No PRE command is consumed from the scheduler — the precharge timing is purely internal. This is the same trick the cmd_encoder uses to fold RDA into one DRAM command.

Init-engine bypass. During init, the `init_cmd_if` path can issue ACT, PRE, MRS, REF directly without going through the scheduler. The FSM honors these as if they came from the scheduler — same state transitions, same counter reloads. The init engine knows the JEDEC sequence; the bank machine just executes it.

14.6 Counter Pool

All counters are **down-counters that saturate at 0**. Each is loaded with a CSR-supplied cycle count on the relevant trigger.

Counter	Load Trigger	Reload Value (cycles)	Saturates At	Width
t_rcd_cnt	ACT issued	TIMINGS_RC_RCD_RP.tRCD	0	8 bits
t_ras_cnt	ACT issued	TIMINGS_RC_RCD_RP.tRAS	0	8 bits
t_rp_cnt	PRE or PREA issued; or RDA/WRA window expiration	TIMINGS_RC_RCD_RP.tRP	0	8 bits
t_rc_cnt	ACT issued	TIMINGS_RC_RCD_RP.tRC	0	8 bits
t_wr_cnt	WR or WRA last beat issued	TIMINGS_CL_CWL_WR.tWR	0	8 bits
t_rfcpb_cnt	REF or REFpb issued	LPDDR2 tRFCpb / DDR2 tRFCab	0	16 bits
t_cl_cnt	RD or RDA issued	TIMINGS_CL_CWL_WR.CL + BL/2	0	8 bits
t_cwl_cnt	WR or WRA issued	TIMINGS_CL_CWL_WR.CWL + BL/2 + tWR	0	8 bits
last_ref_age	REF or REFpb completion (t_rfcpb_cnt → 0)	reload to 0; otherwise +1 per cycle	does not saturate (wide enough for 8 × tREFI)	24 bits

The CSR-supplied values are read from TIMINGS_* registers in csr_apb_fub; the values are live (not staged) since timing parameters are *typically* loaded once at init and not changed at runtime. Software *can* change them at quiet points, but the bank machine doesn't enforce quiet-point staging — it just samples them on each counter load.

`last_ref_age` is non-saturating because DARP needs to distinguish an entry that was refreshed 10 cycles ago from one refreshed 10,000 cycles ago. A 24-bit width supports `2^24 / 200 MHz = 84 ms` of age, well past 8 × tREFI (typically 50 µs).

14.7 Outputs to Scheduler (`accepts_*`)

The `accepts_o` struct is the **combinational readiness output** consumed by the scheduler's Stage-1 readiness computation. Each member is the AND of "FSM state permits this command" and "all relevant counters are 0" and "xbank constraints permit this command":

```
accepts_act = (state == IDLE)
            AND (t_rp_cnt == 0)
            AND (t_rc_cnt == 0)
            AND NOT xbank_blocks_act

accepts_rd  = (state == ACTIVE)
            AND (t_rcd_cnt == 0)
            AND NOT xbank_blocks_rd

accepts_wr  = (state == ACTIVE)
            AND (t_rcd_cnt == 0)
            AND NOT xbank_blocks_wr

accepts_pre = (state == ACTIVE)
            AND (t_ras_cnt == 0)

accepts_ref = (state == IDLE)
            AND (t_rp_cnt == 0)
            AND (t_rc_cnt == 0)
```

These are pure combinational — no flop between the state register and the scheduler. The scheduler's Stage-1 critical path (per §2.7) routes through these comparators.

Why xbank lives outside. Cross-bank constraints (tRRD, tFAW, tCCD, tWTR, tRTW) are shared across all banks of a rank or globally across the channel — they can't be enforced inside a per-bank FSM. The `xbank_timers_fub` (§2.10) aggregates them and presents the per-(rank, bank) AND-mask via `xbank_blocks_i`.

14.8 Refresh Handshake Protocol

The bank machine cooperates with `refresh_mgr_fub` via a two-wire handshake per (rank, bank):

Signal	Direction	Meaning
refresh_req_i	refresh_mgr → bank	"Drain to a quiet state and await my REF / REFpb command"
refresh_gnt_o	bank → refresh_mgr	"I am in IDLE (or REFRESHING) and will not issue any column command"

The bank machine's protocol:

1. **refresh_req asserts** while the bank is in ACTIVE / RD_BUSY / WR_BUSY: the bank does not assert refresh_gnt yet. It finishes its current column command (if any) and falls back to ACTIVE. The scheduler is masked from issuing new column commands to this bank during the refresh window (txn_queue watchpoint — see §2.6 and §2.7).
2. **refresh_req asserts** while the bank is ACTIVE and no column command is in flight: the bank issues an internal precharge (driving accepts_pre high to signal the scheduler can issue PRE; or, if the scheduler is gated for the refresh window, the bank can self-issue PRE on t_ras_cnt == 0). Once state == IDLE and t_rp_cnt == 0, refresh_gnt asserts.
3. **refresh_req asserts** while the bank is in IDLE: refresh_gnt asserts the same cycle.
4. **REF/REFpb arrives** from the scheduler (via the refresh manager's priority path): FSM transitions IDLE → REFRESHING, t_rfcpb_cnt loads, last_ref_age resets to 0.
5. **refresh_req deasserts** after refresh completes: bank machine drops refresh_gnt, transitions REFRESHING → IDLE on t_rfcpb_cnt == 0, and is free to accept new ACTs.

For multi-rank REFab dispatch (per §2.11 refresh_mgr), the refresh manager asserts refresh_req[r][*] for all banks on the target rank simultaneously and waits for all refresh_gnt[r][*] before issuing REF. Other ranks' bank machines see refresh_req[r' ≠ r] = 0 throughout and continue normal operation.

14.9 Open-Row Change Broadcast

When the FSM transitions in a way that changes the open row, the bank machine asserts a one-cycle broadcast to txn_queue_fub:

Transition	open_row_changed_o	new_open_row_o
IDLE → ACTIVATING (via ACT)	1 cycle	The ACT's row
ACTIVE → PRECHARGING → IDLE (after RDA/WRA or explicit PRE)	1 cycle (on entering IDLE)	0 (no row open)
REFRESHING → IDLE	1 cycle	0 (no row open)
ACTIVATING → ACTIVE	not asserted (row was already valid as of ACTIVATING)	—

This is the path that drives `txn_queue`'s `row_hit_cached` refresh, as described in §2.6. The one-cycle lag (broadcast at T, queue refresh visible at T+1) is the deliberate trade described in both this block and the scheduler (§2.7).

`new_open_row_o = 0` when the bank is leaving an open-row state is a convention — the queue's refresh path compares against the new row only when the bank ends in an active state. When ending in IDLE, the queue sets `row_hit_cached = 0` for all entries targeting this (rank, bank) regardless of `new_open_row_o`.

14.10 Per-Instance vs. Aggregated Storage

What lives per-instance (NR × NB copies):

- `state` register (3 bits)
- `open_row` register (ROW_WIDTH bits)
- All counters (~80 bits per instance)
- `auto_pre_pending` flag (1 bit)

What is aggregated at the top level (one copy each):

- `bank_state[NR][NB]` — concatenation of per-instance `state` outputs into a 2-D array
- `bank_accepts[NR][NB]` — concatenation of `accepts` structs
- The DARP candidate-set vector (computed by `refresh_mgr_fub` from the array of `last_ref_age` outputs)
- The scheduler's match vector (computed by the scheduler from `bank_state` and `bank_accepts`)

This separation matters for synthesis: the per-instance flops are local to each bank machine (good for placement); the aggregation buses are wide (NR × NB × ~30 bits = up to ~1000 bits at 4×8) and route across the chip to the scheduler and refresh manager.

Routing concern. At `NUM_RANKS=4`, `NUM_BANKS=8`, the bank-state array routes 32 instances' worth of state and accepts wires to the scheduler. This is a ~1000-bit bus and is the second-widest in the design (after `txn_queue.q_entries_o`). The scheduler's Stage-1 critical path absorbs this in the 200 MHz target with slack; the 500 MHz target requires registered bank-state outputs (which adds one cycle to the scheduler-to-issue latency but is otherwise harmless — bank states change slowly compared to scheduler issue).

14.11 Per-Instance Area Estimate

At default config (ROW_WIDTH=14, COL_WIDTH=10, 8-bit timing counters, 24-bit `last_ref_age`):

Item	Per instance
FSM state (3 bits + 3-bit next)	~10 LUTs
Counter pool (8 down-counters)	~60 flops + 30 LUTs
`open_row` register (14 bits)	14 flops
`last_ref_age` (24 bits)	24 flops
`auto_pre_pending` + combinational `accepts_*` glue	~20 LUTs
Per-instance total	**~120 flops + 60 LUTs ≈ 100–150 LUTs**
32-instance total (4-rank × 8-bank)	**~3,200–4,800 LUTs**

This is ~25% of the controller's total LUT budget at the max-rank config — significant. At single-rank, the bank machines fall to ~800–1,200 LUTs, comfortably in budget.

14.12 CSR Hooks

CSR field	Source signal	Use case
`OBS_ROW_HIT_RANK<R>_BANK<N>` (R)	Driven by scheduler when row-hit picks this bank	Per-bank row-hit telemetry (HAS §6.3)
`OBS_REF_LATENCY_RANK<R>_BANK<N>` (R)	Time from `refresh_req` to `refresh_gnt`	Per-bank refresh-blocking time
`OBS_BANK_OPEN_ROW_R<R>_B<N>` (R)	Current `open_row` value	Debug: which row is currently open

CSR field	Source signal	Use case
OBS_BANK_STATE_R<R>_B<N> (R)	Current FSM state (3-bit encoding)	Debug: bank state for bring-up
STATUS.bank_idle_mask (R)	One bit per (rank, bank): state == IDLE	Quick visibility into bank availability

The per-(rank, bank) observation registers are sparse at multi-rank — at NR=4, NB=8 there are 32 of each. The CSR slave generates them from a YAML template (see §4.2). Software queries the cap_max_ranks and NUM_BANKS capability bits (§4.4) to know which registers exist.

14.13 Verification Notes (cocotb test plan)

Scenario	What it proves
Single ACT → RD → PRE → IDLE on bank 0	Smoke: each counter loads and decrements correctly
ACT → RDA: bank auto-precharges without explicit PRE from scheduler	Auto-precharge flag steers RD_BUSY → PRECHARGING
ACT → WRA: bank auto-precharges; t_wr_cnt window respected	Write recovery before precharge
Scheduler issues RD while t_rcd_cnt > 0	Assertion fires; accepts_rd was 0 — should be impossible
refresh_req asserts during ACTIVATING	refresh_gnt waits until bank reaches IDLE
refresh_req asserts during RD_BUSY	Bank completes RD, returns to ACTIVE, then drains via PRE
refresh_req asserts when bank is IDLE	refresh_gnt asserts same cycle
REF completion → last_ref_age resets to 0	DARP age-tracking correctness
Two adjacent ACTs to different rows on same bank — second blocks on t_rp_cnt	Per-bank tRP enforcement
Cross-bank ACT (handled by xbank_timers) — should NOT affect this bank	Per-bank FSM is rank/bank-local
Multi-rank (NR=2): refresh on rank 0 bank 3; rank 1 bank 3 continues normal ops	Per-rank refresh isolation
Open-row change broadcast: ACT issued, open_row_changed asserts for 1 cycle	Broadcast timing correctness

Scenario	What it proves
Soft reset during `WR_BUSY` — counters clear, state returns to IDLE	Reset behavior

14.14 Open Questions / Future Work

- **CSR coverage of per-bank state.** At NR=4, NB=8 the `OBS_BANK_OPEN_ROW_*` registers consume 32 CSR slots — non-trivial. Worth gating behind a `cap_per_bank_obs` capability bit so smaller builds skip them entirely?
- **Per-instance vs centralized `last_ref_age`.** Could the age counters live in `refresh_mgr_fub` and free up flops per bank machine? Slight area win but adds a wide read port from refresh_mgr to bank_machine for age comparison; punt to v2.
- **Auto-pre flag clear timing.** Currently the `auto_pre_pending` flag clears on entering `PRECHARGING`. If a refresh request arrives mid-RD with auto-pre pending, the bank still does the auto-pre (drains via tRP) before granting refresh. This is correct per JEDEC, but worth a verification scenario (added above).
- **tWR vs tRP composition.** A WRA → PRECHARGING transition needs `max(t_wr_cnt, t_rp_load)` cycles. Implementation: the FSM enters PRECHARGING after `t_wr_cnt` expires and *then* loads `t_rp_cnt`. This is the simplest correct sequence; alternative is to start `t_rp_cnt` early and gate `IDLE` on `max(t_wr_cnt, t_rp_cnt)`. The simpler form is preferred unless characterization shows it costs measurable bandwidth.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

15 Cross-Bank Timers (`xbank_timers_fub`)

Module: `xbank_timers_fub.sv` **Location:** `rtl/fub/` **Category:** FUB **Parent:** `ddr2_lpddr2_ctrl` (one instance) **Status:** Draft v0.1

Architectural context: HAS §3.3 `xbank_timers` and the “Per-Rank vs. Cross-Rank Scope” table added in v0.2. This block-level MAS section is

the implementation view: per-constraint tracker structure, the per-rank vs global scope partition, multi-rank cross-rank gates, and the combinational `blocks_*` outputs.

15.1 Purpose

`xbank_timers_fub` enforces the **cross-bank** JEDEC timing constraints — the ones that span banks of the same rank, or span ranks of the same channel. Per-bank constraints (tRCD, tRAS, tRP, etc.) live in `bank_machine_fub` (§2.9); this FUB owns everything that cannot live in a single bank machine because the relevant resource (the DRAM device's ACT-rate budget, the shared DQ bus, the chip-select setup) is shared.

The FUB produces per-(rank, bank) AND-mask signals (`blocks_act`, `blocks_rd`, `blocks_wr`) that gate each bank machine's `accepts_*` outputs. These masks are the bridge between cross-bank reality and the per-bank state machines' otherwise-local view.

There is **one instance** at the top level — even at multi-rank, the FUB is internally partitioned by scope rather than replicated.

15.2 Constraint Inventory

Per HAS §3.3, the cross-bank constraints partition into three scopes:

Constraint	Scope	Why
`tRRD`	per-rank	Limits ACT-to-ACT stagger inside one DRAM device
`tFAW`	per-rank	Four-Activate-Window — device-local thermal/power limit
`tCCD`	global (channel)	Column-to-column on shared DQ bus
`tWTR`	global (channel)	Write-data → read-data turnaround on shared DQ bus
`tRTW`	global (channel)	Read-data → write-data turnaround on shared DQ bus
`tRTRS`	cross-rank only (NR > 1)	Rank-to-rank DQ-bus handoff on consecutive reads to different ranks
`tCS`	cross-rank only (NR > 1)	Chip-select setup when issuing a command to a different rank than the last

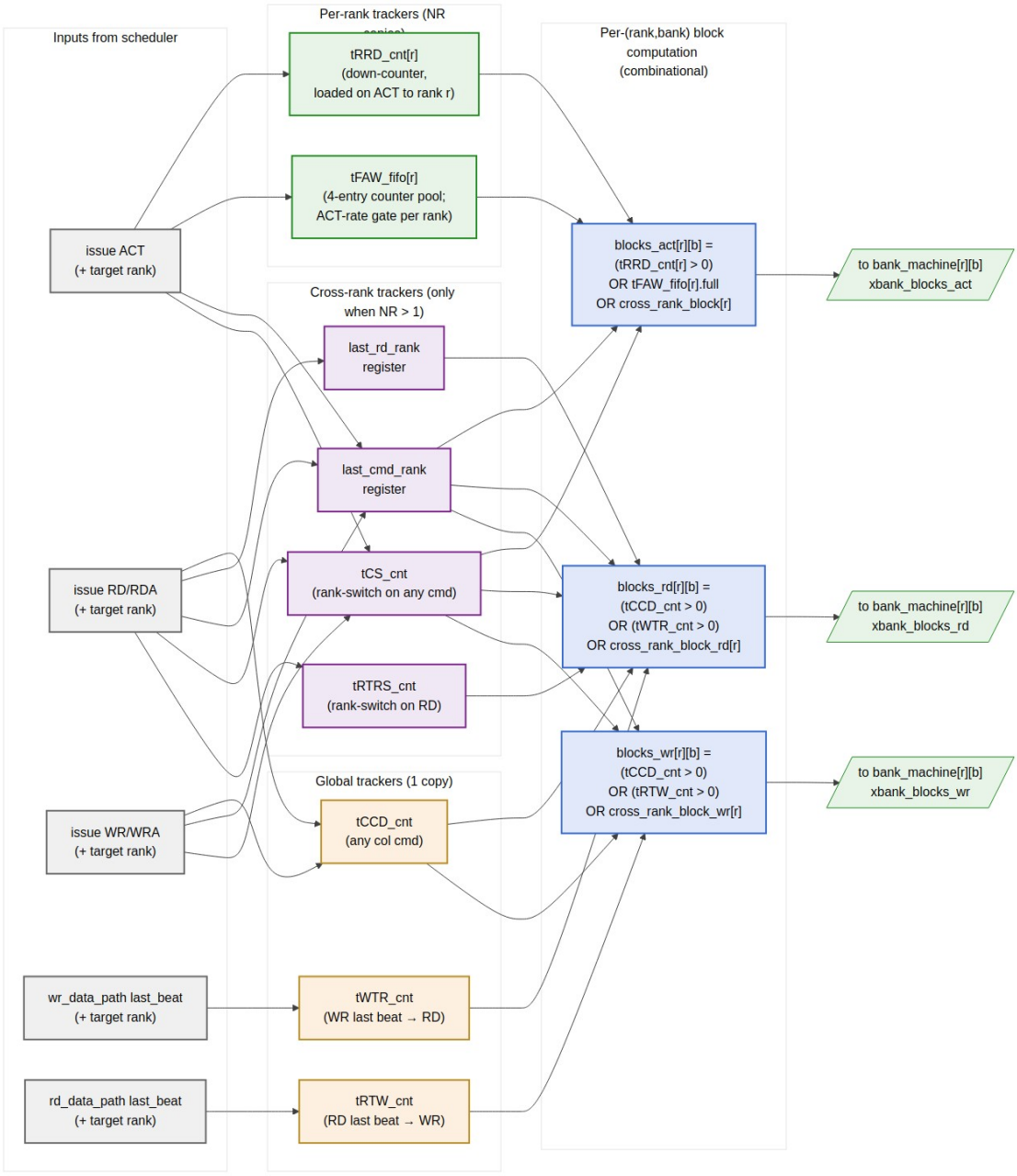
For NUM_RANKS == 1 builds, the cross-rank section is fully omitted at elaboration — the tRTRS_cnt and tCS_cnt flops are not synthesized and the last_rd_rank / last_cmd_rank registers tie off. The other tracker categories are present in every build.

15.3 Synthesis Parameters

Parameter	Source	Effect
NUM_RANKS	top	Per-rank tracker fanout; enables cross-rank gates when > 1
NUM_BANKS	top	Output fanout: blocks_*[NR][NB]
T_RRD_WIDTH	derived	Width of tRRD_cnt (sized to span max CSR-loaded tRRD value)
T_FAW_WIDTH	derived	Width of each tFAW_fifo counter
T_CCD_WIDTH	derived	Width of global tCCD_cnt
T_WTR_WIDTH	derived	Width of global tWTR_cnt
T_RTW_WIDTH	derived	Width of global tRTW_cnt
T_RTRS_WIDTH	derived (NR>1)	Width of tRTRS_cnt (typically 2-bit for DDR2/3 1-cycle tRTRS)
T_CS_WIDTH	derived (NR>1)	Width of tCS_cnt (typically 1-cycle on Artix-class FPGA targets)

The T_*_WIDTH parameters are computed at elaboration from the maximum CSR-loadable timing value. CSR values themselves are live (loaded once at init) and feed the counter-load values directly.

15.4 Block Internals



xbank_timers Constraint Tracker Pool

Source: 08_xbank_timers_scope.mmd

The diagram shows three tracker pools (per-rank, global, cross-rank), the scheduler-issue inputs that load them, and the per-(rank, bank) `blocks_*` combinational outputs.

15.5 Per-Rank Trackers

15.5.1 tRRD_cnt[NR]

One down-counter per rank.

- **Load**: `issue_op_o == ACT` on rank `r` → `tRRD_cnt[r] = TIMINGS_RRD_FAW_WTR.tRRD`
- **Decrement**: `tRRD_cnt[r] > 0` decrements every cycle
- **Effect**: contributes to `blocks_act[r][*]` (gates *all banks* of rank `r`, since tRRD spans banks within the rank)
- **No cross-rank interference**: an ACT on rank 0 does not affect `tRRD_cnt[1]`

At the default tRRD ~5 cycles (DDR2-800), the counter is 3 bits. At 200 MHz the load-to-zero latency is 5 cycles — well within the scheduler's per-cycle reckoning.

15.5.2 tFAW_fifo[NR]

Four-Activate-Window per rank. Implementation is a 4-entry counter pool (not a true FIFO of timestamps):

```
struct tFAW_slot_t {
    logic [T_FAW_WIDTH-1:0] cnt;   // down-counter; 0 means "this slot
is free"
};

tFAW_slot_t tFAW_fifo[NUM_RANKS][4];

// Per cycle, per slot:
//   if (cnt > 0) cnt = cnt - 1;

// On ACT issue to rank r:
//   pick the slot in tFAW_fifo[r] with the smallest cnt (the next-to-
evict slot;
//   one of them is guaranteed to be 0 if blocks_act_faw is currently
low)
//   load that slot's cnt = TIMINGS_RRD_FAW_WTR.tFAW

// blocks_act_faw[r] = AND over all 4 slots: (cnt > 0)
```

The "pick smallest" load policy is equivalent to "replace the oldest entry," which is correct for a rolling-window tFAW check. The check `AND over 4 slots cnt > 0` is "all 4 most-recent ACTs are still within tFAW" — block.

Why not a literal FIFO of timestamps. A timestamp FIFO requires subtraction at check time (`now - oldest_ts > tFAW`?) and a wide subtractor. The counter-pool approach is a 4-entry comparison-to-zero, which is one LUT layer.

At default tFAW ~32 cycles (DDR2-800), each slot's counter is 5 bits. Total `tFAW_fifo` storage per rank: 4 × 5 = 20 flops. Across 4 ranks: 80 flops. Trivial.

15.6 Global Trackers (channel-wide)

15.6.1 `tCCD_cnt`

Single down-counter. Tracks the minimum gap between *any* column command (RD, RDA, WR, WRA) and the *next* column command anywhere on the channel.

- **Load**: any column command issue → `tCCD_cnt = TIMINGS_RRD_FAW_WTR.tCCD`
- **Effect**: contributes to both `blocks_rd[*][*]` and `blocks_wr[*][*]`

At DDR2-800 with BL=4, tCCD = 2 cycles. The counter is 2 bits.

15.6.2 `tWTR_cnt`

Single down-counter. Tracks the minimum gap between the **last write data beat** completing and the **next read** issuing.

- **Load**: `wr_data_path_fub` strobes `wr_last_beat` → `tWTR_cnt = TIMINGS_RRD_FAW_WTR.tWTR`
- **Effect**: contributes to `blocks_rd[*][*]`
- **Subtle point**: loaded from the data-path event, *not* from the WR command issue. This is because tWTR is "last write data on DQ → first read CAS," not "WR command → RD command." The two are different by CWL + BL/2 cycles.

15.6.3 `tRTW_cnt`

Symmetric to tWTR. Tracks the minimum gap between the **last read data beat** completing and the **next write** issuing.

- **Load**: `rd_data_path_fub` strobes `rd_last_beat` → `tRTW_cnt = TIMINGS_CL_CWL_WR.tRTW` (CSR field added in v0.2)
- **Effect**: contributes to `blocks_wr[*][*]`

At DDR2-800 tRTW is typically 0 cycles, but the counter is still synthesized for parameterization and verification consistency. At DDR3+ tRTW becomes a relevant constraint; we want the FUB to be ready.

15.7 Cross-Rank Trackers (only when NUM_RANKS > 1)

These trackers synthesize only when `NUM_RANKS > 1` — at single-rank, they tie off and consume no flops.

15.7.1 `last_rd_rank` register

`$clog2(NR)`-bit register holding the rank of the most recent RD/RDA issue. Used to detect rank-switching for the tRTRS gate.

15.7.2 `tRTRS_cnt`

Single down-counter. Tracks the rank-to-rank read switching delay. DDR2 spec: 1 cycle when consecutive RDs target different ranks; 0 when same rank.

- **Load**: RD/RDA issued to rank `r`, and `r != last_rd_rank` → `tRTRS_cnt = TIMINGS_RTRS.tRTRS`
- **Effect**: contributes to `blocks_rd[r'][*]` *for* *`r' != last_rd_rank`* (block reads to a different rank while the bus handoff window is open)
- **Reset rule**: when an intervening WR or PRE breaks the read chain, `tRTRS_cnt` does not need to fire on the next RD. Implemented by also clearing on any non-RD issue.

15.7.3 `last_cmd_rank` register and `tCS_cnt`

Similar pair for the chip-select setup constraint. When any new command targets a rank different from the previous command, tCS-cycles must pass before the new `CS_n[r']` can drive.

- **Load**: any command issue where target rank `r != last_cmd_rank` → `tCS_cnt = TIMINGS_CS.tCS`
- **Effect**: contributes to `blocks_act[r'][*]`, `blocks_rd[r'][*]`, `blocks_wr[r'][*]` *for* *`r' != last_cmd_rank`*

On Artix-class FPGAs targeting DDR2-800, `tCS` is typically met by IOB setup margin and the counter loads with 0 cycles. The flop and gate exist for parameter-driven verification and for stricter-margin platforms.

15.8 Combinational blocks_* Outputs

The final per-(rank, bank) outputs are pure ORs of the relevant tracker bits:

```
// Common cross-rank block gates (only when NR > 1; else 0)
cross_rank_block[r] = (NR > 1)
                      AND (tCS_cnt > 0)
                      AND (r != last_cmd_rank)

cross_rank_block_rd[r] = cross_rank_block[r]
                       OR ( (NR > 1)
                            AND (tRTRS_cnt > 0)
                            AND (r != last_rd_rank) )

cross_rank_block_wr[r] = cross_rank_block[r]    // no tRTW-rank-switch
gate in v1

// Per-(rank, bank) outputs (same for all banks of a rank)
blocks_act[r][b] = (tRRD_cnt[r] > 0)
                OR (tFAW_fifo[r].all_busy)
                OR cross_rank_block[r]

blocks_rd[r][b] = (tCCD_cnt > 0)
               OR (tWTR_cnt > 0)
               OR cross_rank_block_rd[r]

blocks_wr[r][b] = (tCCD_cnt > 0)
               OR (tRTW_cnt > 0)
               OR cross_rank_block_wr[r]
```

Note that `blocks_*[r][b]` does not depend on `b` — every bank within a rank sees the same xbank gate. The 2-D `blocks_*` output is a fanout convenience for the top-level wiring; the underlying logic is `NR`-wide, then broadcast to `NB` banks.

This is a deliberate fanout: the bank machine port (§2.9) accepts `xbank_blocks_i` as a per-(rank, bank) struct because the wider port is symmetric with the bank machine's other 2-D-indexed signals. The cost is one extra wire-fanout layer at the top; the benefit is the bank-machine module declaration doesn't have to be parameterized differently.

15.9 Interface

15.9.1 Inputs from Scheduler

Signal	Direction	Width	Description
issue_op_i	input	4	Same encoding as scheduler_fub.issue_op_o
issue_rank_i	input	$clog2(NR)	Rank being issued to
issue_strobe_i	input	1	(issue_op_i != NOP) && issue_valid

15.9.2 Inputs from Data Paths

Signal	Direction	Width	Description
wr_last_beat_i	input	1	Last beat of a WR burst pushed to DFI wrdata
wr_last_rank_i	input	$clog2(NR)	Rank of that burst
rd_last_beat_i	input	1	Last beat of an RD burst returned from DFI rddata
rd_last_rank_i	input	$clog2(NR)	Rank of that burst

15.9.3 Inputs from CSR (live)

Signal	Direction	Width	Source
t_rrd_i	input	width	TIMINGS_RRD_FAW_WTR.tRRD
t_faw_i	input	width	TIMINGS_RRD_FAW_WTR.tFAW
t_ccd_i	input	width	TIMINGS_RRD_FAW_WTR.tCCD
t_wtr_i	input	width	TIMINGS_RRD_FAW_WTR.tWTR
t_rtw_i	input	width	TIMINGS_CL_CWL_WR.tRTW
t_rtrs_i	input	4	TIMINGS_XRANK.tRTRS (only when NR>1)
t_cs_i	input	4	TIMINGS_XRANK.tCS (only when NR>1)

15.9.4 Outputs to Bank Machines

Signal	Direction	Width	Description
`blocks_act_o[NR][NB]`	output	NR×NB	Per-(rank, bank) ACT gate
`blocks_rd_o[NR][NB]`	output	NR×NB	Per-(rank, bank) RD gate
`blocks_wr_o[NR][NB]`	output	NR×NB	Per-(rank, bank) WR gate

15.9.5 Debug Outputs

Signal	Description
`dbg_t_rrd_obs_o[NR]`	Live `tRRD_cnt[r]` values for waveform-debug
`dbg_tfaw_busy_count_o[NR]`	Per-rank count of non-zero tFAW slots
`dbg_xrank_block_active_o`	One-bit: any cross-rank gate currently asserted

15.10 Timing Budget

The path from scheduler issue → `blocks_*` update is single-cycle:

```
cycle T:   scheduler asserts issue_strobe_i with issue_op_i = ACT,
issue_rank_i = r
cycle T:   load tRRD_cnt[r], push tFAW_fifo[r], load tCS_cnt (if rank-
switched)
cycle T+1: tRRD_cnt[r] now > 0, blocks_act_o[r][*] = 1
```

The `blocks_*_o` outputs are combinational from the registers; the scheduler's Stage-1 path consumes them in the *same* cycle they update. So the scheduler always sees the post-issue gate state on the next cycle — no race where the scheduler picks two same-rank ACTs back-to-back.

For 500 MHz targets, the per-(rank, bank) fan-out path (NR × NB destinations) may need registered outputs. The 200 MHz default budget is comfortable.

Per-bank fan-out concern. At NR=4, NB=8, each `blocks_*` signal fans out 32 ways. Synthesis will buffer this automatically; no manual replication needed in RTL.

15.11 CSR Hooks

CSR field	Source	Use case
`STATUS.xbank_blocki`	Rolling count of cycles	Bandwidth-headroom

CSR field	Source	Use case
ng_pct (R)	where ANY blocks_* is asserted	telemetry
OBS_TFAW_HITS_RANK<R> (R)	Incremented when ACT was blocked by tFAW for rank r	Characterization: tFAW pressure
OBS_TWTR_HITS (R)	Incremented when RD was blocked by tWTR	DQ-bus turnaround pressure
OBS_XRANK_SWITCHES (R)	Incremented on every last_cmd_rank change (NR>1)	Rank-locality telemetry

The cross-rank counters are only present at NUM_RANKS > 1 builds; at single-rank they tie to zero and read 0.

15.12 Verification Notes (cocotb test plan)

Scenario	What it proves
Two ACTs to different banks of same rank, gap = tRRD - 1 cycle	Second ACT blocks; blocks_act[r][*] was high
Two ACTs to different banks of same rank, gap = tRRD cycles	Second ACT issues at exactly tRRD
Five ACTs to rank 0 within tFAW window	Fifth blocks; tFAW gate engages
Five ACTs spaced across tFAW boundary (4 in + 1 out)	Fifth issues; tFAW slot 1 has aged out
Multi-rank: ACTs on rank 0 do NOT block ACTs on rank 1	Per-rank tRRD / tFAW isolation
Multi-rank: RD to rank 0 then RD to rank 1, gap < tRTRS	Second blocks; blocks_rd[1][*] was high
Multi-rank: RD to rank 0, then PRE to rank 0, then RD to rank 1	Intervening PRE clears tRTRS gate
WR last beat → RD before tWTR elapsed	RD blocks; tWTR gate engaged
RD last beat → WR before tRTW elapsed (DDR3+ scenario)	WR blocks; tRTW gate engaged

Scenario	What it proves
Multi-rank with tCS > 0: cross-rank ACT blocks for tCS cycles	tCS gate works
`NUM_RANKS = 1` build: cross-rank gates synthesize to 0	Synthesis sanity
Heavy multi-rank traffic: telemetry counter increments correctly	CSR observability

15.13 Open Questions / Future Work

- **tRTW cross-rank rank-switch gate.** Currently `cross_rank_block_wr` only depends on tCS (any rank switch), not on a separate "tRTRS-equivalent for writes." DDR4 introduced this; for DDR2/3 it's typically subsumed by tCS + tRTW. Worth a verification scenario at DDR3+ to confirm.
- **tFAW slot-selection policy.** "Pick smallest cnt" works correctly but doesn't necessarily match the literal "evict oldest by arrival." For a 4-entry pool with same-cycle arrivals, the two are equivalent. If multiple ACTs hit in the same cycle (single-issue scheduler means this doesn't currently happen), behavior would need re-spec.
- **Per-rank tCCD?** DDR4 introduced bank groups, with same-group tCCD_L being longer than different-group tCCD_S. For DDR2/LPDDR2 this is not relevant (no bank groups). The DDR3+ family controllers will add a per-bank-group tCCD tracker.
- **Counter-shift vs flop-pool for tFAW.** An alternative implementation is a single shift register that ratchets a tFAW-cycles delay line. Saves one comparator but consumes the same flops. Not worth changing in v1; revisit if Synth flags the comparator chain.

16 Refresh Manager (`refresh_mgr_fub`)

Module: `refresh_mgr_fub.sv` **Location:** `rtl/fub/` **Category:** FUB **Parent:** `ddr2_lpddr2_ctrl`
Status: Draft v0.1

> Architectural context: HAS §3.4 (postponer, REFab/REFpb paths, DARP selection, ZQCS piggyback, per-rank PASR, multi-rank round-robin). The HAS state diagram for the controller-level refresh FSM is in `ddr2_lpddr2_has/assets/mermaid/11_refresh_controller_fsm.png` — refer to that for the state machine; this section is the implementation view (counters, DARP selector, per-rank handshake fan-out, sequencer, timing).

16.1 Purpose

`refresh_mgr_fub` owns DRAM refresh scheduling and the ZQCS-piggyback path. It produces:

- The binary `refresh_wants_o` signal that pauses scheduler issue at the right cycles
- Per-(rank, bank) `refresh_req_o[NR][NB]` handshake assertions to bank machines
- The actual REF / REFpb command issue (routed through the scheduler's pipeline)
- ZQCS commands during the same refresh window when the periodic ZQCS timer fires
- PASR mask propagation to the DRAM via MR16/MR17 writes (per-rank for LPDDR2)

This is the heart of the controller's correctness story: get refresh wrong and DRAM forgets data. The FUB is built around a deterministic postponer (no saturating-pending counter — earlier revisions saturated, which was out-of-JEDEC; v0.2 fixed this) and a per-rank handshake mesh.

For multi-rank builds, the FUB drives REFab in a round-robin across ranks so non-targeted ranks keep servicing column commands during the refresh window — see §3.4 in the HAS for the rationale.

16.2 Synthesis Parameters

Parameter	Source	Effect
NUM_RANKS	top	Per-rank PASR masks, per-rank refresh_req fan-out, round-robin pointer width
NUM_BANKS	top	DARP candidate matrix width
REFPB_POLICY	top	ROUND_ROBIN, OLDEST_FIRST, DARP — picks the synthesized REFpb selector tree
REFRESH_DEFER_MAX	top (default 8)	Postponer batch depth and refresh_owed counter width
MEMTYPE	top	Picks REFab vs REFpb default; LPDDR2 uses REFpb, DDR2 uses REFab
T_REFI_WIDTH	derived	Width of t_refi_cnt
T_RFC_WIDTH	derived	Width of t_rfcpb_cnt / t_rfcab_cnt
T_ZQCS_WIDTH	derived	Width of the ZQCS interval counter

16.3 Top-Level FSM

Six-state FSM mirroring HAS §3.4. The state register is the only flop on the priority path.

State	Held while	Exits via
IDLE	wants_refresh pulse not asserted; ZQCS not due	wants_refresh → WAIT_BANK_GNTS
WAIT_BANK_GNTS	Asserting refresh_req to selected banks; awaiting refresh_gnt	All required refresh_gnt[r][b] high → DO_REFRESH
DO_REFRESH	Refresh sequencer issuing REF/REFpb back-to-back, draining refresh_owed	refresh_owed == 0 → CHECK_ZQCS
CHECK_ZQCS	One-cycle check whether wants_zqcs is asserted	wants_zqcs == 1 → DO_ZQCS, else → RELEASE
DO_ZQCS	Running ZQCS sequence (PRE_ALL → tRP → ZQCS → wait tZQCS)	tZQCS expired → RELEASE
RELEASE	Deasserting refresh_req; one cycle for bank machines to drop grants	All refresh_gnt low → IDLE

The FSM advances at most one state per MC clock; full `WAIT → DO → CHECK → ZQCS → RELEASE → IDLE` traversal of a non-batched single-refresh-with-ZQCS takes ~(tRFCpb + tZQCS + 4) cycles.

16.4 Counter Inventory

Counter	Width	Loaded on / Behavior
`t_refi_cnt`	`T_REFI_WIDTH`	Down-counter from `TIMINGS_RAS_RFC_REFI.tREFI` (or scaled by `tREFI_scale` for LPDDR2 temperature derating); reloads on hitting 0
`postpone_cnt`	`$clog2(REFRESH_DEFER_MAX)`	Down-counter from `REFRESH_DEFER_MAX − 1` to 0; decrements on each `t_refi_cnt` zero; reloads on emitting `wants_refresh`
`refresh_owed`	`$clog2(REFRESH_DEFER_MAX × NR) + 1`	**Non-saturating** counter of refreshes still to issue; +REFRESH_DEFER_MAX when postponer fires; decrements per refresh issued
`t_zqcs_cnt`	`T_ZQCS_WIDTH`	Down-counter for periodic ZQCS interval; reload from `REFRESH_TUNING.zqcs_freq_hz` (live CSR)
`tRFC_inflight`	`T_RFC_WIDTH`	Down-counter inside the sequencer for the *currently-issued* REF/REFpb; gates the next back-to-back issue

The `refresh_owed` counter is **non-saturating** because JEDEC requires the controller to honor every missed refresh. A saturating counter would silently drop deadlines if the workload kept the bus blocked for more than `REFRESH_DEFER_MAX × tREFI`. v0.1 HAS used a saturating counter; v0.2 changed it to the deterministic postponer-based non-saturating counter described here.

The width sizing is generous: `clog2(REFRESH_DEFER_MAX × NR) + 1` covers the worst case where the postponer just fired AND every rank has a queued batch waiting.

16.5 Postponer Logic

The postponer is what aggregates `REFRESH_DEFER_MAX` worth of tREFI deadlines into a single back-to-back refresh batch. This trades worst-case refresh latency for sustained bandwidth (fewer scheduler interrupts).

```
// Per cycle:
if (t_refi_cnt > 0):
    t_refi_cnt = t_refi_cnt - 1
else:
    // tREFI fired
    t_refi_cnt = tREFI_cycles    // reload
    if (postpone_cnt > 0):
        postpone_cnt = postpone_cnt - 1
        // (no refresh emitted yet; defer)
    else:
        // batch complete
        emit wants_refresh        // one-cycle pulse to FSM
        postpone_cnt = REFRESH_DEFER_MAX - 1
        refresh_owed = refresh_owed + REFRESH_DEFER_MAX
```

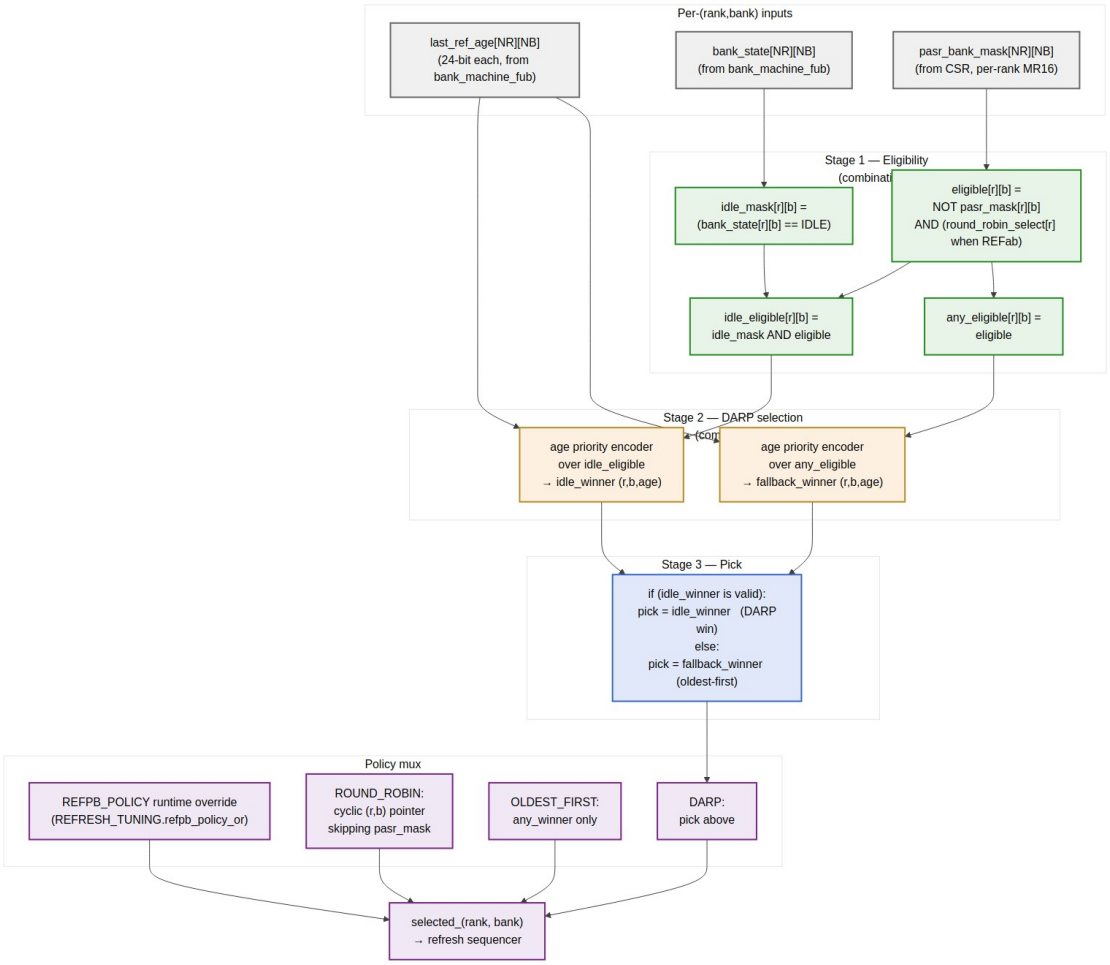
The `wants_refresh` pulse is captured by the FSM's `IDLE → WAIT_BANK_GNTS` transition. Once captured, the FSM stays out of IDLE until `refresh_owed == 0` (sequencer drains the batch).

Setting `REFRESH_DEFER_MAX = 1` disables batching — every tREFI fires one refresh, equivalent to the no-postponer behavior. Setting `REFRESH_DEFER_MAX = 8` (default at JEDEC's ceiling) gathers up to 8 deadlines before firing the back-to-back batch.

LPDDR2 temperature scaling: the reload value for `t_refi_cnt` is `tREFI_cycles / tREFI_scale`, where `tREFI_scale ∈ {1, 2, 4}` from MR4 temperature class (1x at nominal, 2x at high-temp, 4x at very-high-temp). The scale is a live CSR read; software writes it after the SoC has polled MR4 via the LPDDR2 PHY.

16.6 DARP Selection (REFpb Path)

For REFpb, the FUB selects one (rank, bank) tuple to refresh each iteration through the batch drain. The DARP policy (HAS §3.4) is "max age among idle non-PASR-masked tuples; if none idle, fall back to oldest-first."



DARP Selector — eligibility, dual-priority encoder, fallback mux

Source: 10_refresh_darp_selector.mmd

16.6.1 Stage 1 — Eligibility (combinational, NR × NB parallel)

For each (r, b):

```
eligible[r][b]      = NOT pasr_bank_mask[r][b]
idle_mask[r][b]     = (bank_state[r][b] == IDLE)
idle_eligible[r][b] = idle_mask[r][b] AND eligible[r][b]
any_eligible[r][b]  = eligible[r][b]
```

PASR masking is per-rank (HAS §3.4 v0.2 change) — masked banks of one rank don't suppress the same bank on another rank.

16.6.2 Stage 2 — Dual Priority Encoders

Two parallel age-priority encoders over `last_ref_age[NR][NB]`:

```
idle_winner     = age_pe(idle_eligible)    // (r,b) with max age, or
NONE
fallback_winner = age_pe(any_eligible)     // (r,b) with max age
across all eligible
```

For NR=4, NB=8 = 32 candidates, the encoder is a 32-entry tournament — 5 levels of pairwise compare-and-select on 24-bit ages. ~50 LUT-delays at FPGA target — slower than the scheduler's 16-entry tournament (§2.7) but only matters once per refresh batch issue, not every cycle.

16.6.3 Stage 3 — DARP Pick

```
if (idle_winner is valid):
    pick = idle_winner          // DARP win
else:
    pick = fallback_winner      // fall back to oldest-first
```

16.6.4 Policy Mux

`REFRESH_TUNING.refpb_policy_or` overrides the synthesized default. The three policy branches are computed in parallel:

- `ROUND_ROBIN`: cyclic `(r, b)` pointer that skips PASR-masked tuples
- `OLDEST_FIRST`: just `fallback_winner` (skip the DARP idle-test)
- `DARP`: the Stage-3 pick

A 3:1 mux selects the active policy. Un-synthesized policies tie off and the mux returns `pslverr` on CSR write of an unavailable choice (per HAS §6.3 quiet-point semantics).

16.6.5 Round-Robin Pointer

For `ROUND_ROBIN` policy and for REFab (per-rank round-robin), the FUB maintains a `round_robin_(rank, bank)` pointer that increments past PASR-masked tuples after each refresh issue. For REFab, the bank field is ignored and only the rank pointer advances.

16.7 REFab Path (Multi-Rank Round-Robin)

When `MEMTYPE == "DDR2"` or LPDDR2 falls back to REFab, the FUB drives REF with per-rank dispatch. The HAS §3.4 specifies why: driving REF with all `CS_n[*] = 0` would freeze every rank simultaneously, wasting bandwidth on ranks not due. The round-robin keeps the non-target ranks available for column commands.

Sequence per REFab iteration (one drain step):

1. `target_rank = round_robin_rank` (advanced post-issue)

2. Assert `refresh_req[target_rank][*]` for all banks on `target_rank` (none for other ranks)
3. Wait for `AND(refresh_gnt[target_rank][b] for b in 0..NB-1)`
4. Issue REF via the scheduler's refresh-priority path; `cmd_encoder` drives `dfi_cs_n[target_rank] = 0`, others = 1
5. Load `tRFC_inflight = tRFCab_cycles` (DDR2 REFab) or `tRFCab_cycles` (LPDDR2 fallback REFab)
6. Wait for `tRFC_inflight == 0`
7. `refresh_owed = refresh_owed - 1`; advance `round_robin_rank = (round_robin_rank + 1) % NR`
8. If `refresh_owed > 0`, repeat from step 1 with the new `target_rank`

For `NUM_RANKS == 1`, the round-robin pointer is a tied 0 and the path collapses to single-rank REFab.

16.8 REFpb Path (LPDDR2 Default)

REFpb operates on a (rank, bank) tuple — only that one bank on that one rank enters REFRESHING. All other banks (on the same rank and other ranks) keep operating.

Sequence per REFpb iteration:

1. DARP / policy selector picks `target_(rank, bank)` (above)
2. Assert `refresh_req[target_rank][target_bank]` (single bit)
3. Wait for `refresh_gnt[target_rank][target_bank]`
4. Issue REFpb via scheduler; `cmd_encoder` drives `dfi_cs_n[target_rank] = 0`, embeds the bank number in the LPDDR2 CA bus
5. Load `tRFC_inflight = tRFCpb_cycles`
6. Wait for `tRFC_inflight == 0`
7. `refresh_owed = refresh_owed - 1`
8. If `refresh_owed > 0`, repeat from step 1 with a fresh DARP selection (`last_ref_age` for the just-refreshed tuple resets to 0, so it won't immediately win again)

16.9 Periodic ZQCS Piggyback

Periodic ZQCS is implemented as a side-channel on the refresh window. `t_zqcs_cnt` runs in parallel with the postponer:

```
if (t_zqcs_cnt > 0):
    t_zqcs_cnt = t_zqcs_cnt - 1
else:
    t_zqcs_cnt = zqcs_interval_cycles     // reload from
REFRESH_TUNING.zqcs_freq_hz
    wants_zqcs = 1                        // sticky until consumed by
FSM
```

The FSM checks `wants_zqcs` in the `CHECK_ZQCS` state after a refresh batch drains. If asserted, the FSM enters `DO_ZQCS` and runs the ZQCS sequence:

1. Issue PRE_ALL (idempotent on the just-refreshed ranks; required on ranks that may have stayed ACTIVE during partial refresh)
2. Wait tRP
3. Issue ZQCS command
4. Wait tZQCS (~64–256 cycles)
5. Clear `wants_zqcs`; transition to `RELEASE`

This piggybacks on the "all banks idle" guarantee that the refresh window already provides — no separate bus-blocking event is needed for ZQCS. The cost is a small latency at the end of refresh batches (only when ZQCS is actually due).

Setting `REFRESH_TUNING.zqcs_freq_hz = 0` disables periodic ZQCS (init-only mode).

16.10 Per-Rank PASR Handling

PASR (Partial Array Self-Refresh) is an LPDDR2 feature that lets the SoC mask off DRAM regions that hold no data, saving refresh power. The mask is per-rank because each rank has its own MR16/MR17.

- `pasr_bank_mask[NR]` — `NUM_BANKS`-bit per-rank register, written by SoC via `PASR_BANK_MASK_RANK<R>` CSR
- `pasr_seg_mask[NR]` — segment mask, written via `PASR_SEG_MASK_RANK<R>`

The FUB uses `pasr_bank_mask[r][b]` to skip masked banks in the DARP selector (Stage 1 eligibility). The segment mask is propagated to DRAM via MR17 writes.

16.10.1 PASR Propagation Protocol

Per HAS §3.4 / Linux PASR Framework reference:

1. SoC writes new mask to CSR
2. Refresh manager treats the staging copy as advisory until the next self-refresh entry OR the next quiet point
3. On self-refresh entry (or quiet point), the new mask is propagated to DRAM via MR16/MR17 writes (one write per rank per register)
4. Mask is now active; subsequent refreshes honor it

The propagation is bundled with self-refresh entry to avoid an extra bus-blocking event during normal operation. If software needs immediate propagation, it asserts `CTRL.config_apply` and the FUB propagates at the next quiet point.

16.11 Interface

16.11.1 Outputs to Scheduler

Signal	Direction	Width	Description
`refresh_wants_o`	output	1	Binary “refresh due now” — pauses scheduler new-issue per §2.7
`refresh_issue_o`	output	struct	When in DO_REFRESH, drives the next REF/REFpb command directly into the scheduler’s command-issue path (scheduler relays to cmd_encoder)

16.11.2 Per-(Rank, Bank) Handshake

Signal	Direction	Width	Description
`refresh_req_o[NR][NB]`	output	NR×NB	Asserted to banks that should drain to IDLE
`refresh_gnt_i[NR][NB]`	input	NR×NB	Bank machines acknowledge “in IDLE, ready”
`bank_state_i[NR][NB]`	input	per BM	Used for DARP idle-mask
`last_ref_age_i[NR][NB]`	input	24-	Used for age-priority encoder

Signal	Direction	Width	Description
		bit each	
refresh_done_for_o[NR][NB]	output	NR×NB	“This bank has been handed back to the scheduler” — drives txn_queue.refresh_done_for_i per §2.6

16.11.3 CSR (live)

Signal	Direction	Width	Source
cfg_refpb_policy_or_i	input	2	REFRESH_TUNING.refpb_policy_or
cfg_refresh_defer_active_i	input	4	REFRESH_TUNING.refresh_defer_active
cfg_zqcs_freq_hz_i	input	16	REFRESH_TUNING.zqcs_freq_hz
cfg_trefi_scale_i[NR]	input	2 × NR	Per-rank LPDDR2 temperature derating from MR4 readback
cfg_pasr_bank_mask_i[NR]	input	NB × NR	Per-rank PASR bank mask
cfg_pasr_seg_mask_i[NR]	input	8 × NR	Per-rank PASR segment mask

16.11.4 Init-Engine Coordination

Signal	Direction	Width	Description
init_in_progress_i	input	1	Disables postponer and FSM during init
init_mrs_passthru_o	output	struct	Forwards MRS writes (MR16/MR17 PASR propagation) to init path

16.11.5 Self-Refresh Coordination (with power_state)

Signal	Direction	Width	Description
sr_entry_req_i	input	1	power_state_fub requests self-refresh entry

Signal	Direction	Width	Description
sr_entry_gnt_o	output	1	refresh_mgr ack — no auto-refresh in progress
sr_active_i	input	1	DRAM is in self-refresh; t_refi_cnt paused
sr_exit_done_i	input	1	DRAM has exited self-refresh; resume t_refi_cnt

16.11.6 Telemetry

Signal	Description
dbg_refresh_owed_o	Current refresh_owed value
dbg_postpone_cnt_o	Current postponer state
dbg_darp_idle_count_o	Number of idle-eligible tuples this cycle
dbg_state_o	Current FSM state (3-bit encoded)
dbg_target_rank_o	(REFab) current round_robin_rank
dbg_pasr_active_o	OR over all PASR masks — “any mask is non-zero”

16.12 Timing Budget

The DARP selector is the slowest path inside this FUB:

Path	Levels (FPGA estimate)	Budget
bank_state[r][b] → idle_eligible[r][b]	2 LUT levels	0.5 ns
last_ref_age[r][b] → age priority encoder (32-entry, 5 levels)	5 × 2 LUT = 10 levels	1.6 ns
Encoder output → 3-way policy mux → target_(rank, bank)	2 LUT levels	0.4 ns
Routing / setup margin		0.3 ns
Total		**2.8 ns**

At 200 MHz (5 ns cycle) this closes with ~2.2 ns slack. At 500 MHz it misses by ~800 ps and needs the encoder pipelined (one register between the dual encoders and the policy mux — adds one cycle to DARP latency, but DARP only fires once per refresh issue, so the throughput impact is zero).

The FSM transitions are all single-cycle. Total refresh-batch latency in cycles is `(REFRESH_DEFER_MAX × max(tRFCab, tRFCpb)) + (zqcs cycles if applicable) + ~5 FSM cycles`. At default config (REFRESH_DEFER_MAX=8, tRFCpb=8 cycles for LPDDR2-1066) this is ~70 cycles per batch fire.

16.13 CSR Hooks

CSR field	Source signal	Use case
`OBS_REFRESH_PENDING_MAX` (R)	Watermark of `refresh_owed`	Detect refresh backpressure
`OBS_REFRESH_DEFER_HIST_0..3` (R)	4-bin histogram of postpone_cnt at fire-time	Characterization: postponer health
`OBS_REF_LATENCY_RANK<R>_BANK<N>` (R)	Time from `refresh_req` to `refresh_gnt` for that bank	Per-bank refresh-blocking distribution
`STATUS.refresh_mgr_state` (R)	`dbg_state_o`	Bring-up debug
`STATUS.refresh_target_rank` (R)	`dbg_target_rank_o`	(NR>1) which rank is currently refreshing
`STATUS.pasr_active` (R)	`dbg_pasr_active_o`	LPDDR2 PASR-active flag

16.14 Verification Notes (cocotb test plan)

Scenario	What it proves
`REFRESH_DEFER_MAX = 1`; every tREFI fires one refresh	Postponer disabled mode
`REFRESH_DEFER_MAX = 8`; postpone 8 deadlines then drain	Batching mode
Bandwidth-starved workload exceeds 8×tREFI; `refresh_owed` grows past 8	Non-saturating counter holds the deadline correctly
DDR2 REFab on single-rank: all banks → REFRESHING; gnt waits then asserts	Single-rank REFab smoke
DDR2 REFab on NR=2: rank 0 refreshes; rank 1 banks continue normal ops	Per-rank REFab dispatch
LPDDR2 REFpb with DARP: pick targets idle	DARP selector idle preference

Scenario	What it proves
bank with max age	
LPDDR2 REFpb with DARP: all banks busy → fall back to oldest-first	DARP fallback path
`REFPB_POLICY = ROUND_ROBIN`; cyclic refresh skipping PASR-masked banks	Round-robin policy
Per-rank PASR: rank 0 bank 3 masked; rank 1 bank 3 not masked	Per-rank PASR isolation
`zqcs_freq_hz = 0`; periodic ZQCS disabled, only init-time ZQCS	ZQCS disable
`zqcs_freq_hz = 1`; ZQCS fires once per second, piggybacked on refresh	Periodic ZQCS
Self-refresh entry: `sr_entry_req` asserts; `sr_entry_gnt` waits for current refresh to complete	Self-refresh handshake
LPDDR2 temperature derate from MR4: tREFI_scale = 2x → t_refi_cnt loads at half value	Temperature-compensated refresh
Multi-rank PASR write via CSR + quiet-point; MR16 propagation visible	PASR update protocol
Refresh during in-flight read on different rank (NR=2): read completes uninterrupted	Per-rank refresh non-interference

16.15 Open Questions / Future Work

- **REFab on multiple ranks in parallel.** Currently REFab dispatches to one rank at a time in round-robin. An alternative is to drive REF on multiple ranks simultaneously (e.g., 2-at-a-time on 4-rank systems) — saves wall-clock refresh time at the cost of more concurrently-blocked ranks. Trade-off depends on whether the workload is bottlenecked by per-rank throughput or by aggregate refresh budget. Punt to v2 with characterization data.
- **DARP look-ahead.** Currently DARP picks the (rank, bank) with max age and idle. A look-ahead variant could weight age by predicted upcoming traffic to that bank — if `txn_queue` has pending column commands queued

for a bank, prefer to refresh a different bank that's about to go idle. Adds a wire from `txn_queue` to the DARP selector; valuable but adds complexity. Not in v1.

- **Postponer adaptive depth.** `REFRESH_DEFER_MAX` is static at elaboration; `refresh_defer_active` is the runtime tunable (up to MAX). Could the active value auto-tune based on observed refresh-pending history? Would close a characterization loop in hardware. Discuss with software team — likely lives in firmware, not RTL.
- **PASR mask write coalescing.** If software writes PASR_BANK_MASK_RANK for all ranks back-to-back, the FUB currently propagates each on its own quiet point. Could coalesce into one MR16/MR17 burst. Minor optimization; only matters if PASR is reconfigured frequently. Punt unless software flags it.

---

17 Init Engine (`init_engine_fub`)

Module: `init_engine_fub.sv` **Location:** `rtl/fub/` **Category:** FUB **Parent:** `ddr2_lpddr2_ctrl` **Status:** Draft v0.1

> Architectural context: HAS §3.5. The HAS flow diagram is in `ddr2_lpddr2_has/assets/mermaid/05_init_engine_flow.png` and is the canonical step-decode reference — this section is the implementation view (ROM organization, step encoding, FSM, bypass path into bank machines, multi-rank MRS loop, sim shortcut).

17.1 Purpose

`init_engine_fub` walks the JEDEC cold-boot DRAM initialization sequence — the deterministic recipe of CKE wiggles, RESET_N pulses, MRS writes, ZQ calibration, and PRE/REF settling cycles that must complete before any normal traffic can issue.

The init sequence is **memtype-specific** (DDR2 vs LPDDR2 differ at MRS layout, ZQ flow, and the existence of a RESET_N pin), and **partially multi-rank-aware** (per-rank MRS loads and per-rank ZQ calibration retries). The FUB resolves both via a single microprogram architecture: a small

step-table ROM per memtype, a step interpreter FSM, and a bypass path that pushes commands directly into the bank-machine and command-encoder fabric without going through the FR-FCFS scheduler.

The init engine is "above" the scheduler in the controller's command-priority hierarchy. While `init_in_progress` is asserted, the scheduler is gated (per §2.7 Stage 2 `init_mask`); only the init engine drives DRAM commands. Once init completes, control hands to the scheduler and refresh manager for normal operation.

17.2 Synthesis Parameters

Parameter	Source	Effect
`MEMTYPE`	top	Picks which step-table ROM is instantiated; tied to `cmd_encoder` MEMTYPE
`NUM_RANKS`	top	Per-rank MRS loop count; per-rank ZQ calibration retries
`SIM_INIT_SCALE`	top (default 1)	Divides post-step delays for sim runs (silicon always 1)
`INIT_ROM_DEPTH`	derived	Number of entries in the active step-table ROM (DDR2 ~32, LPDDR2 ~28)
`STEP_PAYLOAD_WIDTH`	derived	Width of the per-step payload field (carries MRS values, delay counts, etc.)

17.3 Microprogram ROM Organization

The init sequence is encoded as a small ROM of step records. Each step has the same fixed-width encoding regardless of opcode — the opcode field tells the interpreter how to read the payload field.

17.3.1 Step Record Encoding

Field	Width	Description
`opcode`	3	One of `SET_CTRL_BITS`, `DELAY`, `MRS_LOAD`, `ISSUE_CMD`, `WAIT_FOR_BIT`, `END`
`payload`	20	Opcode-dependent — see decoder table below
`post_delay`	12	Cycles to wait after the step completes (scaled by `SIM_INIT_SCALE`)

Field	Width	Description
per_rank	1	When 1, the step iterates over all ranks (only meaningful for MRS_LOAD and ISSUE_CMD)

Total per-step width: 36 bits. ROM depth is ~32 for DDR2 and ~28 for LPDDR2; total ROM footprint ~150 bytes per memtype.

17.3.2 Opcode Decoder

Opcode	Payload Interpretation
SET_CTRL_BITS	{cke[NR-1:0], odt[NR-1:0], reset_n} — drive these output-pin states for the duration of the step
DELAY	delay_cycles[19:0] — explicit delay above and beyond post_delay; used for very-long settling delays like tINIT0 (200 µs)
MRS_LOAD	{mr_index[2:0], mr_value[15:0]} — issue MRS to MR mr_index with value mr_value. When per_rank == 1, iterates over ranks 0 .. NR−1
ISSUE_CMD	{op[3:0], bank[2:0], row_or_col[13:0]} — issue raw DRAM command (PREA, REF, ZQCS, ZQCL). When per_rank, issues to each rank
WAIT_FOR_BIT	{bit_id[3:0], timeout_us[15:0]} — poll a status bit (e.g., dfi_init_complete); fail to INIT_ERROR on timeout
END	None — assert init_done_o, transition to ACTIVE

17.3.3 Memtype Selection

```
generate
    if (MEMTYPE == "DDR2") begin : g_rom_ddr2
        init_steps_ddr2_rom #(
            .DEPTH(INIT_ROM_DEPTH)
        ) u_rom (.addr(step_pc_o), .data(step_record_i));
    end
    else begin : g_rom_lpddr2
        init_steps_lpddr2_rom #(
            .DEPTH(INIT_ROM_DEPTH)
        ) u_rom (.addr(step_pc_o), .data(step_record_i));
    end
endgenerate
```

The ROMs live in rtl/macro/init_steps_ddr2_rom.sv and rtl/macro/init_steps_lpddr2_rom.sv. They are written by hand from JEDEC initialization sequences and are *not* generated — the sequences are deliberate, short, and reviewable.

The choice of separate ROM modules (rather than a single ROM with a memtype-indexed lookup) keeps the unused ROM out of the synthesis pass when MEMTYPE is fixed at elaboration. The resulting bit-stream contains only the relevant init recipe.

17.4 FSM

The interpreter is a five-state FSM:

State	Held while	Exits via
IDLE	CTRL.init_start not asserted; controller sits in POST_RESET	init_start rising edge → FETCH
FETCH	One cycle to address the ROM and latch the step record	Step record valid → EXECUTE
EXECUTE	Step's action being performed (drive ctrl bits, issue command, etc.)	Action complete → POST_DELAY (if post_delay > 0) or FETCH (next step)
POST_DELAY	Down-counter from post_delay (scaled by SIM_INIT_SCALE)	Counter == 0 → FETCH (next step), or END if last
INIT_ERROR	A WAIT_FOR_BIT timed out or ZQ retried out — terminal state	Software writes CTRL.init_force_restart → IDLE

There is no separate state per opcode — the EXECUTE state runs a small combinational dispatch on the opcode field. This keeps the FSM minimal and the init_step_dbg CSR (per HAS §6.3 STATUS) reports the step PC directly.

17.5 Step Execution Detail

17.5.1 SET_CTRL_BITS

Drives the controller's output-pin staging registers:

```
dfi_cke[NR-1:0] = payload[NR-1+NR : NR]
dfi_odt[NR-1:0] = payload[NR-1 : 0]
dfi_reset_n     = payload[NR*2]
```

These outputs route through power_state_fub → odt_ctrl_fub → gear_dfi_fub to the DFI master pins. While init is in progress, the power-state FSM mirrors the init engine's dfi_cke instead of its own.

17.5.2 DELAY

Loads a separate `long_delay_cnt` register with `payload[19:0]` and waits in `EXECUTE` until it reaches zero. `SIM_INIT_SCALE > 1` divides the load value at elaboration *and* at run-time CSR write (so the runtime override `INIT_TUNING.init_timeout_ms` honors the scale too). For tINIT0-class delays (200 µs at 200 MHz = 40 K cycles), this counter is 20 bits.

17.5.3 MRS_LOAD

Issues an MRS DRAM command through the bypass-into-encoder path:

```
init_cmd_op_o        = MRS
init_cmd_bank_o      = mr_index           // the bank field encodes
the MR index in JEDEC
init_cmd_row_or_col  = mr_value
init_cmd_rank_o      = current_rank       // iterates 0..NR-1 if
per_rank
init_cmd_strobe_o    = 1
```

The bank machines see the MRS issue and don't change state (MRS doesn't affect bank state); the command flows through `cmd_encoder` → `gear_dfi` to the DFI master. When `per_rank == 1`, the FSM uses a small `rank_iter` counter and stays in `EXECUTE` across NR cycles, advancing one rank per MC clock. tMRD-class minimum-gap is honored via `post_delay`.

17.5.4 ISSUE_CMD

Generic command issue — same path as MRS_LOAD but with the opcode and bank/column from the payload. Used for PREA (all-bank precharge), REF (initial refresh batches), ZQCL (init-time ZQ calibration long), and ZQCS (init-time ZQ short).

17.5.5 WAIT_FOR_BIT

Polls a controller-internal status signal (`bit_id` is one of: `dfi_init_complete`, `zq_done`, `mrs_ack`, etc.). The interpreter holds `EXECUTE` until the bit asserts. A timeout counter loaded from `payload[19:4]` (in microseconds, scaled appropriately) drives `INIT_ERROR` if the bit doesn't assert in time.

For ZQ calibration specifically, the timeout-retry path consults `INIT_TUNING.zq_retries`:

```
if (timeout AND zq_retry_cnt < zq_retries):
    zq_retry_cnt = zq_retry_cnt + 1
    re-execute ZQ command and re-poll        // back-edge to FETCH on
same step
else if (timeout AND retries exhausted):
    → INIT_ERROR
```

17.5.6 END

Asserts `init_done_o` for one cycle (driving the `irq_init_done` IRQ and `STATUS.init_done = 1`), then transitions to `IDLE` for the next init request.

17.6 Bypass Path Into Bank Machines

Init commands do not go through the scheduler. Instead, the init engine has a direct path into each bank machine and into the command encoder, via the `init_cmd_if` interface declared in §2.9:

```
init_cmd_op_o      → all bank machines (each watches its own rank/bank
match)
init_cmd_rank_o    → cmd_encoder rank-select
init_cmd_bank_o    → cmd_encoder bank-select; bank_machine identity
match
init_cmd_row_or_col → cmd_encoder address operand
init_cmd_strobe_o  → all bank machines and cmd_encoder
```

Bank machines treat init commands as if they came from the scheduler — they transition states accordingly (PRE → PRECHARGING, REF → REFRESHING, etc.). The scheduler does not observe these commands' issue; from its point of view, the bus is just "blocked by init."

The command encoder accepts init commands on the bypass path and serializes them onto DFI through the same gear logic as scheduler-issued commands. There is no separate init-mode signal in `cmd_encoder`; the path is symmetric.

17.7 Multi-Rank Init Loop

For per-rank steps (`per_rank == 1`), the FSM uses a `rank_iter` counter:

```
EXECUTE state:
    if (per_rank):
        issue command to rank_iter
        rank_iter = (rank_iter + 1) % NR
        if (rank_iter == 0):                  // wrapped, all ranks
done
            advance to POST_DELAY / FETCH
        else:
            stay in EXECUTE (next cycle issues to next rank)
    else:
        issue command (rank-agnostic; CS_n[*] = 0)
        advance to POST_DELAY / FETCH
```

This means a per_rank MRS_LOAD step takes NR cycles in EXECUTE. For NR=4, eight cycles for an MR0+MR1+MR2+MR3 sequence on 4 ranks — small overhead in the init context (where individual delays run hundreds of microseconds anyway).

The tMRD minimum-gap between MRS to the same rank is honored via post_delay. The minimum-gap between MRS to *different* ranks is shorter (tMRD_inter_rank, typically 0 or 1 cycle) and is absorbed by the natural pipelining of the EXECUTE loop.

17.8 SIM_INIT_SCALE — The 200µs → 200 cycles trick

JEDEC init has several multi-hundred-µs delays (tINIT0 = 200 µs, tINIT1 = 500 µs, tINIT3 = 200 µs). At 200 MHz silicon clock, that's 40,000 cycles each — and simulation at cycle-accurate speeds would take minutes per test just for init.

SIM_INIT_SCALE divides all delay loads at elaboration:

```
effective_delay = delay_from_rom / SIM_INIT_SCALE
```

Setting SIM_INIT_SCALE = 10000 reduces tINIT0 from 40 K cycles to 4 cycles in sim. Silicon always uses SIM_INIT_SCALE = 1; the scale is parameter-only, not CSR (writing it would corrupt silicon init).

The scale applies to both DELAY opcode payloads and post_delay fields. The WAIT_FOR_BIT timeout is *not* scaled — timing-out on a DFI handshake should still be detected in sim.

17.9 Init-in-Progress Gating

While the FSM is anywhere except IDLE:

- init_in_progress_o = 1
- Scheduler is gated (per §2.7 Stage 2)
- Refresh manager is gated (per §2.11 FSM)
- Power-state FSM mirrors init's dfi_cke outputs
- AXI4 slave is *not* gated at the AXI boundary (AXI bursts can accumulate in aw_buf / ar_buf while init runs); they just can't reach DFI

When END executes, init_in_progress_o drops, scheduler and refresh resume, and accumulated AXI bursts begin issuing through the normal path.

17.10 Interface

17.10.1 Outputs to Bank Machines (init bypass)

Signal	Direction	Width	Description
init_cmd_strobe_o	output	1	A new init command is being issued this cycle
init_cmd_op_o	output	4	DRAM command opcode (same encoding as scheduler issue)
init_cmd_rank_o	output	$clog2(NR)	Target rank
init_cmd_bank_o	output	$clog2(NB)	Target bank / MR index for MRS
init_cmd_row_o	output	ROW_WIDTH	Row (for ACT) or MR value (for MRS)
init_cmd_col_o	output	COL_WIDTH	Column (for RD/WR; init doesn't normally use)

17.10.2 Outputs to Power-State / ODT / Pins

Signal	Direction	Width	Description
init_cke_o[NR]	output	NR	Direct CKE drive during init (power_state muxes this in)
init_odt_o[NR]	output	NR	Direct ODT drive during init
init_reset_n_o	output	1	DDR2 RESET_N pin drive (tied 1 for LPDDR2 builds)

17.10.3 Status / IRQ Outputs

Signal	Direction	Width	Description
init_in_progress_o	output	1	High during any non-IDLE state
init_done_o	output	1	One-cycle pulse on END; drives irq_init_done
init_error_o	output	1	Latched on INIT_ERROR; drives irq_init_error

Signal	Direction	Width	Description
init_step_pc_o	output	8	Current step PC; drives STATUS.init_step_dbg

17.10.4 Inputs from CSR

Signal	Direction	Width	Source
cfg_init_start_i	input	1	CTRL.init_start (rising edge starts)
cfg_init_force_restart_i	input	1	CTRL.init_force_restart
cfg_zq_retries_i	input	4	INIT_TUNING.zq_retries
cfg_init_timeout_ms_i	input	8	INIT_TUNING.init_timeout_ms
cfg_mr_values_i[4]	input	16 × 4	MR0..MR3 from CSR (override the ROM default)

The MR0..MR3 CSR override path is important — software writes the MR values before asserting init_start, and the init engine substitutes them for the ROM-baked defaults during MRS_LOAD steps. This is how the LPDDR2 boot agent picks burst length, CAS latency, etc. without re-ROM-ing the controller.

17.10.5 Inputs from DFI (status polling)

Signal	Direction	Width	Source
dfi_init_complete_i	input	1	DFI status sub-interface
zq_done_i	input	1	Internal: ZQ window expired without error
mrs_ack_i	input	1	Bank machine acknowledges MRS settled

17.11 CSR Hooks

CSR field	Source	Use case
STATUS.init_done (R)	Latched init_done_o	Software polls for init complete
STATUS.init_error (R/clear)	Latched init_error_o	Software polls for init failure

CSR field	Source	Use case
STATUS.init_step_dbg (R)	init_step_pc_o	Bring-up: where did init get stuck?
OBS_INIT_DURATION_MS (R)	Total init duration counter	Telemetry: how long did init take

17.12 Verification Notes (cocotb test plan)

Scenario	What it proves
DDR2 init from cold reset to init_done	Full DDR2 step sequence executes
LPDDR2 init from cold reset to init_done	Full LPDDR2 step sequence executes
Multi-rank init (NR=2): each MRS_LOAD step issues to both ranks	Per-rank MRS loop
WAIT_FOR_BIT dfi_init_complete times out → INIT_ERROR	Timeout path
ZQ calibration fails, retries zq_retries times, then INIT_ERROR	ZQ retry logic
Software overrides MR0 via CSR; init uses the override, not ROM default	MR override path
SIM_INIT_SCALE = 10000: tINIT0 delay completes in 4 cycles instead of 40 K	Sim shortcut works
init_force_restart mid-sequence → IDLE → restart from step 0	Restart path
AXI bursts queued during init → no DFI issue until init_done asserts	Init-in-progress gating
Scheduler init_mask correctly blocks normal traffic during init	Cross-FUB integration
Init completes; refresh manager's first wants_refresh fires after at least tREFI cycles	Refresh handoff timing

17.13 Open Questions / Future Work

- **MR4 readback during init.** LPDDR2 requires reading MR4 to detect device temperature class. Currently the init engine doesn't do this — it leaves MR4 readback to the post-init power-state FSM. Could fold it into the init sequence (as a `WAIT_FOR_BIT` or a new `MR_READ` opcode), at the cost of a slightly longer init. Likely worth doing in v2 to remove the post-init MR4 race.
- **Step ROM versioning.** When silicon ships and a new DRAM part requires a tweaked init sequence, the ROM has to change. Should there be a CSR-side "ROM patch" path (writeable RAM that overlays specific ROM entries)? Adds CSR area but enables in-field updates. Punt until a real DRAM-compat issue forces the question.
- **Parallel multi-rank init.** Currently the per-rank loop is serial — NR cycles per per-rank step. Could issue MRS to multiple ranks in parallel (broadcast MRS with CS_n driven on multiple ranks). Saves init cycles at the cost of more cross-rank gate logic. Probably not worth it; init runs once at boot.
- **Soft-reset semantics during init.** If `mc_rst_n` asserts during init, the FUB cleanly returns to IDLE. But if `soft_reset` (CSR write) asserts during init, behavior is currently to also return to IDLE. Should the CSR soft-reset complete the current step first to avoid DRAM in an inconsistent state? Worth a verification scenario to document the current behavior.

---

18 Power-State FSM (`power_state_fub`)

Module: `power_state_fub.sv` **Location:** `rtl/fub/` **Category:** FUB **Parent:** `ddr2_lpddr2_ctrl` **Status:** Draft v0.1

> Architectural context: HAS §3.5. The HAS state diagram for power transitions is in `ddr2_lpddr2_has/assets/mermaid/04_power_state_fsm.png` — refer to it for state transitions. This section is the implementation view (per-rank

FSM array, per-rank CKE routing, self-refresh coord with refresh manager, quiet-point detector, init-time CKE handoff).

18.1 Purpose

power_state_fub controls DRAM power state. It tracks per-rank power-state independently so that, for example, rank 0 can stay ACTIVE servicing AXI traffic while rank 1 is in SELF_REFRESH for thermal or workload reasons. The FUB drives the per-rank dfi_cke[NR] outputs that DRAM uses to enter / exit power-down states, and coordinates with refresh_mgr_fub on self-refresh entry / exit.

The FUB also owns the **quiet-point detector** consumed by csr_apb_fub for CSR override staging. Quiet point is "nothing in flight" — defined as txn_queue empty (or only PENDING), refresh manager IDLE, no DFI command pipelined, every rank in ACTIVE (not in any low-power state).

18.2 Synthesis Parameters

Parameter	Source	Effect
NUM_RANKS	top	One per-rank FSM instance per rank
MEMTYPE	top	DPD state is LPDDR2-only; DDR2 builds elide the DPD transitions
APD_THRESHOLD_WIDTH	derived	Width of the apd_idle_cnt per-rank idleness counter (16-bit)
SRF_THRESHOLD_WIDTH	derived	Width of the srf_idle_cnt (24-bit)

18.3 Per-Rank FSM Array

The FUB instantiates NUM_RANKS independent FSMs, one per rank. Each per-rank FSM has the four canonical states:

State	DRAM behavior	dfi_cke[r]
ACTIVE	DRAM ready for commands; refreshes flow through refresh_mgr	1
ACTIVE_POWER_DOWN (APD)	CKE de-asserted; DRAM ignores commands; refreshes paused	0

State	DRAM behavior	dfi_cke[r]
SELF_REFRESH (SRF)	DRAM internally refreshes; controller's tREFI paused	0
DEEP_POWER_DOWN (DPD)	LPDDR2-only; full power-off; SoC must re-init this rank	0

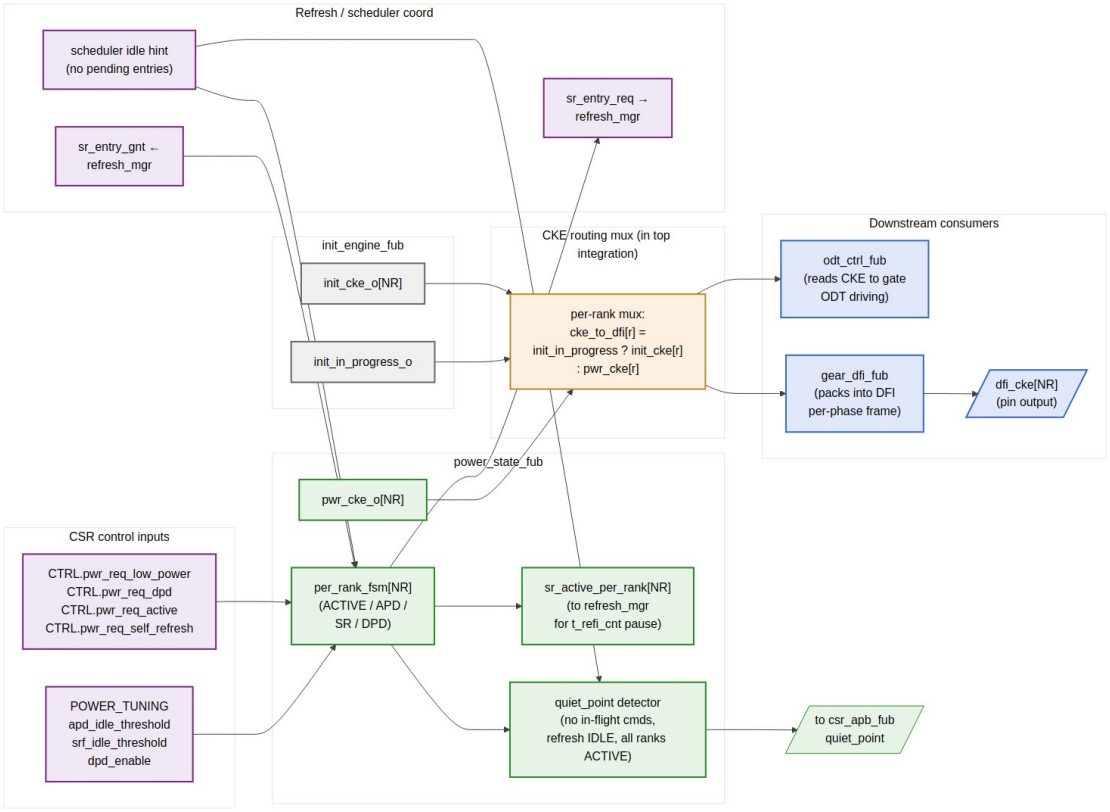
Transitions are software-requested via per-rank CSR control bits (CTRL.pwr_req_* are broadcast to all ranks in v1; a v2 enhancement would add per-rank control) OR auto-triggered by idleness counters (apd_idle_cnt, srf_idle_cnt).

18.3.1 Per-Rank vs. Channel-Wide Trigger Sources

Trigger	Scope	Notes
CTRL.pwr_req_low_power (R/W)	Channel-wide	All ranks → APD
CTRL.pwr_req_self_refresh	Channel-wide	All ranks → SRF
CTRL.pwr_req_dpd	Channel-wide (LPDDR2)	All ranks → DPD
CTRL.pwr_req_active	Channel-wide	All ranks → ACTIVE
apd_idle_cnt[r] expired	Per-rank	Auto-APD on rank r idleness
srf_idle_cnt[r] expired	Per-rank	Auto-SRF on rank r idleness
AXI traffic to rank r	Per-rank	Forces rank r back to ACTIVE

The per-rank auto-triggers mean that traffic localized to a subset of ranks naturally lets the others enter low-power without software intervention. This is the multi-rank power-saving story.

18.4 CKE Routing Topology



Per-Rank CKE Routing — init override, FSM-driven CKE, downstream consumers

Source: 11_power_state_cke_routing.mmd

The diagram shows the full CKE routing topology — how init-time CKE drive overrides FSM-driven CKE during init, and how the muxed CKE per rank flows to `odt_ctrl`, `gear_dfi`, and the DFI pins.

18.4.1 Init Override

During init (`init_in_progress == 1`), the init engine drives `init_cke_o[NR]` directly. The CKE mux in the top integration overrides `pwr_cke_o[NR]` with `init_cke_o[NR]`:

```
assign cke_to_dfi[r] = init_in_progress ? init_cke[r] : pwr_cke[r];
```

When init completes, the init engine drops `init_in_progress`, and `pwr_cke_o[NR]` (initialized to all-1 at FSM reset) drives.

18.4.2 Downstream Consumers

- `odt_ctrl_fub` reads CKE to gate ODT driving (ODT is only valid when CKE is asserted)

- `gear_dfi_fub` packs CKE into the per-phase DFI frame
- The DFI `dfi_cke[NR]` pin output is driven from the gear

18.5 Per-Rank FSM Transitions

```
ACTIVE → APD:
  trigger: CTRL.pwr_req_low_power || apd_idle_cnt[r] expired
  pre-conditions: bank_state[r][*] all IDLE (no row open)
  drive: dfi_cke[r] = 0
  wait: tXP (exit-power-down latency budget) baked into the entry, no
count needed

APD → ACTIVE:
  trigger: CTRL.pwr_req_active || AXI traffic to rank r || refresh_mgr
wants_refresh
  drive: dfi_cke[r] = 1
  wait: tXP cycles (the rank cannot accept commands until tXP elapsed)

ACTIVE → SRF:
  trigger: CTRL.pwr_req_self_refresh || srf_idle_cnt[r] expired
  pre-conditions:
    - bank_state[r][*] all IDLE
    - refresh_mgr.sr_entry_req asserted
    - refresh_mgr.sr_entry_gnt asserted (refresh_mgr has paused its
own scheduling)
  drive: issue SREFE (self-refresh entry) DRAM command, then
dfi_cke[r] = 0
  notify: sr_active_per_rank[r] = 1 → refresh_mgr pauses t_refi_cnt
for rank r

SRF → ACTIVE:
  trigger: CTRL.pwr_req_active || AXI traffic to rank r
  drive: dfi_cke[r] = 1, then issue SREFX (self-refresh exit) DRAM
command
  wait: tXSR (~ tRFC + 10 ns) before the rank accepts commands
  notify: sr_active_per_rank[r] = 0 → refresh_mgr resumes t_refi_cnt
          AND fires sr_exit_done_i to refresh_mgr (full handshake
closes)

ACTIVE → DPD:  (LPDDR2 only)
  trigger: CTRL.pwr_req_dpd
  pre-conditions: bank_state[r][*] all IDLE, refresh_mgr coordinated
  drive: dfi_cke[r] = 0 with special DPD encoding on CA bus
  rank is now fully powered down; software must full-init it to exit
```

```
DPD → ACTIVE: (LPDDR2 only)
  Not directly transitionable; software writes CTRL.init_force_restart
+ select the
  rank's MR sequence; the init_engine re-walks the per-rank MRS_LOAD
steps for rank r
```

Reset state per rank is `ACTIVE` with `dfi_cke[r] = 1`.

18.6 Quiet-Point Detector

`quiet_point_o` is asserted when **all** of the following hold for at least one MC cycle:

```
quiet_point = (txn_queue.q_occupancy == 0
               OR all queue entries are PENDING)        // §2.6
             AND (refresh_mgr.dbg_state == IDLE)        // §2.11
             AND (init_engine.init_in_progress == 0)    // §2.12
             AND (cmd_encoder pipeline drained)
             AND (all ranks' per_rank_fsm[r] == ACTIVE)  // this FUB
```

The detector is purely combinational from the inputs — `csr_apb_fub` samples it directly to time CSR override commits. When `CTRL.config_apply == 1`, the FUB also asserts a "quiet-point hold" signal that drains the scheduler and refresh manager to expedite the quiet point (rather than waiting for natural quiescence).

The drain phase is bounded — typically < 256 MC cycles for a healthy queue — and is the only operational disruption from a runtime override.

18.7 Idleness Counters

Per-rank `apd_idle_cnt[r]` and `srf_idle_cnt[r]`:

```
// Per cycle, per rank:
if (per_rank_fsm[r] == ACTIVE AND no AXI traffic targets rank r):
    if (apd_idle_cnt[r] < APD_THRESHOLD): apd_idle_cnt[r] =
apd_idle_cnt[r] + 1
    if (srf_idle_cnt[r] < SRF_THRESHOLD): srf_idle_cnt[r] =
srf_idle_cnt[r] + 1
else:
    apd_idle_cnt[r] = 0
    srf_idle_cnt[r] = 0

// Trigger conditions:
auto_apd_trigger[r] = (apd_idle_cnt[r] == cfg_apd_idle_threshold_i)
auto_srf_trigger[r] = (srf_idle_cnt[r] == cfg_srf_idle_threshold_i)
```

The thresholds come from `POWER_TUNING.apd_idle_threshold` (16-bit, default 1024 cycles ≈ 5 µs at 200 MHz) and `POWER_TUNING.srf_idle_threshold` (24-bit, default 16 M cycles ≈ 80 ms at 200 MHz). Software can disable auto-triggers by setting the threshold to its max value (no-trigger ceiling).

"AXI traffic targets rank r" is computed by ORing all `q_entries[i].valid AND (q_entries[i].rank == r)` from the `txn_queue` snapshot. This is the same wire the scheduler uses; reading it adds zero new buses.

18.8 Self-Refresh Coordination with `refresh_mgr_fub`

The handshake (per HAS §3.4 and §2.11):

1. `power_state_fub` decides to enter SRF on rank `r` (CSR-requested or auto)
2. `power_state_fub` asserts `sr_entry_req_o = 1`
3. `refresh_mgr_fub` waits until it's safe (no auto-refresh in flight) and asserts `sr_entry_gnt_i = 1`
4. `power_state_fub` issues SREFE command (via `cmd_encoder` bypass path), then drops `dfi_cke[r] = 0`
5. `power_state_fub` asserts `sr_active_per_rank_o[r] = 1`
6. `refresh_mgr_fub` pauses `t_refi_cnt` while any rank is in SR
7. When software requests exit (or AXI traffic to rank `r`), `power_state_fub` drives `dfi_cke[r] = 1`, issues SREFX, waits tXSR
8. `power_state_fub` drops `sr_active_per_rank_o[r] = 0`, asserts `sr_exit_done_o = 1` for one cycle
9. `refresh_mgr_fub` resumes `t_refi_cnt` and the FSM returns to ACTIVE for that rank

If multiple ranks enter SR at different times, `refresh_mgr_fub` only resumes `t_refi_cnt` when the **last** rank exits SR (i.e., when `OR(sr_active_per_rank_o[*]) == 0`). The handshake is per-rank but the refresh-pause is channel-wide — refresh of any non-SR rank still needs to honor JEDEC tREFI.

18.9 Interface

18.9.1 Per-Rank CKE Outputs

Signal	Direction	Width	Description
pwr_cke_o[NR]	output	NR	Per-rank CKE drive (muxed against init in top)
pwr_state_o[NR]	output	NR × 2	Per-rank state encoding (ACTIVE / APD / SRF / DPD)

18.9.2 Refresh Manager Handshake

Signal	Direction	Width	Description
sr_entry_req_o	output	1	Channel-wide request to enter SR (any rank pending)
sr_entry_gnt_i	input	1	refresh_mgr ack — no auto-refresh in flight
sr_active_per_rank_o[NR]	output	NR	Per-rank SR-active
sr_exit_done_o	output	1	One-cycle pulse on per-rank SREFX completion

18.9.3 Scheduler Coordination

Signal	Direction	Width	Description
txn_queue_rank_pending_i[NR]	input	NR	OR-reduce of `q_entries[*].rank == r AND valid` from §2.6
quiet_point_o	output	1	The big OR-AND signal documented above

18.9.4 Init Engine Coordination

Signal	Direction	Width	Description
init_in_progress_i	input	1	From init_engine_fub
init_cke_i[NR]	input	NR	From init_engine_fub (muxed at top)

18.9.5 Command Encoder Bypass (for SREFE / SREFX / DPDE)

Signal	Direction	Width	Description
pwr_cmd_strobe_o	output	1	Power-related DRAM command being issued
pwr_cmd_op_o	output	4	SREFE, SREFX, DPDE, DPDX
pwr_cmd_rank_o	output	$clog2(NR)	Target rank

18.9.6 CSR

Signal	Direction	Width	Source
cfg_pwr_req_low_power_i	input	1	CTRL.pwr_req_low_power
cfg_pwr_req_dpd_i	input	1	CTRL.pwr_req_dpd
cfg_pwr_req_active_i	input	1	CTRL.pwr_req_active
cfg_pwr_req_self_refresh_i	input	1	CTRL.pwr_req_self_refresh
cfg_apd_idle_threshold_i	input	16	POWER_TUNING.apd_idle_threshold
cfg_srf_idle_threshold_i	input	24	POWER_TUNING.srf_idle_threshold
cfg_dpd_enable_i	input	1	POWER_TUNING.dpd_enable (LPDDR2 only)

18.9.7 Status / IRQ

Signal	Direction	Width	Description
status_history_o	output	32	Last 8 channel-wide power-state transitions (4-bit encoding each); drives STATUS_HISTORY CSR
dbg_state_per_rank_o[NR]	output	NR × 2	Per-rank FSM state

18.10 CSR Hooks

CSR field	Source	Use case
STATUS.power_state (R)	Encoded combined state	Quick health check
STATUS_HISTORY (R)	status_history_o	Bring-up: power-state oscillation debug
STATUS.power_state_per_rank_R<R> (R)	dbg_state_per_rank_o[R]	Per-rank state
POWER_TUNING.apd_idle_threshold (R/W)	cfg_apd_idle_threshold_i	Software tunable
POWER_TUNING.srf_idle_threshold (R/W)	cfg_srf_idle_threshold_i	Software tunable

18.11 Verification Notes (cocotb test plan)

Scenario	What it proves
Cold reset → init → ACTIVE for all ranks	Reset / init sequencing
CTRL.pwr_req_low_power; rank 0 enters APD; AXI traffic resumes ACTIVE	APD entry/exit (channel-wide trigger)
Auto-APD on rank-0 idleness while rank-1 has traffic	Per-rank auto-trigger; rank-1 stays ACTIVE
CTRL.pwr_req_self_refresh; all ranks enter SRF; t_refi_cnt pauses	Self-refresh entry coordination with refresh_mgr
SRF exit on AXI traffic; SREFX command issued; tXSR honored	Self-refresh exit timing
LPDDR2 build, CTRL.pwr_req_dpd; ranks enter DPD; software re-init exits	DPD entry / re-init exit (LPDDR2)
DDR2 build, CTRL.pwr_req_dpd; ignored (no DPD on DDR2)	DPD-not-available behavior
Multi-rank: rank 0 SRF + rank 1 ACTIVE; refresh manager pauses t_refi_cnt	Multi-rank SRF correctness
quiet_point asserts when all conditions hold	Quiet-point detector
CTRL.config_apply triggers fast quiet-point drain	Software-forced quiet point

Scenario	What it proves
Race: AXI traffic arrives same cycle as auto-APD trigger; rank stays ACTIVE	Priority of traffic over auto-trigger
STATUS_HISTORY captures 8 transitions on rapid CSR power requests	History register correctness

18.12 Open Questions / Future Work

- **Per-rank CSR control.** v1 has channel-wide `CTRL.pwr_req_*` bits. A v2 enhancement is per-rank requests (`CTRL.pwr_req_low_power_rank_R<R>`). Useful for software that knows it has consolidated workload onto rank 0 and wants rank 1+ in SR; not in v1 to keep the CSR map flat.
- **Software-driven SRF wake on AXI hint.** When SR is software-forced, the FUB still wakes on AXI traffic. Some use cases want "stay in SR until software explicitly says go." Could add a `POWER_TUNING.srf_lock` bit. Punt to v2.
- **Per-rank quiet-point.** Currently `quiet_point` is channel-wide. A per-rank quiet point would let CSR overrides apply during partial SR. Adds complexity to `csr_apb_fub` commit logic; probably not worth it.
- **DPD wake latency.** DPD exit requires full re-init (~100s of µs). Bring-up will want to characterize this — should the FUB time the wake and expose it via `OBS_DPD_WAKE_LATENCY_US`? Add if the bring-up team asks.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

19 Command Encoder (`cmd_encoder_fub`)

Module: `cmd_encoder_fub.sv` **Location:** `rtl/fub/` **Category:** FUB **Parent:** `ddr2_lpddr2_ctrl` **Status:** Draft v0.1

> Architectural context: HAS §3.6. This block is the **memtype-swappable encoder** — the one stage of the controller where DDR2 and LPDDR2 actually diverge at the wire level. Everything upstream is rank/bank-tuple-abstract; everything downstream is DFI-frame-abstract.

19.1 Purpose

`cmd_encoder_fub` takes an abstract DRAM command operand `(op, rank, bank, row, col, auto_pre)` and emits the wire-level DFI command bits:

- For **DDR2**: traditional `dfi_ras_n` / `dfi_cas_n` / `dfi_we_n` strobes plus `dfi_address` and `dfi_bank`
- For **LPDDR2**: the JEDEC 20-bit CA-bus packed encoding per DFI v2.1 §3.1 Table 1

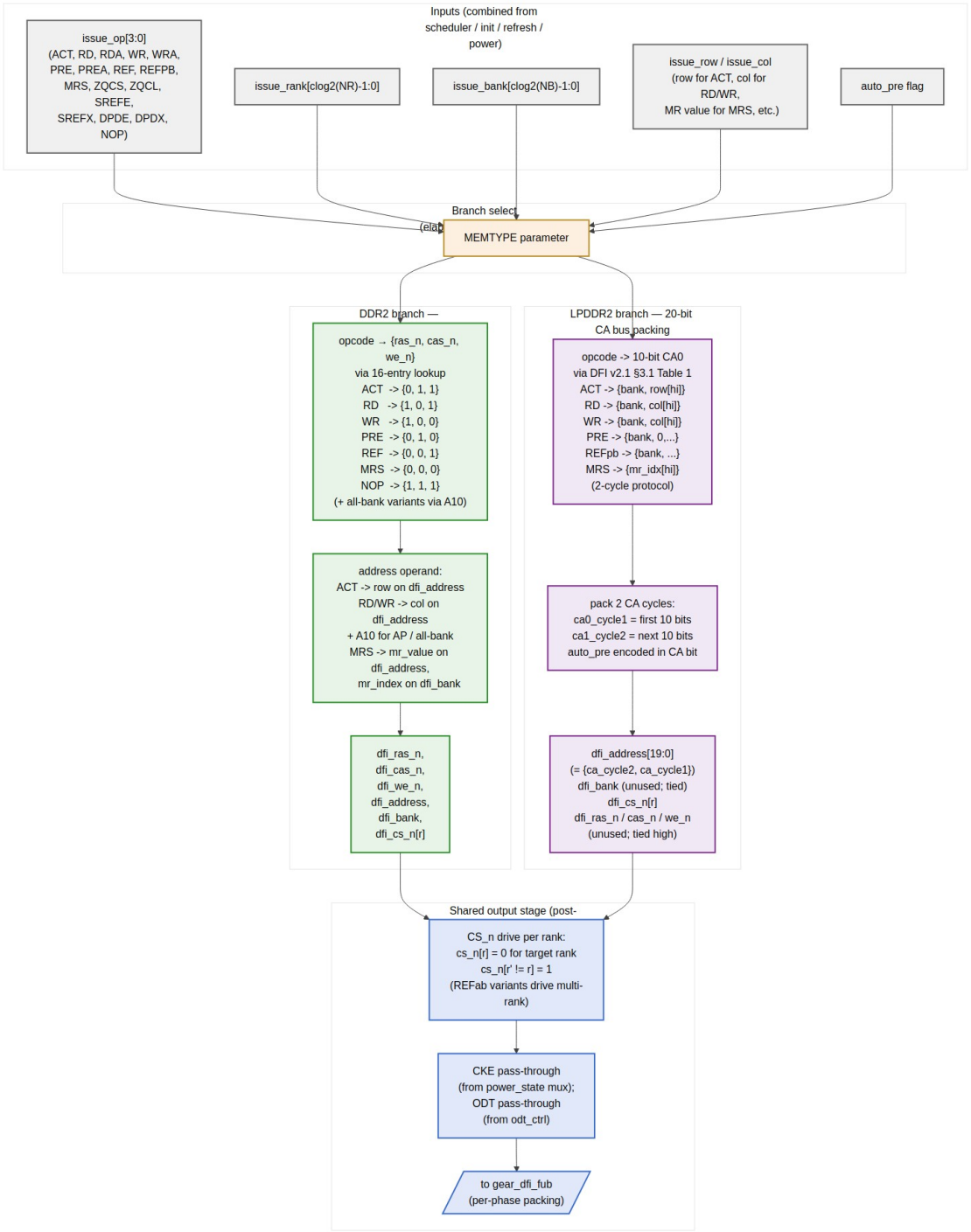
Per-rank `dfi_cs_n[NR]` driving is also this FUB's job — it derives the per-rank chip-select pattern from the issue's target rank and the command type (single-rank commands drive one CS_n low; REFab with per-rank dispatch from §2.11 drives the appropriate single rank).

The FUB is purely combinational at the per-command level — the input arrives from the scheduler / init / refresh / power-state bypass paths, the encoded output flows the same cycle to `gear_dfi_fub` for per-phase packing. There is no state inside the encoder; that lives upstream (scheduler, init, refresh) or downstream (gear, bank machines).

19.2 Synthesis Parameters

Parameter	Source	Effect
`MEMTYPE`	top	Picks the DDR2 or LPDDR2 branch (only one branch is synthesized)
`NUM_RANKS`	top	Width of `dfi_cs_n[NR]` output
`NUM_BANKS`	top	Bank-field width
`ROW_WIDTH`	top	Row operand width
`COL_WIDTH`	top	Column operand width
`DFI_ADDR_WIDTH`	top	Width of `dfi_address` output (14 for DDR2, 20 for LPDDR2)

19.3 Branch Selection (Elaboration-Time)



Command-Encoder Branches — DDR2 strobes vs LPDDR2 CA-bus, shared per-rank CS_n stage

Source: 12_cmd_encoder_branches.mmd

```
generate
    if (MEMTYPE == "DDR2") begin : g_ddr2_enc
        ddr2_cmd_encoder #(
            .NUM_RANKS(NUM_RANKS),
            .NUM_BANKS(NUM_BANKS),
            .ROW_WIDTH(ROW_WIDTH),
            .COL_WIDTH(COL_WIDTH),
            .DFI_ADDR_WIDTH(DFI_ADDR_WIDTH)
        ) u_enc ( ... );
    end
    else begin : g_lpddr2_enc
        lpddr2_cmd_encoder #(
            .NUM_RANKS(NUM_RANKS),
            .NUM_BANKS(NUM_BANKS),
            .ROW_WIDTH(ROW_WIDTH),
            .COL_WIDTH(COL_WIDTH),
            .DFI_ADDR_WIDTH(DFI_ADDR_WIDTH)
        ) u_enc ( ... );
    end
endgenerate
```

Only one encoder is in silicon per build. The two encoders share the per-rank CS_n drive stage (next section), which is post-branch.

19.4 DDR2 Encoding Branch

DDR2 uses the classic {RAS_n, CAS_n, WE_n} strobe encoding with the row/column operand on dfi_address. The 16-entry lookup is purely combinational:

Opcode	ras_n	cas_n	we_n	dfi_address	dfi_bank	Auto-pre via A10
ACT	0	1	1	row	bank	n/a
RD	1	0	1	{col, A10=0}	bank	A10=0
RDA	1	0	1	{col, A10=1}	bank	A10=1
WR	1	0	0	{col, A10=0}	bank	A10=0
WRA	1	0	0	{col, A10=1}	bank	A10=1
PRE	0	1	0	A10=0 (specific bank)	bank	n/a
PREA	0	1	0	A10=1 (all banks)	x	n/a
REF	0	0	1	x	x	n/a

Opcode	ras_n	cas_n	we_n	dfi_address	dfi_bank	Auto-pre via A10
MRS	0	0	0	MR value	MR index (0..3)	n/a
ZQCS	1	1	0	A10=0	x	n/a
ZQCL	1	1	0	A10=1	x	n/a
NOP	1	1	1	x	x	n/a

The A10 bit doubles as the auto-precharge flag (for RD/WR) and the all-bank flag (for PRE). Both meanings are mutually exclusive — there's no ambiguity because the same opcode never uses both.

For multi-cycle DDR2 commands that don't exist in the base set (e.g., DDR2 doesn't have a per-bank refresh — REFpb opcode is silently encoded as REF in DDR2 builds via a generate-time assertion).

19.4.1 DDR2-Specific Multi-Cycle Behavior

DDR2 commands are single-cycle at the DFI command interface. The encoder's output is valid one MC cycle after the issue strobe; gear_dfi packs it into the appropriate DFI phase slot. There is no command-side multi-cycle protocol on DDR2 — all such complexity is on the data path side.

19.5 LPDDR2 Encoding Branch

LPDDR2 uses a packed 20-bit Command-Address (CA) bus per DFI v2.1 §3.1 Table 1. The CA bus is 10 bits wide and runs at DDR rate, so each logical DRAM command occupies 2 CA cycles (CA0 on rising edge, CA1 on falling edge equivalent — gear_dfi handles the phase split).

The encoder produces a flat 20-bit `dfi_address[19:0]` where:

- `dfi_address[9:0]` = first CA cycle bits (CA0)
- `dfi_address[19:10]` = second CA cycle bits (CA1)

19.5.1 CA-Bus Encoding (Selected Subset)

LPDDR2's CA encoding is more compact than the JEDEC spec text suggests. Per DFI v2.1 §3.1 Table 1, the relevant bit patterns (this table is the canonical lookup):

Opcode	CA0 (first cycle, bits 9..0)	CA1 (second cycle, bits 9..0)
ACT	`{Row[16:14], 0, 0, 1, BA[2:0], Row[13:11]}`	`{Row[10:0]}`

Opcode	CA0 (first cycle, bits 9..0)	CA1 (second cycle, bits 9..0)
RD	{0, 0, 1, AP, BA[2:0], 0, 0, Col[9]}	{0, Col[11:10], Col[8:1]}
WR	{0, 0, 0, AP, BA[2:0], 0, 0, Col[9]}	{0, Col[11:10], Col[8:1]}
PRE	{1, 0, 0, 1, AB, BA[2:0], 0, 0}	{0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
REFpb	{1, 0, 1, 0, 0, BA[2:0], 0, 0}	{0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
REFab	{1, 0, 1, 0, 1, 0, 0, 0, 0, 0}	{0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
MRW	{0, 1, 1, 0, MRA[7:0]}	{OP[7:0], 0, 0}
MRR	{0, 1, 0, 0, MRA[7:0]}	{0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
ZQC	{1, 0, 0, 1, 0, ZQ_type[1:0], 0, 0, 0}	(don’t care)
NOP	all-1	all-1

The flat 20-bit `dfi_address` is the concatenation of CA0 and CA1; `gear_dfi_fub` splits them across the appropriate DFI phases.

19.5.2 Address-Bit Spreading

LPDDR2 spreads row and column bits across CA0 and CA1 in a non-contiguous pattern. The encoder is responsible for the spreading; the rest of the controller uses contiguous `row[ROW_WIDTH-1:0]` and `col[COL_WIDTH-1:0]` representations. This conversion is the only “interesting” combinational work in the LPDDR2 branch — the rest is just lookup table indexing.

The DDR2 strobes (`dfi_ras_n` / `dfi_cas_n` / `dfi_we_n`) are tied high in LPDDR2 builds. They are present at the DFI port (per HAS §2.4) because the same top-level header serves both memtypes, but they carry no information.

19.6 Per-Rank CS_n Drive (Shared Post-Branch Stage)

After the memtype-specific encoding, the shared stage drives `dfi_cs_n[NR]` per the issue’s `issue_rank` and `issue_op`:

```
// Single-rank commands (default for most opcodes):
for r in 0..NR-1:
    dfi_cs_n[r] = (r == issue_rank) ? 0 : 1

// Special cases:
```

```
NOP / no-issue:
    dfi_cs_n[*] = 1                  // no rank selected

REFab from refresh_mgr round-robin:
    dfi_cs_n[round_robin_rank] = 0
    dfi_cs_n[other ranks] = 1        // per HAS §3.4 v0.2 multi-rank
REFab

REFab from init (init-time global REF):
    dfi_cs_n[*] = 0                  // init can broadcast to all
ranks
```

The init-time `dfi_cs_n[*] = 0` case is the only opcode that drives multiple CS_n low simultaneously — and even that is only during init, when no other rank-targeted command is active.

The driver also coordinates with `odt_ctrl_fub` (see §2.16) which needs to know which rank is being targeted and what command type to apply the correct ODT termination rules. The `odt_ctrl` interface receives `issue_rank` and `issue_op` as inputs and computes `dfi_odt[NR]` independently.

19.7 Bypass-Path Sources

The encoder accepts commands from four upstream sources, multiplexed at the input boundary:

Source	Path	Active when
`scheduler_fub`	`sched_cmd_*` (normal traffic)	`init_in_progress == 0`
`init_engine_fub`	`init_cmd_*` (cold boot)	`init_in_progress == 1`
`refresh_mgr_fub`	`refresh_issue_*` (REF / REFpb)	Inside refresh-priority window
`power_state_fub`	`pwr_cmd_*` (SREFE, SREFX, DPDE, DPDX)	Power-state transitions

The input mux is priority-ordered (init > power > refresh > scheduler). At any given cycle exactly one strobe is asserted (the upstream FUBs coordinate to ensure this); the encoder treats input as a single combined `issue_*` bus and doesn't need to know which source.

19.8 Interface

19.8.1 Issue Input (combined from all upstream sources)

Signal	Direction	Width	Description
issue_strobe_i	input	1	A command is being issued this cycle
issue_op_i	input	4	Opcode (encoded per scheduler §2.7)
issue_rank_i	input	$clog2(NR)	Target rank
issue_bank_i	input	$clog2(NB)	Target bank (or MR index for MRS/MRW)
issue_row_i	input	ROW_WIDTH	Row operand (for ACT, MRS value also goes here)
issue_col_i	input	COL_WIDTH	Column operand (for RD/WR)
issue_auto_pre_i	input	1	Auto-pre flag (for RDA/WRA)
issue_all_bank_i	input	1	All-bank flag (for PREA / REFab)

19.8.2 DFI Command Outputs

Signal	Direction	Width	Description
dfi_cs_n_o[NR]	output	NR	Per-rank chip-select
dfi_ras_n_o	output	1	DDR2 RAS strobe (tied high for LPDDR2)
dfi_cas_n_o	output	1	DDR2 CAS strobe (tied high for LPDDR2)
dfi_we_n_o	output	1	DDR2 WE strobe (tied high for LPDDR2)
dfi_address_o	output	DFI_ADDR_WIDTH	DDR2: row/col operand; LPDDR2: packed 20-bit CA
dfi_bank_o	output	$clog2(NB)	DDR2: bank operand; LPDDR2: tied (bank is in CA bus)

19.8.3 ODT / Power Coordination (pass-through)

Signal	Direction	Width	Description
`dfi_cke_i[NR]`	input	NR	Pre-muxed CKE from CKE-routing topology (§2.13)
`dfi_cke_o[NR]`	output	NR	Pass-through to gear
`dfi_odt_i[NR]`	input	NR	From `odt_ctrl_fub`
`dfi_odt_o[NR]`	output	NR	Pass-through to gear

The CKE and ODT are pass-throughs here because they're already finalized upstream; the encoder isn't allowed to modify them.

19.8.4 Telemetry

Signal	Description
`dbg_last_op_o`	Last issued opcode — drives `STATUS.issue_op_obs`
`dbg_last_rank_o`	Last issued rank

19.9 Timing Budget

The encoder is single-cycle combinational from input to output. At the worst case (LPDDR2 ACT with full row-bit spreading across CA0+CA1), the path is:

Path	Levels (FPGA)	Budget
`issue_op_i` → opcode lookup table	2 LUT levels	0.4 ns
`issue_row_i / issue_col_i` → CA bit spreading network	3 LUT levels	0.6 ns
`issue_rank_i` + opcode → per-rank `dfi_cs_n_o`	2 LUT levels	0.4 ns
Routing / setup margin		0.3 ns
Total		**1.7 ns**

Comfortable for 500 MHz. The encoder is rarely on the critical path — that's dominated by the scheduler upstream.

19.10 CSR Hooks

CSR field	Source	Use case
`STATUS.issue_op_obs` (R)	`dbg_last_op_o`	What was the last DRAM command issued
`STATUS.issue_rank_obs` (R)	`dbg_last_rank_o`	What rank was last targeted

19.11 Verification Notes (cocotb test plan)

Scenario	What it proves
DDR2 build: ACT to bank 3 row 0x1234; `{ras_n, cas_n, we_n} = {0, 1, 1}`, dfi_address = 0x1234, dfi_bank = 3	DDR2 ACT encoding
DDR2 build: RDA to bank 3 col 0x040; A10 set; `{ras_n, cas_n, we_n} = {1, 0, 1}`	DDR2 auto-precharge via A10
DDR2 build: PREA; A10 set; dfi_bank = don't care	DDR2 all-bank precharge via A10
LPDDR2 build: ACT to bank 3 row 0x1234; CA0/CA1 packed per DFI v2.1 §3.1	LPDDR2 ACT encoding
LPDDR2 build: WR to bank 3 col 0x040 with auto-pre; AP bit set in CA0	LPDDR2 auto-precharge via CA bit
LPDDR2 build: REFpb to bank 3; CA encoding matches §3.1 table	LPDDR2 REFpb encoding
LPDDR2 build: REFab; CA encoding matches §3.1 table	LPDDR2 REFab encoding
Multi-rank: ACT to rank 1; `dfi_cs_n = {0, 1}` (rank 1 selected); rank 0 inactive	Per-rank CS_n drive
Multi-rank REFab from refresh_mgr round-robin: only target rank's CS_n low	Per-rank REFab dispatch
Init-time REF: `dfi_cs_n[*] = 0` (all ranks targeted)	Init broadcast CS_n
NOP cycle: `dfi_cs_n[*] = 1`	No-issue idle pattern
Two back-to-back commands in same MC cycle (impossible — single-issue scheduler)	Sanity: only one strobe asserted at a time

Scenario	What it proves
DDR2 build accidentally invokes REFpb opcode	Elaboration-time assertion fires

19.12 Open Questions / Future Work

- **LPDDR2 chip-id (multi-die) encoding.** LPDDR2 supports a chip-id bit for stacked-die parts. v1 doesn't expose this — the controller treats each rank as a single die. When DDR3+ family adds 3DS support, the encoder will need a `chip_id_i` input that goes into the CA bus encoding (per LPDDR2 §x.y). Punt to v2.
- **MRR (mode-register read) path.** LPDDR2 supports MRR for software to read DRAM state (temperature, ZQ status). v1 encodes MRR but the read-data path back through DFI is not in scope yet — needs `rd_data_path_fub` cooperation to deliver the MR value back to CSR. Add in v0.2 of MAS.
- **DDR2 OCD calibration commands.** DDR2 OCD (Off-Chip Driver) calibration uses extended MRS sequences. Not in v1; revisit when characterization on real DDR2 silicon flags drive-impedance issues.
- **Encoder LUT generation.** Currently the LPDDR2 CA-bus table is hand-coded from DFI v2.1 §3.1. Could be generated from a JSON/YAML spec file — easier to maintain when DFI v3+ changes the encoding (HAS v6.0 scope memory notes the changes). Punt; revisit at DDR3-LPDDR3 family controller.

---

20 DFI Gear (`gear_dfi_fub`)

Module: `gear_dfi_fub.sv` **Location:** `rtl/fub/` **Category:** FUB **Parent:** `ddr2_lpddr2_ctrl`
Status: Draft v0.1

> Architectural context: HAS §3.6 `gear_dfi`. This block is the gear-ratio packer: it absorbs the DFI per-phase dimension so upstream FUBs see

one command per MC clock and downstream DFI sees an N-phase frame per MC clock.

20.1 Purpose

DFI v2.1 runs at the PHY clock; the controller side runs at the MC clock, which is the PHY clock divided by N_PHASES. Each MC cycle therefore corresponds to N consecutive PHY-side DFI cycles ("phases"). gear_dfi_fub packs the controller's single-issue command stream and burst-of-N data beats into the right phase slots, and unpacks the rddata return into the controller's single-cycle view.

The FUB is purely combinational on the command path (cmd flows from cmd_encoder to phase-0 slot, idle to other slots) and lightly registered on the data paths (one MC cycle of pipeline latency for wr_data_path packing; one MC cycle for rd_data_path reconstruction).

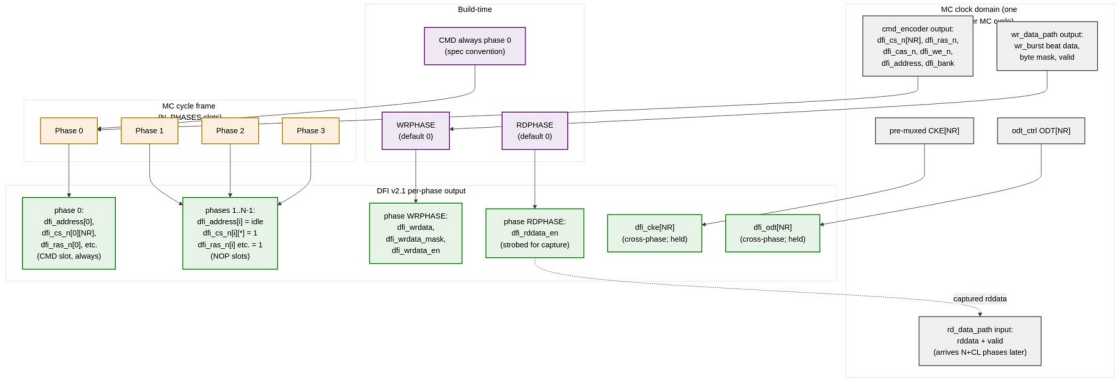
Upstream FUBs are gear-blind: scheduler, init, refresh, power, cmd_encoder all see one issue per MC clock. Downstream is the DFI pin layer where the per-phase signals route to the PHY.

20.2 Synthesis Parameters

Parameter	Source	Effect
N_PHASES	top (default 4)	Number of DFI phases per MC clock ({1, 2, 4})
WRPHASE	top (default 0)	Phase slot that carries dfi_wrdata, dfi_wrdata_en, dfi_wrdata_mask
RDPHASE	top (default 0)	Phase slot that strobes dfi_rddata_en
DFI_DATA_WIDTH	top	Per-phase DFI data width
AXI_DATA_WIDTH	top	Source data width (consumed by wr_data_path upstream)

The constraint WRPHASE < N_PHASES and RDPHASE < N_PHASES is checked at elaboration.

20.3 Frame Packing View



Gear Packing — per-phase slot layout, WRPHASE, RDPHASE, NOP fill

Source: 13_gear_dfi_packing.mmd

Each MC cycle produces one “frame” of N DFI cycles. The command always goes in phase 0 by spec convention (PHY samples cmds on phase 0); write data goes in `WRPHASE`; read-data-enable is strobed in `RDPHASE`.

20.4 Command Lane

The command from `cmd_encoder` is single-cycle-valid in the MC domain. The gear replicates each command signal across the per-phase output, **but only phase 0 carries the real command** — other phases drive idle (NOP) patterns:

```
for p in 0..N_PHASES-1:
    if (p == 0):
        // Phase 0 carries the actual command
        dfi_cs_n[p][r]   = cmd_encoder.dfi_cs_n_o[r]      // for all
ranks r
        dfi_ras_n[p]     = cmd_encoder.dfi_ras_n_o
        dfi_cas_n[p]     = cmd_encoder.dfi_cas_n_o
        dfi_we_n[p]      = cmd_encoder.dfi_we_n_o
        dfi_address[p]   = cmd_encoder.dfi_address_o
        dfi_bank[p]      = cmd_encoder.dfi_bank_o
    else:
        // Other phases: idle pattern (NOP-equivalent)
        dfi_cs_n[p][*]   = 1
        dfi_ras_n[p]     = 1
        dfi_cas_n[p]     = 1
        dfi_we_n[p]      = 1
        dfi_address[p]   = 0
        dfi_bank[p]      = 0
```

The PHY clock runs N× faster than the MC clock; the PHY samples commands on phase 0 each MC cycle and ignores the other phases (they're NOPs). This is the simplest gear and works at all N_PHASES values.

For `N_PHASES = 1` (no gear), there's only one phase and the command goes directly through with no replication.

20.4.1 Why Always Phase 0?

DDR2 / LPDDR2 PHYs typically sample command at phase 0 of the MC clock. Some PHYs allow command at other phases via a "CMDPHASE" parameter; the controller doesn't currently expose this since the bring-up board's a7ddrphy uses the conventional layout. If a future PHY requires it, add a `CMDPHASE` parameter symmetric with `WRPHASE`/`RDPHASE`.

20.5 Write Data Lane

When the controller issues a write, `wr_data_path_fub` produces N consecutive data beats over `BL/2` MC cycles (where BL = burst length, typically 4 or 8). The gear packs these beats into `WRPHASE` of each successive MC cycle:

```
// In wr_data_path's beat-per-MC-cycle output:
//   wr_beat_data, wr_beat_mask, wr_beat_valid

for p in 0..N_PHASES-1:
    if (p == WRPHASE):
        dfi_wrdata[p]       = wr_data_path.wr_beat_data
        dfi_wrdata_mask[p]  = wr_data_path.wr_beat_mask
        dfi_wrdata_en[p]    = wr_data_path.wr_beat_valid
    else:
        dfi_wrdata[p]       = 0
        dfi_wrdata_mask[p]  = 0      // mask all
        dfi_wrdata_en[p]    = 0
```

For a BL=4 burst at `N_PHASES = 4`, the burst fits in **one** MC cycle (4 phases × 1 beat = 4 beats). For BL=4 at `N_PHASES = 2`, the burst takes **2** MC cycles. For BL=4 at `N_PHASES = 1`, the burst takes **4** MC cycles.

The width-conversion from AXI's `AXI_DATA_WIDTH` to DFI's `DFI_DATA_WIDTH` happens upstream in `wr_data_path`; gear just routes pre-packed beats to the WRPHASE slot.

20.5.1 tCWL Latency Handling

The DRAM expects the first write data beat at `CWL` PHY cycles after the WR command. Since the command is on phase 0 and the data is on WRPHASE of a later MC cycle, the gear's "later MC cycle" must satisfy `((MC_cycle_delta × N_PHASES) + WRPHASE) == CWL`. The cmd_encoder's

WR command issue and the wr_data_path's data-push are aligned by the scheduler; gear just passes them through. Verification cross-checks this against the live `CL_CWL_WR` CSR values.

20.6 Read Data Lane

For reads, the gear has the inverse job: capture per-phase rddata returns from the PHY into a stream that `rd_data_path_fub` can consume one beat per MC cycle.

```
// Phase-by-phase rddata input from PHY:
for p in 0..N_PHASES-1:
    if (p == RDPHASE):
        rd_data_path.rd_beat_data  = dfi_rddata[p]
        rd_data_path.rd_beat_valid = dfi_rddata_valid[p]
    // Other phases are ignored (no real read data)

// Strobe rddata_en in RDPHASE on the cycle the data should be valid:
dfi_rddata_en[RDPHASE] = rd_strobe_at_CL    // computed from scheduler
issue + CL
```

The `dfi_rddata_en` is the strobe to the PHY saying "capture this DRAM data beat at this phase." It's asserted exactly `CL` PHY cycles after the corresponding RD command. The gear converts the MC-cycle-relative timing to phase-relative.

20.6.1 CL-Aware Pre-Drive of `dfi_rddata_en`

The gear has a small N-deep shift register that delays `rd_strobe_at_CL` by the appropriate number of phases. The strobe enters when the RD/RDA command issues (from cmd_encoder) and shifts out at the right cycle. The shift register is `(max_CL × N_PHASES)` deep — small (default 32 entries × 1 bit at 7-series).

20.7 Cross-Phase Signals: CKE and ODT

Some signals are held across the entire MC cycle (cross-phase): CKE and ODT in particular. The gear drives them identically on every phase:

```
for p in 0..N_PHASES-1:
    dfi_cke[p][r]  = cke_from_power_state[r]    // for all ranks r
    dfi_odt[p][r]  = odt_from_odt_ctrl[r]
```

These signals don't change within an MC cycle — they reflect the controller's CKE/ODT decision made at the start of the cycle. The PHY samples them at its own rate (DDR-style ODT timing) using the held value.

20.8 DFI v2.1 Status / Update Lane

The DFI status sub-interface (`dfi_init_complete`, `dfi_ctrlupd_req`, `dfi_phyupd_req`, etc.) is **not gear-packed** — it's a single-bit-per-event signal that crosses MC ↔️ PHY clock domains through standard CDC, independent of the phase. The gear passes them through without modification.

20.9 Interface

20.9.1 From `cmd_encoder_fub` (one cycle, MC clock)

Signal	Direction	Width	Description
`cmd_cs_n_i[NR]`	input	NR	Per-rank CS_n from cmd_encoder
`cmd_ras_n_i`	input	1	
`cmd_cas_n_i`	input	1	
`cmd_we_n_i`	input	1	
`cmd_address_i`	input	`DFI_ADDR_WIDTH`	
`cmd_bank_i`	input	`$clog2(NB)`	
`cmd_strobe_i`	input	1	1 when a command is being issued this cycle
`cmd_rd_strobe_i`	input	1	1 when the issue is a RD/RDA (for rddata_en shift register)
`cmd_cl_i`	input	4	CAS latency for this RD (snapshot at issue)

20.9.2 From `wr_data_path_fub` and `rd_data_path_fub`

Signal	Direction	Width	Description
`wr_beat_data_i`	input	`DFI_DATA_WIDTH`	One beat per MC cycle
`wr_beat_mask_i`	input	`DFI_DATA_WIDTH/8`	
`wr_beat_valid_i`	input	1	
`rd_beat_data_o`	output	`DFI_DATA_WIDTH`	Forwarded to rd_data_path

Signal	Direction	Width	Description
rd_beat_valid_o	output	1	

20.9.3 From power_state_fub and odt_ctrl_fub

Signal	Direction	Width	Description
cke_i[NR]	input	NR	Per-rank CKE (already-muxed init/normal)
odt_i[NR]	input	NR	Per-rank ODT

20.9.4 DFI Master Outputs (per-phase)

Signal	Direction	Width	Description
dfi_cs_n_o[NP][NR]	output	NP × NR	Per-phase per-rank CS_n
dfi_ras_n_o[NP]	output	NP	Per-phase RAS
dfi_cas_n_o[NP]	output	NP	Per-phase CAS
dfi_we_n_o[NP]	output	NP	Per-phase WE
dfi_address_o[NP]	output	NP × DFI_ADDR_WIDTH	Per-phase address operand
dfi_bank_o[NP]	output	NP × $clog2(NB)	Per-phase bank
dfi_wrdata_o[NP]	output	NP × DFI_DATA_WIDTH	Per-phase write data
dfi_wrdata_mask_o[NP]	output	NP × DFI_DATA_WIDTH/8	Per-phase byte mask
dfi_wrdata_en_o[NP]	output	NP	Per-phase write enable
dfi_rddata_en_o[NP]	output	NP	Per-phase read-data-enable strobe
dfi_cke_o[NR]	output	NR	Per-rank CKE (cross-phase)

Signal	Direction	Width	Description
dfi_odt_o[NR]	output	NR	Per-rank ODT (cross-phase)

20.9.5 DFI Read-Data Inputs (per-phase, from PHY)

Signal	Direction	Width	Description
dfi_rddata_i[NP]	input	NP × DFI_DATA_WIDTH	Per-phase read data from PHY
dfi_rddata_valid_i[NP]	input	NP	Per-phase rddata valid

20.10 Pipeline Staging

The command path is single-cycle combinational (zero gear pipeline latency). The write data path has one MC-cycle pipeline stage (the beat-data flop) so the AXI-data-width to DFI-data-width width-conversion in wr_data_path can register its output. The read data path is single-cycle from PHY input to rd_data_path output.

At N_PHASES = 4, DFI_DATA_WIDTH = 32, the per-phase data bus is 128 bits wide (4 × 32) on the gear output — manageable in 7-series routing. At N_PHASES = 4, DFI_DATA_WIDTH = 128, it's 512 bits — wider but still OK with appropriate placement.

20.11 CSR Hooks

CSR field	Source	Use case
STATUS.gear_dfi_wrphase_obs (R)	WRPHASE parameter echo	Software-visible build config
STATUS.gear_dfi_rdphase_obs (R)	RDPHASE parameter echo	Software-visible build config
STATUS.gear_dfi_n_phases_obs (R)	N_PHASES parameter echo	Software-visible build config

There are no runtime tunables — gear ratio is build-time only.

20.12 Verification Notes (cocotb test plan)

Scenario	What it proves
`N_PHASES = 1`: command and data on the single phase	No-gear smoke
`N_PHASES = 2`: command on phase 0, write data on phase 0 (`WRPHASE=0`)	2x gear baseline
`N_PHASES = 4, WRPHASE = 0`: BL=4 write completes in 1 MC cycle (4 beats × 1 phase)	Full-width gear
`N_PHASES = 4, WRPHASE = 2`: write data offset by 2 phases relative to cmd	Off-zero WRPHASE
Read with CL=5, N_PHASES=4: `dfi_rddata_en` asserts at the correct phase	CL-aware shift register
Read in flight; PHY drives rddata; gear forwards beat-per-MC-cycle to rd_data_path	Read data reconstruction
Command on phase 0, NOPs on other phases — PHY sees only the one cmd	Phase-replication idle correctness
Per-rank CKE held cross-phase	CKE pass-through
Per-rank ODT held cross-phase	ODT pass-through
`WRPHASE = N_PHASES` (illegal) at elaboration → assertion fires	Elaboration-time bounds check
Multi-rank: ACT on rank 1; phase 0 has `dfi_cs_n[0][1]=0`; other ranks have CS_n=1	Per-rank CS_n routes through phase 0

20.13 Open Questions / Future Work

- **Configurable CMDPHASE.** Currently command is always on phase 0. Some PHYs allow command on other phases. Worth a build parameter symmetric with WRPHASE/RDPHASE if the controller targets non-7-series PHYs. Punt; revisit at DDR3+ family.
- **Two-cycle command paths.** LPDDR2's 2-cycle CA-bus protocol is already absorbed in cmd_encoder (which emits a 20-bit dfi_address); the gear sees one MC-cycle frame. But if a future memtype has commands that span multiple MC cycles (rather than multiple phases within one MC cycle), the gear would need a multi-cycle replicator. Not in v1.

- **Late rddata capture.** If a PHY has variable rddata return latency (training-dependent), the controller may need a CL_SHIFT register driven by training rather than fixed CSR value. Today the shift-register depth is built from a CSR-loaded CL — works for fixed-latency PHYs only. Add at DDR3+ family controller.
- **N_PHASES = 8 or higher.** DDR3+ and LPDDR3+ support higher gear ratios. The current shift-register and replicator design scales linearly to N_PHASES = 8 without architectural changes; the routing buses just get wider. Validate when we get there.

---

21 ODT Control (odt_ctrl_fub)

Module: odt_ctrl_fub.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** ddr2_lpddr2_ctrl **Status:** Draft v0.1

> Architectural context: HAS §3.6 and the ODT_RULE_MULTIRANK parameter in HAS §5.2. This is the FUB where the famous "ODT-high on the non-accessed rank during read" JEDEC rule lives. It is also the single most multi-rank-specific block in the design — for NUM_RANKS=1, the FUB synthesizes to a trivial tie-off.

21.1 Purpose

odt_ctrl_fub produces the per-rank dfi_odt[NR] signals that drive each DRAM rank's On-Die Termination network. ODT is a JEDEC-mandated impedance-matching feature: a DDR2/3 DRAM device contains internal termination resistors that the controller selectively turns on to terminate the DQ/DQS bus at the appropriate end of a read or write.

The rule is asymmetric:

- **During a read**, the rank being read **drives** the DQ bus. *Other* ranks should drive their ODT high to terminate the bus from the controller side of the channel.

- **During a write**, the controller drives the DQ bus. The *target* rank drives ODT high to terminate at the receiving end.

For a single-rank point-to-point system (`NUM_RANKS=1`), ODT serves no purpose — there's nothing else on the bus — and the FUB ties `dfi_odt[0] = 0` always. For multi-rank, the JEDEC cross-termination rules apply.

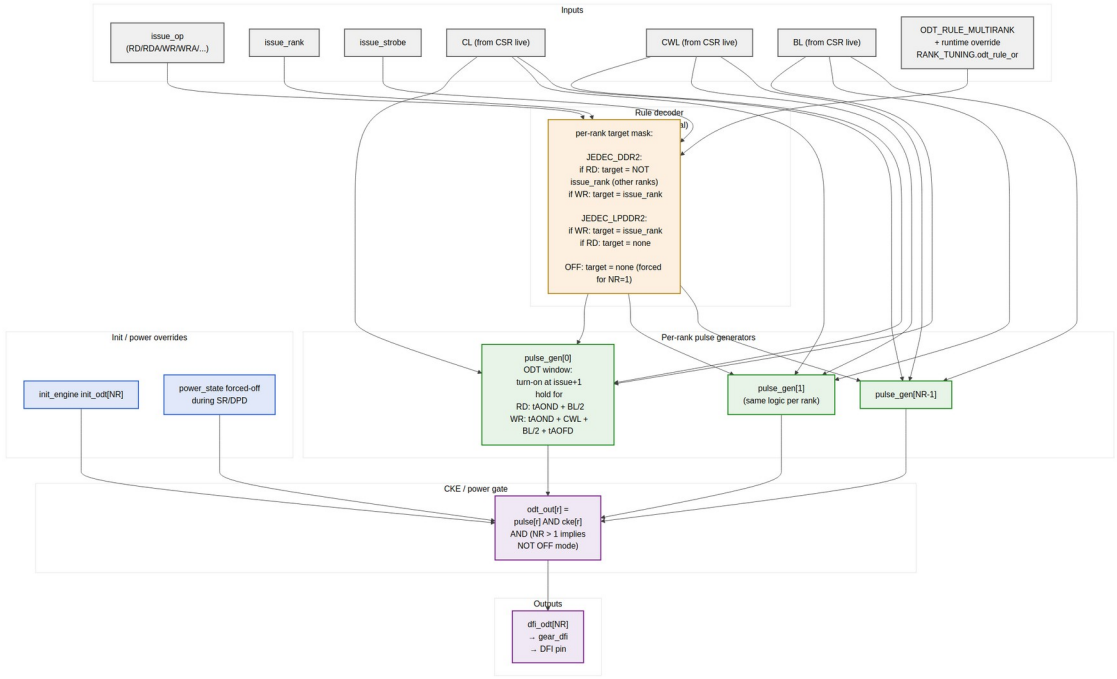
The FUB is implementation-light: a small combinational rule decoder feeding per-rank pulse generators that produce the timed ODT windows.

21.2 Synthesis Parameters

Parameter	Source	Effect
`NUM_RANKS`	top	Per-rank pulse generator fanout; `NUM_RANKS=1` collapses to tie-off
`ODT_RULE_MULTIRANK`	top	One of `JEDEC_DDR2`, `JEDEC_LPDDR2`, `OFF`; forced to `OFF` when NR=1
`MEMTYPE`	top	Picks DDR2 vs LPDDR2 turn-on/turn-off timings (tAOND/tAOFD)
`T_AOND_WIDTH`	derived	Width of the ODT turn-on delay counter (typically 3 bits)
`T_AOFD_WIDTH`	derived	Width of the ODT turn-off delay counter

The `ODT_RULE_MULTIRANK = OFF` mode is the forced value at single-rank — even if software writes `RANK_TUNING.odt_rule_or = JEDEC_DDR2`, the CSR slave returns `pslverr` because the option isn't available.

21.3 ODT Rules



ODT Rules — decoder, per-rank pulse generators, CKE / power gating

Source: 14_odt_ctrl_rules.mmd

21.3.1 JEDEC_DDR2 Rule (Default for Multi-Rank DDR2)

```
For each issued RD / RDA targeting rank R:
    For each rank r in 0..NR-1:
        if (r != R):
            odt_target[r] = 1     // ODT-high on non-accessed ranks
        else:
            odt_target[r] = 0     // accessed rank drives data, no ODT

For each issued WR / WRA targeting rank R:
    For each rank r in 0..NR-1:
        if (r == R):
            odt_target[r] = 1     // ODT-high on accessed rank
(receiver)
        else:
            odt_target[r] = 0     // other ranks idle
```

This is the textbook DDR2 multi-rank ODT pattern from JESD79-2 §x.y.

21.3.2 JEDEC_LPDDR2 Rule

LPDDR2 systems are usually single-rank point-to-point. The few multi-rank LPDDR2 systems use a simpler rule:

```
For each issued WR / WRA targeting rank R:
    odt_target[R] = 1                // ODT-high on accessed rank
    odt_target[r != R] = 0           // others idle

For each issued RD / RDA:
    odt_target[*] = 0                // no ODT during read on LPDDR2
(per LPDDR2 spec)
```

LPDDR2 multi-rank usage is uncommon enough that this rule is mostly a placeholder — the v1 controller defaults to JEDEC_DDR2 for either memtype when NR > 1; software can override to JEDEC_LPDDR2 if the board demands it.

21.3.3 OFF Rule

```
odt_target[*] = 0   always
```

Forced when `NUM_RANKS=1`. The FUB collapses to a tie-off — no pulse generators, no rule decoder, no flops.

21.4 Per-Rank Pulse Generators

Each rank gets a small pulse generator that converts the rule decoder's `odt_target[r]` into a properly-timed ODT window. The window for a DDR2 read on a non-accessed rank, for example, is:

```
turn-on at:    cycle (issue + tAOND)              // issue is the RD
command issue cycle
turn-off at:   cycle (issue + tAOND + BL/2)        // BL/2 = number of
beats over which the read drives DQ
```

For a DDR2 write on the accessed rank:

```
turn-on at:    cycle (issue + tAOND + CWL - 1)    // turn on 1 cycle
before first write beat
turn-off at:   cycle (issue + tAOND + CWL + BL/2 + tAOFD)
```

The pulse generator is implemented as a small shift register (depth = `max(CL, CWL) + BL/2 + tAOFD ~ 16 cycles`). When `odt_target[r] = 1` for the current issue, a "pulse plan" record (turn-on delay, hold duration) is loaded into the shift register; each cycle, the bit corresponding to "is the ODT window currently active" is read out.

21.4.1 Concrete Timing Example — DDR2-800, CL=5, CWL=5, BL=4

Cycle (from RD issue)	What's happening	ODT on non-accessed rank
0	RD issued by scheduler	0 (idle)
1	gear_dfi presents on phase 0	0
2	tAOND (2 cycles for DDR2-800)	1 (turn-on)
3	DRAM begins CL-counting	1
4	continue	1
5	continue	1
6	first data beat (CL cycles after issue)	1
7	data beat 2	1
8	data beat 3	1
9	data beat 4 (last for BL=4)	1
10	tAOFD (1 cycle for DDR2-800)	0 (turn-off)

So the ODT window is `cycles 2..9` (8 cycles total) for a CL=5 BL=4 read. The pulse generator handles this with a counter that loads at issue time and decrements per cycle.

21.5 Concurrent-Issue Edge Case

Per the scheduler design (§2.7), only one command issues per MC cycle. So at most one pulse-plan is loaded per cycle per rank. But the *window* of a previously-loaded pulse can overlap with a new issue:

- Cycle 0: RD to rank 0 issued → ODT-on rank 1 from cycle 2..9
- Cycle 4: RD to rank 1 issued → ODT-on rank 0 from cycle 6..13

In cycle 6..9, both ranks need ODT-on at the same time (the first read's window overlapping the second read's window). The per-rank pulse generators handle this independently — each has its own shift register and OR-aggregator. The final `dfi_odt[r]` is the OR of all in-flight pulses for that rank.

The shift register is sized to hold ~3 in-flight pulses per rank (rare for back-to-back same-rank-target reads; more common at high gear ratios with deep queues).

21.6 CKE / Power-State Gating

ODT is only valid when CKE is asserted — if a rank is in APD, SR, or DPD, its ODT should be off regardless of pulse-plan state. The output stage gates each pulse with the per-rank CKE:

```
dfi_odt_o[r] = pulse_active[r]
            AND cke_i[r]
            AND (ODT_RULE_MULTIRANK != "OFF")
```

When `power_state_fub` drops CKE on a rank, ODT for that rank goes to 0 immediately, even mid-pulse. This is correct — the DRAM stops listening to the command bus and won't see the ODT anyway.

21.7 Init-Time ODT

During init, the init engine drives `init_odt[NR]` directly (per the SET_CTRL_BITS opcode in §2.12). The output mux selects `init_odt[r]` when `init_in_progress == 1`:

```
dfi_odt_o[r] = init_in_progress ? init_odt_i[r] : (pulse_active[r] AND cke_i[r] AND ...)
```

Init typically holds ODT at a default (often 0) throughout the init sequence; the JEDEC init flow doesn't require ODT to be on during the MRS sequence.

21.8 Runtime Override

`RANK_TUNING.odt_rule_or` is the runtime override per HAS §5.4:

Value	Meaning
00	Use build-time `ODT_RULE_MULTIRANK` default
01	Force JEDEC_DDR2
10	Force JEDEC_LPDDR2
11	Force OFF (only valid when NR=1; else `pslverr` on write)

Override takes effect at the next quiet point (per §4.3). Writing an un-synthesized rule returns `pslverr` immediately on APB.

The override mux is a small 4:1 mux in the rule decoder:

```
active_rule = cfg_odt_rule_or == 00 ? ODT_RULE_MULTIRANK
            : cfg_odt_rule_or == 01 ? JEDEC_DDR2
            : cfg_odt_rule_or == 10 ? JEDEC_LPDDR2
                                    : OFF
```

21.9 Interface

21.9.1 Issue Input (from cmd_encoder bypass)

Signal	Direction	Width	Description
issue_strobe_i	input	1	A command is being issued this cycle
issue_op_i	input	4	Same opcode as cmd_encoder
issue_rank_i	input	$clog2(NR)	Target rank

21.9.2 CSR (live)

Signal	Direction	Width	Source
cfg_cl_i	input	4	TIMINGS_CL_CWL_WR.CL
cfg_cwl_i	input	4	TIMINGS_CL_CWL_WR.CWL
cfg_bl_i	input	4	Burst length (from MR0)
cfg_t_aond_i	input	T_AOND_WIDTH	ODT turn-on delay
cfg_t_aofd_i	input	T_AOFD_WIDTH	ODT turn-off delay
cfg_odt_rule_or_i	input	2	RANK_TUNING.odt_rule_or

21.9.3 CKE / Init Coordination

Signal	Direction	Width	Description
cke_i[NR]	input	NR	From power_state_fub (already-muxed with init)
init_in_progress_i	input	1	From init_engine_fub
init_odt_i[NR]	input	NR	From init_engine_fub (drives during init)

21.9.4 Outputs (to gear)

Signal	Direction	Width	Description
dfi_odt_o[NR]	output	NR	Final per-rank ODT to gear_dfi

21.9.5 Telemetry

Signal	Description
dbg_odt_pulse_active_o[NR]	Per-rank pulse-active state (for waveform)
dbg_odt_rule_active_o	Currently-active rule (2-bit, after override)

21.10 Multi-Rank Fan-Out and Area

Configuration	Per-instance area	Total area
NR=1 (tied off)	~0 LUTs	0 LUTs
NR=2, JEDEC_DDR2	~30 LUTs/rank + ~16 flops/rank	~100 LUTs
NR=4, JEDEC_DDR2	~30 LUTs/rank + ~16 flops/rank	~250 LUTs

The pulse-generator shift register dominates the per-rank flop count. The rule decoder is a small 4:1 mux per rank. Total at maximum config is ~0.5% of the controller's overall LUT budget — negligible.

21.11 CSR Hooks

CSR field	Source	Use case
STATUS.odt_rule_active (R)	dbg_odt_rule_active_o	Software check of effective rule
RANK_TUNING.odt_rule_or (R/W)	cfg_odt_rule_or_i	Runtime rule override
OBS_ODT_PULSE_PCT_R<R> (R)	Rolling fraction of cycles ODT-on per rank	Power telemetry

21.12 Verification Notes (cocotb test plan)

Scenario	What it proves
NR=1: `dfi_odt[0]` always 0 regardless of traffic	Tie-off behavior
NR=2 JEDEC_DDR2: RD to rank 0; rank 1 ODT-on during the read window	Cross-rank read ODT
NR=2 JEDEC_DDR2: WR to rank 0; rank 0 ODT-on during the write window	Same-rank write ODT
NR=2: back-to-back RD to rank 0 then rank 1; both ranks ODT-on during overlap	Concurrent pulses, OR aggregation
NR=4: RD to rank 1; ranks 0, 2, 3 ODT-on; rank 1 off	Multi-rank cross-termination
ODT turn-on at exactly `tAOND` cycles after issue; turn-off at `BL/2 + tAOFD`	Timing precision per JEDEC
Power-down on rank 1 during a RD to rank 0 → rank 1 ODT goes to 0 mid-pulse	CKE gating
Init sequence: `init_odt[NR]` drives output regardless of pulse state	Init bypass
Runtime override JEDEC_DDR2 → OFF; ODT goes to 0 at next quiet point	Runtime rule override
Runtime override to JEDEC_LPDDR2 (NR=2 LPDDR2 build): RD on rank 0 → no ODT	LPDDR2 rule
NR=1 + write `RANK_TUNING.odt_rule_or = JEDEC_DDR2` → APB `pslverr`	Single-rank override rejection
Multi-rank simulator: bring-up engineer captures `OBS_ODT_PULSE_PCT_R<R>`	Power-telemetry plumbing works

21.13 Open Questions / Future Work

- **ODT during precharge / refresh.** Currently the rule decoder only emits ODT pulses for RD / RDA / WR / WRA. During PRE, REF, ZQCS the ODT is off. This matches typical JEDEC interpretation, but some board designs want ODT held high during refresh to avoid bus floating. Could add a

RANK_TUNING.odt_during_refresh bit. Not in v1; revisit if bring-up flags signal-integrity issues.

- **Dynamic ODT (DDR3+).** DDR3 introduced Rtt_Wr (write ODT) vs Rtt_Nom (idle ODT), allowing different termination values during different windows. DDR2/LPDDR2 only has Rtt_Nom. The FUB has hooks for dynamic ODT (a 2-bit odt_level_o[NR] output is reserved) but the v1 controller doesn't drive them. Add in DDR3-LPDDR3 family controller.
- **ODT during write-leveling.** DDR3+ has a write-leveling phase where ODT is held in a specific pattern. DDR2 doesn't have write leveling, so this is a DDR3+ concern. Reserve the wl_odt_pattern_i input on this FUB for forward compat.
- **Per-pulse shift register depth tuning.** Currently sized for 3 concurrent pulses per rank. At very deep queues + high gear ratio, more could overlap. Worth a verification scenario to confirm the depth holds under stress; bump if it doesn't.

---

22 Write Data Path (wr_data_path_fub)

Module: wr_data_path_fub.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** ddr2_lpddr2_ctrl **Status:** Draft v0.1

> Architectural context: HAS §3.7. The micro-architecture closely mirrors the **stream** axi_write_engine (projects/components/stream/rtl/fub/axi_write_engine.sv): a streaming pipeline with **no FSM**, only flags, counters, and shift registers. State is implicit in pointers and inflight counts; sequence is enforced by data-flow handshakes, not by enumerated states.

22.1 Purpose

wr_data_path_fub moves write-data beats from the AXI4-slave's w_buf into DFI write-data form, aligned to the CWL window after the corresponding WR/WRA command issue. Width-

conversion (AXI `DATA_WIDTH` → `N_PHASES × DFI_DATA_WIDTH` per MC cycle) happens here; byte-strobe (`wstrb`) propagates with the data.

The FUB is **pulled by `wr_cmd_cam_fub`**, not by the scheduler. When the scheduler picks a write entry, the CAM's `beat_pull_*` interface begins requesting per-beat data from this FUB. The FUB walks the AXI W-buffer at the supplied pointer (`w_buf_ptr` + `beats_issued`), width-converts, and presents the result to `gear_dfi_fub` on the WRPHASE slot of the appropriate MC cycle.

There are no enumerated FSM states. Internal state is:

- A pointer that advances on each `beat_pull_strb`
- A shift register tracking how many beats have been pushed to DFI per inflight slot
- A small CL/CWL alignment shift register that times the first beat's appearance on `wr_beat_valid`

This is exactly the "streaming pipeline + flags" model from stream's `axi_write_engine`, adapted to the slave-side direction (AXI host → DRAM rather than internal SRAM → AXI memory).

22.2 Stream Uarch Heritage

This FUB is modeled after stream's `axi_write_engine` with the following deliberate parallels:

Property	Stream `axi_write_engine`	This FUB
No enumerated FSM	Yes — streaming flags only	Yes — streaming flags only
Streaming data path (no internal buffering)	Yes — passthrough SRAM → AXI W	Yes — passthrough w_buf → DFI wrdata
Per-transaction outstanding tracking	Per-channel inflight flags (V1) or counters (V2/V3)	Per-CAM-slot beat counter, in `wr_cmd_cam_fub` rather than here
Pre-allocation handshake	SRAM `wr_alloc_size` reserves space	`w_buf` allocation reserves space in `axi4_slave_fub` (upstream)
Combinational AR / W outputs	Yes (option to register for timing)	`wr_beat_*` outputs are registered (1-cycle pipeline stage)
Strobe-on-handshake	`done_strobe` on AW	`wr_beat_last` strobe on

Property	Stream `axi_write_engine`	This FUB
completion	handshake	last beat issued; CAM holds completion until `b_complete` from `xbank_timers`
Per-PERFORMANCE mode parameter	`PERFORMANCE` ∈ LOW/MED/HIGH	`WR_DATAPATH_PIPELINE` ∈ {0, 1} for v1; deeper modes deferred to v2

Where stream tracks per-channel state across `NUM_CHANNELS`, the DDR controller tracks per-CAM-slot state across `WR_CAM_DEPTH`. The “channel” / “slot” analogy is direct.

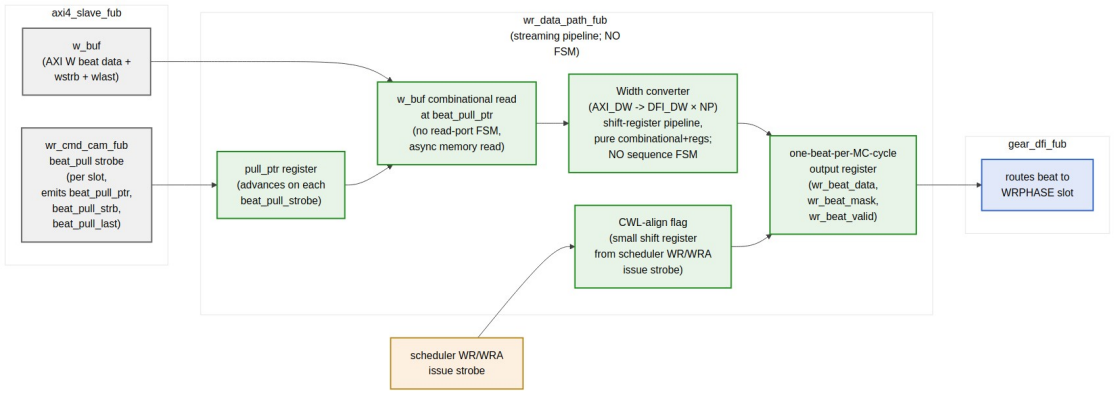
The most important constraint: **no FSMs.** When you see a state-like behavior described below (e.g., “first beat aligned to CWL”), it is implemented as a shift register, not as enumerated states. This keeps the FUB’s silicon footprint minimal and the timing well-behaved across the FPGA / ASIC range.

22.3 Synthesis Parameters

Parameter	Source	Effect
`AXI_DATA_WIDTH`	top	Source data width
`DFI_DATA_WIDTH`	top	Per-phase destination width
`N_PHASES`	top	Phases per MC cycle (1, 2, or 4)
`WR_CAM_DEPTH`	top	Number of inflight write slots tracked
`WR_DATAPATH_PIPELINE`	top (default 0)	0 = single-cycle (default); 1 = registered intermediate stage
`CWL_MAX`	derived	Max CAS Write Latency for the CWL-align shift register

`WR_DATAPATH_PIPELINE=0` matches stream’s V1 baseline — one MC cycle from `beat_pull_strb` to `wr_beat_valid` on gear. `=1` adds a register stage in the width-converter for 500 MHz close, costing one extra cycle of latency but adding zero throughput penalty (the pipeline is single-issue anyway).

22.4 Block Pipeline View



Write Data Path — streaming pipeline, no FSM, w_buf read + width conv + CWL align

Source: 15_wr_data_path_pipeline.mmd

22.5 Datapath Stages (all combinational unless noted)

22.5.1 Stage 1 — Beat Pull and W-Buf Read

On each MC cycle:

```
if (wr_cam.beat_pull_strb):
    pull_ptr_next = wr_cam.beat_pull_ptr     // not "current+1"; the
CAM supplies the absolute pointer
    strb_ptr_next = wr_cam.beat_pull_strb_ptr
    is_last       = wr_cam.beat_pull_last
    slot_active   = wr_cam.beat_pull_slot

w_buf_beat_data = w_buf[pull_ptr_next]       // combinational
distributed-RAM read
w_buf_beat_strb = wstrb_buf[strb_ptr_next]   // companion strobe
buffer
```

No state machine: `pull_ptr_next` is supplied by the CAM, so the FUB doesn't need to remember "where are we in the burst." The CAM owns burst-walk state (per §2.5); this FUB just reads at the supplied pointer.

22.5.2 Stage 2 — Width Conversion (AXI → DFI)

The conversion is a shift-register pipeline:

```
// Case 1: AXI_DATA_WIDTH == N_PHASES × DFI_DATA_WIDTH (typical)
//         1 AXI beat → 1 MC cycle of N_PHASES DFI beats; no further
conversion needed
wr_beat_data_next = w_buf_beat_data
wr_beat_mask_next = w_buf_beat_strb

// Case 2: AXI_DATA_WIDTH > N_PHASES × DFI_DATA_WIDTH
//         1 AXI beat → multiple MC cycles
// A small shift register (sub_beat_cnt) tracks which sub-beat is
being emitted
//   sub_beat_cnt steps from 0..K-1 where K = AXI_DW / (NP × DFI_DW)
//   No FSM — just a saturating counter that resets when the AXI beat
is fully drained

// Case 3: AXI_DATA_WIDTH < N_PHASES × DFI_DATA_WIDTH
//         Multiple AXI beats → 1 MC cycle
// An accumulator register concatenates K AXI beats into one MC frame
//   K = (NP × DFI_DW) / AXI_DW; same shift-register pattern, just
direction reversed
```

The width-conversion is the only place where any "state" exists, and even then it's a simple saturating counter (sub_beat_cnt) — no enumerated states, no transition table. The CAM's `beat_pull_strb` rate is matched to the conversion ratio so the upstream side automatically backpressures the right amount.

22.5.3 Stage 3 — CWL Alignment

The first beat to DFI must appear `CWL` PHY cycles after the scheduler's WR/WRA command issue. That's `ceil(CWL / N_PHASES)` MC cycles plus a phase-level adjustment.

A small CWL-align shift register absorbs the latency:

```
// On scheduler issue strobe (WR / WRA / init WR):
cwl_pipe.shift_in(slot, valid=1)

// Each MC cycle:
cwl_pipe.shift_left()

// At depth = cwl_in_mc_cycles, the slot index pops out → drive
wr_beat_valid for this slot
```

The shift register depth is `(CWL_MAX × N_PHASES + 1) / N_PHASES` ≈ 4 entries at default config. Per-entry width is `clog2(WR_CAM_DEPTH) + 1` (slot index plus valid bit) ≈ 5 bits at default. Total CWL-align shift register storage: ~20 flops. Trivial.

No FSM. The shift register *is* the timing alignment — the position in the shift register IS the cycle offset from issue.

22.5.4 Stage 4 — Output Register

```
wr_beat_data_o  ← wr_beat_data_next  (registered)
wr_beat_mask_o  ← wr_beat_mask_next  (registered)
wr_beat_valid_o ← cwl_pipe.head.valid (registered)
wr_beat_last_o  ← is_last AND cwl_pipe.head.valid (registered)
```

Per-MC-cycle output to `gear_dfi_fub`. The single register stage matches stream's V1 default for combinational AW output with optional intermediate register for timing.

22.6 Outstanding Tracking — Per CAM Slot

Stream's `axi_write_engine` tracks per-channel inflight state as a simple flag (PIPELINE=0) or counter (PIPELINE=1). Here, the per-burst tracking lives **in `wr_cmd_cam_fub` (§2.5)**:

What	Where	Stream analog
Burst-walk state (which beat next)	`wr_cmd_cam.beats_issued[slot]`	`r_w_drain_cnt[channel]` in axi_write_engine
Last-beat detection	`wr_cmd_cam.beat_pull_last`	`r_w_last_beat[channel]`
Inflight outstanding	`wr_cmd_cam.b_pending[slot]`	`r_b_outstanding[channel]`
B-response timing	`wr_cmd_cam` waits for `b_complete` from `xbank_timers`	`r_b_received` flag

The FUB itself stays stateless beyond the width-converter and CWL-align shift register. This is the deliberate parallel to stream — burst-walk state lives in the engine's "command queue" structure, the datapath itself is a streaming pipeline.

22.7 Width-Conversion Cases (Concrete Numbers)

AXI_DATA_WIDTH	DFI_DATA_WIDTH	N_PHASES	Conversion	Sub-beat counter width
64	32	2	1:1 (64 == 32×2)	none
64	16	4	1:1 (64 == 16×4)	none
128	32	4	1:1 (128 == 32×4)	none
128	16	4	1:2 (128 → 2 MC cycles of 16×4=64)	1 bit

AXI_DATA_WIDTH	DFI_DATA_WIDTH	N_PHASES	Conversion	Sub-beat counter width
256	32	4	1:2 (256 → 2 MC cycles of 32×4=128)	1 bit
64	32	4	2:1 (2 AXI beats accumulate → 1 MC cycle)	1 bit

The most common configs (left rows) need no sub-beat conversion at all — the width converter is a pure wire connection. The complexity only appears for asymmetric width combinations, and even then the sub-beat counter is a single-flop saturating counter.

22.8 Interface

22.8.1 From `wr_cmd_cam_fub` (the pull side)

Signal	Direction	Width	Description
`beat_pull_strb_i`	input	1	Pull a beat this cycle
`beat_pull_slot_i`	input	`$clog2(WR_CAM_DEPTH)`	Which CAM slot
`beat_pull_ptr_i`	input	`W_BUF_PTR_WIDTH`	Absolute pointer into w_buf
`beat_pull_strb_ptr_i`	input	`W_BUF_PTR_WIDTH`	Absolute pointer into wstrb buffer
`beat_pull_last_i`	input	1	This is the last beat of the burst

22.8.2 From `axi4_slave_fub` (the source buffer)

Signal	Direction	Width	Description
`w_buf_data_i`	input	`AXI_DATA_WIDTH × W_BUF_DEPTH`	The full w_buf as a wide bus (read-port to FUB)
`wstrb_buf_i`	input	`AXI_DATA_WIDTH/8 × W_BUF_DEPTH`	The companion strobe buffer

The `w_buf` is implemented as distributed RAM in `axi4_slave_fub`; the FUB reads it asynchronously at `pull_ptr`. This is the same pattern as stream's SRAM-to-engine streaming.

22.8.3 From `scheduler_fub` (CWL alignment)

Signal	Direction	Width	Description
`wr_issue_strobe_i`	input	1	Scheduler issued a WR/WRA this cycle
`wr_issue_slot_i`	input	`$clog2(WR_CAM_DEPTH)`	Which CAM slot

22.8.4 To `gear_dfi_fub`

Signal	Direction	Width	Description
`wr_beat_data_o`	output	`N_PHASES × DFI_DATA_WIDTH`	One MC-cycle frame's worth of write data
`wr_beat_mask_o`	output	`N_PHASES × DFI_DATA_WIDTH/8`	Companion byte mask
`wr_beat_valid_o`	output	1	Frame is valid this cycle
`wr_beat_last_o`	output	1	Last beat of the burst
`wr_beat_slot_o`	output	`$clog2(WR_CAM_DEPTH)`	CAM slot (for `xbank_timers` last-beat strobe)

22.8.5 To `xbank_timers_fub`

Signal	Direction	Width	Description
`wr_last_beat_o`	output	1	Used by `xbank_timers.tWTR_cnt` reload (§2.10)
`wr_last_rank_o`	output	`$clog2(NR)`	Rank of the burst (from CAM via slot lookup)

22.8.6 Debug

Signal	Description
`dbg_pull_ptr_o`	Current pull_ptr value
`dbg_cwl_pipe_depth_o`	CWL-align shift register fill level
`dbg_sub_beat_cnt_o`	Sub-beat counter (when width-conversion is active)

22.9 Timing Budget

Single-cycle path from `beat_pull_strb_i` to `wr_beat_data_o`:

Path	Levels (FPGA)	Budget
`beat_pull_strb_i` + `beat_pull_ptr_i` → w_buf combinational read	3 LUT levels	0.6 ns
w_buf read → width-converter mux	2 LUT levels	0.4 ns
Width converter → `wr_beat_data_next` to register	1 LUT level	0.2 ns
Routing / setup margin		0.3 ns
Total		**1.5 ns**

Closes at 500 MHz with slack at default config (when no asymmetric width conversion). At asymmetric configs (the 1:2 or 2:1 cases), the sub-beat-counter selection adds ~0.3 ns; still closes at 500 MHz. The `WR_DATAPATH_PIPELINE=1` build adds a register at the width-converter output for ASIC targets >500 MHz.

22.10 CSR Hooks

CSR field	Source	Use case
`STATUS.wr_beats_in_flight` (R)	Counter of pull_strb pulses minus last_beats	Live in-flight beat count
`OBS_AXI_W_LATENCY_AVG` (R)	Avg time from W intake to wr_beat_last	Telemetry for write latency

22.11 Verification Notes (cocotb test plan)

Scenario	What it proves
Single BL=4 write, AXI_DW = N_PHASES × DFI_DW (1:1)	Smoke: beats flow through with no width conversion
Single BL=8 write, 1:1 width	Multi-beat burst smoke
BL=4 write, asymmetric AXI_DW >	Sub-beat conversion correctness

Scenario	What it proves
N_PHASES × DFI_DW (1:2)	
BL=4 write, asymmetric AXI_DW < N_PHASES × DFI_DW (2:1)	Accumulating conversion correctness
First beat appears exactly CWL cycles after scheduler issue	CWL alignment shift register
Back-to-back writes to different CAM slots: each gets its own CWL alignment	Per-slot CWL pipe correctness
`WR_DATAPATH_PIPELINE=1`: same correctness, extra cycle of latency	Pipeline parameter
`wr_last_beat_o` strobes exactly once per burst, matching `xbank_timers.tWTR` reload	tWTR plumbing
Reset during a mid-burst pull: pull_ptr clears, no spurious wr_beat_valid	Reset behavior
`wr_beat_data_o` byte-mask matches `wstrb` from AXI W	Strobe propagation

22.12 Open Questions / Future Work

- **Multi-beat width conversion latency.** At `AXI_DW = 128, DFI_DW × NP = 64` (1:2 case), each AXI beat takes 2 MC cycles to emit. Could overlap with CWL alignment if multiple bursts are in flight; current design serializes per-burst. Verify with traffic mix and decide whether to add cross-burst pipelining (V2-style).
- **V2 / V3 performance modes.** Stream's V2 (command-pipelined) and V3 (OoO drain) would map to "interleave beats from multiple in-flight CAM slots." The CAM already supports multiple in-flight slots; the FUB would need a small per-slot beat-counter shadow rather than tracking the active slot. Punt to v2 of MAS.
- **Strobe-less writes.** Some hosts always write full beats and don't drive `wstrb`. Detecting this could let the strobe RAM and mask pipeline be omitted at elaboration. Minor area win; punt.
- **Skid buffer between this FUB and gear_dfi.** Currently the gear is expected to always accept the beat in the cycle it's offered. If the gear adds backpressure (it currently doesn't), a one-deep skid buffer would be

needed here. Confirm gear's no-backpressure invariant in a verification scenario.

23 Read Data Path (rd_data_path_fub)

Module: rd_data_path_fub.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** ddr2_lpddr2_ctrl **Status:** Draft v0.1

Architectural context: HAS §3.7. The micro-architecture closely mirrors the **stream** axi_read_engine (projects/components/stream/rtl/fub/axi_read_engine.sv): a streaming pipeline with **no FSM**, only flags, counters, and shift registers. Per-burst state is implicit in pointers and an ID-indexed inflight ring buffer; sequence is enforced by data-flow handshakes, not by enumerated states.

23.1 Purpose

rd_data_path_fub captures DRAM read-data beats arriving from gear_dfi_fub, width-converts them to AXI form, routes them to the correct AXI ID based on the in-flight CAM slot mapping, and presents them on the AXI R channel for axi4_slave_fub to drive back to the host.

CL (CAS Latency) alignment is the inverse of wr_data_path_fub's CWL alignment: when the scheduler issues a RD/RDA, the first beat will appear from DFI rddata exactly CL PHY cycles later. A small CL-aware shift register holds the slot index so it pops out at the right cycle to tag the arriving beats.

There are no enumerated FSM states. Internal state is:

- A small CL-align shift register that times slot-index handoff to the inflight ring buffer
- An ID-indexed ring buffer of (slot, beats_remaining) tuples for in-flight reads
- A width-conversion shift register pipeline

This is exactly the "streaming pipeline + flags" model from stream's `axi_read_engine`, adapted to the slave-side direction (DRAM → AXI host rather than memory → internal SRAM).

23.2 Stream Uarch Heritage

This FUB is modeled after stream's `axi_read_engine` with the following deliberate parallels:

Property	Stream `axi_read_engine`	This FUB
No enumerated FSM	Yes — streaming flags only	Yes — streaming flags only
Streaming data path	Direct passthrough AXI R → SRAM	Direct passthrough DFI rddata → AXI R
ID-based completion routing	`m_axi_rid` → channel ID	`dfi_rddata` arrival time → slot index via CL pipe; slot ID → AXI ID via CAM
Pre-allocation handshake	`rd_alloc_size` reserves SRAM space	rd_cmd_cam slot reservation in `axi4_slave_fub` (upstream)
Combinational AR / R outputs	Yes (option to register for timing)	`r_beat_*` outputs are registered (1-cycle pipeline stage)
Strobe-on-handshake completion	`done_strobe` on AR handshake	`entry_complete` to rd_cmd_cam on last beat
Out-of-order completion across IDs	Inherent — channel ID is part of AXI ID	Inherent — slot index travels with the data
Per-PERFORMANCE mode parameter	`PERFORMANCE` ∈ LOW/MED/HIGH	`RD_DATAPATH_PIPELINE` ∈ {0, 1} for v1; deeper modes deferred to v2

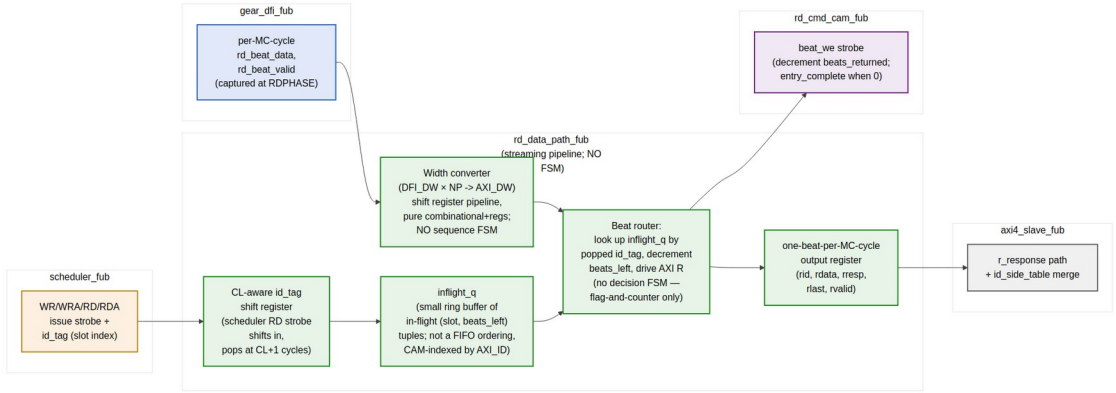
Where stream's read engine tracks per-channel inflight reads, this FUB tracks per-CAM-slot inflight reads. The slot index in `rd_cmd_cam_fub` is the analog of stream's channel ID.

No FSMs anywhere in this FUB. When the description below uses words like "first beat" or "burst completion" it does NOT mean an enumerated state — it's encoded in the inflight ring buffer's `beats_remaining` counter and the data-flow valid/last bits.

23.3 Synthesis Parameters

Parameter	Source	Effect
AXI_DATA_WIDTH	top	Output (destination) width
DFI_DATA_WIDTH	top	Per-phase source width
N_PHASES	top	Phases per MC cycle
RD_CAM_DEPTH	top	Number of inflight read slots tracked
RD_DATAPATH_PIPELINE	top (default 0)	0 = single-cycle; 1 = registered intermediate stage
CL_MAX	derived	Max CAS latency for the CL-align shift register

23.4 Block Pipeline View



Read Data Path — streaming pipeline, no FSM, CL-aware id tag + inflight ring + width unconv

Source: 16_rd_data_path_pipeline.mmd

23.5 Datapath Stages (all combinational unless noted)

23.5.1 Stage 1 — CL-Aligned Slot Tagging

When the scheduler issues RD/RDA, the slot index is shifted into a small CL-align register:

```
// On scheduler RD/RDA issue strobe:
cl_pipe.shift_in(slot, valid=1)

// Each MC cycle:
```

```
cl_pipe.shift_left()

// At depth = cl_in_mc_cycles + 1, the slot index pops out → enters
inflight_q
```

The shift register depth is `(CL_MAX × N_PHASES + 1) / N_PHASES` ≈ 4 entries at default config. Per-entry width is `clog2(RD_CAM_DEPTH) + 1` ≈ 5 bits. ~20 flops total.

No FSM — the position in the shift register IS the cycle offset from issue.

23.5.2 Stage 2 — Inflight Ring Buffer

When the CL pipe pops a slot, it's pushed into an ID-indexed inflight ring buffer:

```
struct inflight_t {
    logic valid;
    logic [$clog2(RD_CAM_DEPTH)-1:0] slot;
    logic [BURST_LEN_WIDTH-1:0]       beats_remaining;
};

inflight_t inflight_q[RD_CAM_DEPTH];        // CAM-indexed, NOT FIFO-
ordered

// On CL pipe pop:
inflight_q[popped_slot].valid           = 1
inflight_q[popped_slot].slot            = popped_slot
inflight_q[popped_slot].beats_remaining =
burst_len_from_cam[popped_slot]
```

The ring buffer is CAM-indexed by the slot number (which is, in turn, the AXI ID echo from the CAM). Because the CAM enforces "at most one entry per AXI ID at a time" (or `MAX_PER_ID_READS = N` per HAS §3.1), there's never a collision on `inflight_q[slot]`.

No FSM — each entry is just (valid, slot, counter), and the counter is decremented when a beat is consumed. When `beats_remaining` reaches 0, the entry's valid clears.

23.5.3 Stage 3 — Width Conversion (DFI → AXI)

The conversion is the inverse of `wr_data_path_fub` Stage 2:

```
// Case 1: AXI_DATA_WIDTH == N_PHASES × DFI_DATA_WIDTH (typical)
//         1 MC cycle of DFI beats → 1 AXI beat; pure wire connection
axi_beat_data = rd_beat_data

// Case 2: AXI_DATA_WIDTH > N_PHASES × DFI_DATA_WIDTH
//         K MC cycles → 1 AXI beat
// Accumulator concatenates K MC cycles before forwarding
```

```
// Case 3: AXI_DATA_WIDTH < N_PHASES × DFI_DATA_WIDTH
//         1 MC cycle → K AXI beats
// Shift register emits K sub-beats over K successive cycles
//   sub_beat_cnt is a saturating counter, NOT a state machine
```

Same sub-beat-counter pattern as the write path. No enumerated states.

23.5.4 Stage 4 — Beat Router and AXI R Drive

The beat router takes the converted data and looks up the destination slot from the inflight ring:

```
// On rd_beat_valid (from gear_dfi):
//   target_slot = head_of_cl_pipe                // the slot the CL
pipe is currently serving
//   data        = width_converted_beat
//   beat_last   = (inflight_q[target_slot].beats_remaining == 1)
//
//   inflight_q[target_slot].beats_remaining = beats_remaining - 1
//   if (beats_remaining was 1):
//     inflight_q[target_slot].valid = 0
//     entry_complete strobe to rd_cmd_cam for slot
//
// Drive AXI R outputs:
//   r_beat_id   = cam.axi_id[target_slot]
//   r_beat_data = data
//   r_beat_resp = OKAY (or SLVERR on DFI rddata error)
//   r_beat_last = beat_last
//   r_beat_valid = 1
```

The beat router is again **flag-and-counter only** — no state register, no transition table. The `beats_remaining` counter is the per-burst progress; the CAM slot is the AXI-ID anchor; the gear's `rd_beat_valid` is the data-flow handshake.

23.5.5 Stage 5 — Output Register

```
r_beat_id_o    ← r_beat_id    (registered)
r_beat_data_o  ← r_beat_data  (registered)
r_beat_resp_o  ← r_beat_resp  (registered)
r_beat_last_o  ← r_beat_last  (registered)
r_beat_valid_o ← r_beat_valid (registered)
```

Per-MC-cycle output to `axi4_slave_fub`'s `r_response_fifo`. The single register stage matches stream's V1 default.

23.6 Out-of-Order Completion (Inherent)

Stream's read engine handles OoO completion across channels because the AXI ID carries the channel index. This FUB handles OoO across AXI IDs because the CL pipe tags each in-flight read with its slot index *before* the data arrives.

The first beat of a read to AXI ID 0x3 might arrive in the same MC cycle as the last beat of an earlier read to AXI ID 0x7. The CL pipe sourced the slot indexes in the correct order; the inflight ring buffer tracks each independently; the AXI R channel returns each beat with its correct ID.

Per-ID ordering is preserved because each AXI ID has at most `MAX_PER_ID_READS` (typically 1 in v1) in flight, so beats to the same ID can never arrive out of order from the DRAM (a single in-flight read returns all its beats in burst order). Cross-ID ordering is *not* preserved — that's the whole point of OoO.

This matches the AXI4 ordering semantics: per-ID in-order, cross-ID OoO (when `AXI_OOO_ACROSS_IDS = true`).

23.7 CL Alignment Concrete Example

At DDR2-800, `CL = 5`, `N_PHASES = 4`:

MC cycle from RD issue	What's happening	CL pipe head
0	scheduler RD issued	empty
0	CL pipe.shift_in(slot=3)	slot=3
1	gear_dfi presents RD on phase 0	slot=3
2	tied to CL counter inside DRAM	slot=3
2 (shift_left)		slot=3 → next position
3	DFI rddata begins arriving	slot=3 → head
3	inflight_q[3].valid = 1; beats_remaining = burst_len	slot=3
3	first beat: drive r_beat_id = cam.axi_id[3], r_beat_valid = 1	
4	second beat	
5	third beat	

MC cycle from RD issue	What's happening	CL pipe head
6	last beat; beats_remaining → 0; inflight_q[3].valid = 0; entry_complete to rd_cmd_cam	

This is the rough timing — actual cycles depend on `CL`, `N_PHASES`, `BL`, and the gear's per-phase mapping. The CL pipe absorbs all of that complexity in its shift register depth.

23.8 Interface

23.8.1 From `scheduler_fub` (CL alignment intake)

Signal	Direction	Width	Description
`rd_issue_strobe_i`	input	1	Scheduler issued a RD/RDA this cycle
`rd_issue_slot_i`	input	`$clog2(RD_CAM_DEPTH)`	Which CAM slot
`rd_issue_burst_len_i`	input	`BURST_LEN_WIDTH`	Total beats for the burst (for inflight_q counter)

23.8.2 From `gear_dfi_fub` (per-MC-cycle beat input)

Signal	Direction	Width	Description
`rd_beat_data_i`	input	`N_PHASES × DFI_DATA_WIDTH`	One MC-cycle frame's worth of read data
`rd_beat_valid_i`	input	1	Frame is valid this cycle

23.8.3 From `rd_cmd_cam_fub` (slot → AXI ID lookup)

Signal	Direction	Width	Description
`cam_axi_id_i[RD_CAM_DEPTH]`	input	RD_CAM_DEPTH × AXI_ID_WIDTH	Per-slot AXI ID (combinational read)

23.8.4 To `rd_cmd_cam_fub` (entry-complete strobe)

Signal	Direction	Width	Description
`entry_complete_strb_o`	output	1	Last beat of a burst was just emitted
`entry_complete_slot_o`	output	`$clog2(RD_CAM_DEPTH)`	Which slot just completed

23.8.5 To `axi4_slave_fub` (AXI R-channel push)

Signal	Direction	Width	Description
`r_beat_id_o`	output	`AXI_ID_WIDTH`	rid
`r_beat_data_o`	output	`AXI_DATA_WIDTH`	rdata
`r_beat_resp_o`	output	2	rresp (OKAY / SLVERR)
`r_beat_last_o`	output	1	rlast
`r_beat_valid_o`	output	1	rvalid
`r_beat_ready_i`	input	1	rready (back-pressure from axi4_slave's r_response_fifo)

23.8.6 To `xbank_timers_fub`

Signal	Direction	Width	Description
`rd_last_beat_o`	output	1	Used by `xbank_timers.tRTW_cnt` reload (§2.10)
`rd_last_rank_o`	output	`$clog2(NR)`	Rank of the burst

23.8.7 Debug

Signal	Description
`dbg_cl_pipe_depth_o`	CL-align shift register fill level

Signal	Description
dbg_inflight_count_o	Number of in-flight reads (popcount of inflight_q.valid)
dbg_sub_beat_cnt_o	Sub-beat counter (when width-conversion is active)

23.9 Backpressure Handling

The AXI R-channel ready (r_beat_ready_i) is the only backpressure source. If the host's R-channel master stalls and the r_response_fifo fills, rd_data_path_fub cannot pull from gear_dfi. Currently the gear has no backpressure signal — it expects to deliver every rd_beat as it arrives from the PHY.

A small skid buffer (1-2 deep) inside this FUB absorbs the brief stalls between R-channel handshakes. For sustained backpressure, the architecture relies on the scheduler not issuing new RD commands when r_beat_ready_i has been low for R_STALL_THRESHOLD cycles — this hint flows back to the scheduler via the STATUS.axi_r_stall CSR observation.

23.10 CSR Hooks

CSR field	Source	Use case
STATUS.rd_beats_in_flight (R)	popcount over inflight_q.valid	Live in-flight burst count
STATUS.axi_r_stall (R)	r_beat_ready_i low for N cycles	Host backpressure indicator
OBS_AXI_R_LATENCY_AVG (R)	Avg time from rd_issue_strobe to first beat	AXI read latency telemetry
OBS_AXI_R_LATENCY_P99 (R)	99th percentile of the above	Tail latency

23.11 Verification Notes (cocotb test plan)

Scenario	What it proves
Single BL=4 read, AXI_DW = N_PHASES × DFI_DW (1:1)	Smoke: beats flow through with no width conversion

Scenario	What it proves
Single BL=8 read, 1:1 width	Multi-beat burst smoke
First beat arrives exactly CL cycles after scheduler issue	CL alignment shift register
OoO across IDs: read to ID 0x3 then read to ID 0x7; ID 0x7 returns first	OoO across IDs preserves rid tagging
Per-ID in-order: two reads to same ID complete in issue order	Per-ID ordering preservation
Asymmetric width AXI_DW > DFI_DW × NP: K-cycle accumulator drives 1 AXI beat	Accumulating width conversion
Asymmetric width AXI_DW < DFI_DW × NP: 1 MC cycle drives K AXI beats	Sub-beat width conversion
`entry_complete` to rd_cmd_cam fires exactly once per burst at last beat	CAM completion strobe
`rd_last_beat_o` to xbank_timers fires for tRTW reload	xbank tRTW plumbing
AXI R back-pressure (rready=0): skid buffer absorbs 1-2 beats	Backpressure smoke
DFI rddata error during burst (rresp = SLVERR): propagates to AXI R	Error propagation
Reset during in-flight read: inflight_q clears, no spurious r_beat_valid	Reset behavior
Concurrent: AR for ID 0x3 + last-beat of ID 0x7 + first beat of ID 0x5; all three correct in same cycle	Concurrent-ID throughput

23.12 Open Questions / Future Work

- **Skid buffer sizing.** Currently spec'd at 1-2 deep. Under heavy host backpressure, may need to be deeper. Verify with traffic mix; bump if needed.
- **V2 / V3 performance modes.** Stream's V2 / V3 modes for read engine include OoO completion (V3) which this FUB already supports inherently. The V2 "command-pipelined" mode is already the baseline here since CL

pipe is multi-deep. The MAS-level mode parameter is therefore vestigial; revisit at v0.2 of MAS.

- **DFI rddata error handling.** Currently `rresp = OKAY` always; the DFI rddata-error signal is not wired in v1. Add at the DFI status sub-interface level when bring-up flags PHY error conditions.
- **Read interleaving with refresh.** When a refresh interrupts in-flight reads (per §2.11 refresh handshake), the bank machine drains the read first before granting refresh. The inflight_q should naturally drain through the CL pipe; verify the timing in a concurrent refresh-during-read test.

24 AXI4 Slave Protocol

Per HAS §2.4 §1 for the full per-channel signal list. This chapter is the **wire-level contract** specific to this controller — what's supported, what's relaxed, what's omitted, and the timing / ordering / backpressure semantics the integrator can rely on.

For the FUB that implements this interface, see §2.2 (`axi4_slave_fub`).

24.1 Compliance Profile

Aspect	Support level
AXI4 (ARM IHI 0022) signal set	Full AW/W/B/AR/R
AXI4-Lite	Not supported (use APB CSR for control)
AXI4-Stream	Not applicable
AXI5	Not supported

24.2 Supported Features (v1)

- INCR burst type (mandatory)
- Burst length 1..256 beats

- Burst size $2^{0..2}$7 bytes
- ID-aware OoO completion (when `AXI_OOO_ACROSS_IDS = true`; default)
- Per-ID in-order completion (AXI4 spec mandate; always enforced)
- 4 KB burst boundary check (slave returns `SLVERR` on cross-4KB bursts)
- `awqos`/`arqos` observed and forwarded to scheduler (not yet used in v1 priority function)

24.3 Optional Features (build-time)

Feature	Parameter	Default
FIXED burst type	`SUPPORT_FIXED_BURST`	`false`
WRAP burst type	`SUPPORT_WRAP_BURST`	`false`
OoO across IDs	`AXI_OOO_ACROSS_IDS`	`true`

When `SUPPORT_FIXED_BURST = false`, an AW or AR with `awburst = 00` returns `SLVERR` on the corresponding B / R response.

24.4 Unsupported Features

- AXI4 exclusive accesses (`awlock`, `arlock` observed but ignored; behaves as normal access)
- AXI4 user signals (no `AWUSER`/`WUSER`/`BUSER`/`ARUSER`/`RUSER` ports)
- AXI4 cache-coherent extensions (`awcache`/`arcache` observed but no behavior)

24.5 Backpressure Semantics

The slave asserts backpressure via standard AXI handshake:

Channel	Backpressure reason
AW	`aw_buf` full in `axi4_slave_fub`, or `txn_queue` at `SCHED_TUNING.txn_queue_high_water`
W	`w_buf` full in `axi4_slave_fub`
AR	`ar_buf` full in `axi4_slave_fub`, or `txn_queue` at high water
B	Standard `bready` handshake; controller does not stall waiting for `bready`
R	Standard `rready` handshake; sustained stall triggers

Channel	Backpressure reason
	`STATUS.axi_r_stall` hint to scheduler (per §2.18)

The high-water threshold (`SCHED_TUNING.txn_queue_high_water`) is software-tunable to provide a few-burst headroom before hard backpressure. Default is `TXN_QUEUE_DEPTH - 2`.

24.6 Response Ordering

Two ordering rules apply:

1. **Per-ID in-order** — Always enforced. Two reads with the same ID complete in issue order; two writes with the same ID generate B responses in issue order.
2. **Cross-ID OoO** — Default on (`AXI_OOO_ACROSS_IDS = true`). Reads/writes with different IDs can complete in any order. When set to `false`, the slave enforces global in-order completion at the response side regardless of scheduler decisions.

Per HAS §3.1, even when scheduling is OoO at the DRAM layer, AXI response ordering is honored at the response side.

24.7 4 KB Burst Boundary

AXI4 specifies that a burst must not cross a 4 KB address boundary. The slave checks this at AW/AR intake:

```
boundary_cross = ( (addr & 0xFFF) + ((arlen+1) << arsize) ) > 0x1000
```

On detection, the slave still accepts the burst (the spec doesn't permit deassert of `arready`/`awready` mid-handshake on a valid request), but marks it for `SLVERR` response on B / R. The DRAM-side issue is suppressed for the offending burst.

24.8 Timing Contract

Path	Behavior
`awvalid` → `awready` accept	≤ 1 cycle when `aw_buf` has space
`awready` accept → CAM push	1 cycle
`aw_buf` → DRAM issue	Depends on scheduler / refresh state (typically 0–~256 cycles)
Last W beat → B response	tCWL + tWR window + scheduler backlog
`arvalid` → first R beat	DRAM access latency (typically 30–100 ns)

Path	Behavior
	+ bus rounding
Successive R beats (same burst)	1 per MC cycle (sustained streaming)

The slave does not register a registered output on its AXI ports by default. If timing closure requires it on a specific target, an external `axi_register_slice` from the AMBA library is the right answer; the controller deliberately does not embed one.

24.9 Per-ID Outstanding Limits

Limit	Parameter	Default
Total in-flight reads	`RD_CAM_DEPTH`	16
Total in-flight writes	`WR_CAM_DEPTH`	16
Per-ID in-flight reads	`MAX_PER_ID_READS`	4
Per-ID in-flight writes	`MAX_PER_ID_WRITES`	4

When `AXI_OOO_ACROSS_IDS = false`, per-ID limits reduce to 1 (the slave enforces strict in-order across all IDs at the response side).

24.10 Error Responses

Condition	Response	Hint to software
Cross-4 KB burst	`SLVERR`	None — software bug
FIXED burst when `SUPPORT_FIXED_BURST = false`	`SLVERR`	Recompile or rework software
WRAP burst when `SUPPORT_WRAP_BURST = false`	`SLVERR`	Recompile or rework software
Address outside `AXI_ADDR_WIDTH` range	`SLVERR`	None — software bug
DFI rddata-error from PHY (rare)	`SLVERR`	`irq_overflow` asserts
Init not done	`SLVERR`	Poll `STATUS.init_done` before traffic

DECERR is never returned — the slave's address space is monolithic.

24.11 Open Questions / Future Work

- **QoS-driven priority.** `awqos`/`arqos` are observed but not yet used in the scheduler. v2 would feed them into the Stage-2 priority mask (see §2.7).

- **AXI4 user signals.** Some SoCs use USER signals for sideband info (security tags, cache hints). If the SoC needs them, add `USER_WIDTH` parameters and forward through the id_side_table.
- **AXI register slice option.** A build-time `INSERT_AXI_REGSLICE` parameter could embed register slices at the AXI boundary for very-high-frequency targets. Not in v1.

---

25 DFI v2.1 Master Protocol

Per HAS §2.4 §2 for the full per-phase signal list and HAS §3.6 for the architectural view. This chapter is the wire-level contract specific to this controller — what's driven, what's tied, what's sampled, what's deferred.

For the FUBs that implement this interface, see §2.14 (`cmd_encoder`), §2.15 (`gear_dfi`), §2.16 (`odt_ctrl`), §2.17 (`wr_data_path`), §2.18 (`rd_data_path`).

25.1 DFI Spec Reference

Canonical: DFI 2.1 specification (Denali cleartext copy at `/home/seang/github/cold_storage/MemorySpecs/`). The controller's implementation deliberately covers the subset of DFI 2.1 sub-interfaces needed for DDR2/LPDDR2 operation; higher-version sub-interfaces are not driven.

25.2 Sub-Interface Support Summary

Sub-Interface	Direction	Supported in v1?	Notes
Command	Controller → PHY	Yes	Per-phase, see HAS §2.4 §2
Write-Data	Controller → PHY	Yes	`WRPHASE` slot per MC cycle
Read-Data	PHY →	Yes	`RDPHASE` slot capture

Sub-Interface	Direction	Supported in v1?	Notes
	Controller		
Status	PHY → Controller	Yes (limited)	`dfi_init_complete` only
Update	Bidirectional	Yes	`ctrlupd_*` and `phyupd_*` per spec
Training	Bidirectional	No	Not required for DDR2/LPDDR2 v2.1; deferred
Frequency Change	Bidirectional	No	No use case in v1
Low-Power	Controller → PHY	No	CKE is used directly; no PHY-side coordination needed
Error	PHY → Controller	No (placeholder)	`dfi_error` is sampled but ignored in v1
CRC	Both	No	Not in DDR2/LPDDR2

25.3 Command Sub-Interface

25.3.1 Phase Topology

Per HAS §2.4 §2, the command bus is replicated `N_PHASES` times. Per HAS §3.6 and MAS §2.15, phase 0 always carries the actual command; phases 1..N-1 drive NOP-equivalent idle.

25.3.2 Per-Rank Chip-Select

`dfi_cs_n[NP][NR]` is two-dimensionally indexed in SystemVerilog — `NP` outer (phase), `NR` inner (rank). Concretely at `N_PHASES=4, NUM_RANKS=2`:

```
output logic [3:0][1:0] dfi_cs_n;   // [phase][rank]
```

The controller drives exactly one rank's CS_n low per single-rank command, except for init-time REF (which broadcasts) and DDR2 multi-rank REFab (which uses round-robin per HAS §3.4 v0.2). See §2.14 for the per-rank drive logic.

25.3.3 DDR2 Strobes in LPDDR2 Builds

`dfi_ras_n`, `dfi_cas_n`, `dfi_we_n` are present in the port list in all builds (per HAS §2.4 — same top-level port set for both memtypes). In LPDDR2 builds they tie high; the PHY ignores them.

25.4 Write-Data Sub-Interface

The controller drives `dfi_wrdata_en` exactly `tCWL` PHY cycles after the corresponding WR/WRA command. CWL alignment is handled by `wr_data_path_fub` (§2.17) via its CWL-align shift register — the FUB does not need to know the absolute PHY cycle.

25.4.1 Per-Phase Mask

`dfi_wrdata_mask[p]` is wide enough to address each byte lane of `dfi_wrdata[p]`. A mask bit of 1 means "do not write this byte" (per DFI convention, matches AXI `wstrb = 0`).

25.4.2 Per-Rank `dfi_wrdata_cs_n`

DFI 2.1 includes a per-rank chip-select on the write-data path. The controller drives it from the same source as the command-side `dfi_cs_n` for the cycle the write data is presented on (i.e., one phase, post-CWL alignment).

25.5 Read-Data Sub-Interface

The controller asserts `dfi_rddata_en` exactly `tCL` PHY cycles after the corresponding RD/RDA command. The PHY's `dfi_rddata` and `dfi_rddata_valid` arrive shortly thereafter; `rd_data_path_fub` (§2.18) captures them through its CL-aware shift register.

25.5.1 Per-Rank `dfi_rddata_cs_n`

Symmetric to the write side. Drives which rank is being read.

25.5.2 Read-Data Error Handling

`dfi_rddata` carries no error bit in DFI 2.1. If the PHY indicates a corrupted read, it does so via `dfi_error` (currently sampled but not acted upon) or via `dfi_rddata_valid` not asserting. The controller treats a missing valid as a soft error and propagates `SLVERR` on the corresponding AXI R response.

25.6 Status Sub-Interface

`dfi_init_complete` is the only status signal sampled in v1. It's used by `init_engine_fub` (§2.12) during the cold-boot sequence — the `WAIT_FOR_BIT dfi_init_complete` step polls this until the PHY signals init complete or the timeout fires.

`dfi_dram_clk_disable` is driven during power-down to enable PHY clock gating; this is the PHY-side companion to `dfi_cke`.

25.7 Update Sub-Interface

The controller and PHY can each request a "controller-update" or "phy-update" window. Currently:

- **`dfi_ctrlupd_req`/`dfi_ctrlupd_ack`** — driven by `power_state_fub` (§2.13) at quiet points. The controller request is rare (mostly during init for PHY calibration handoff).
- **`dfi_phyupd_req`/`dfi_phyupd_ack`** — accepted at any quiet point. The controller grants by stalling new traffic until ack is dropped.

The `dfi_phyupd_type` is observed but not currently used to gate behavior.

25.8 CDC Topology

There is **no CDC** in the DFI interface — the controller drives DFI on `mc_clk`, and the PHY is responsible for any CDC to the DRAM clock. This is the canonical DFI architecture and is what `a7ddrphy` (LiteDRAM's Xilinx 7-series PHY) does.

25.9 Pin Tie-Offs

Pin	Tie-off (when applicable)
`dfi_ras_n`, `dfi_cas_n`, `dfi_we_n`	Tied high in LPDDR2 builds
`dfi_bank` (LPDDR2 builds)	Tied 0 (bank is in CA bus)
`dfi_freq_*`	Tied (no frequency change support)
`dfi_lp_*`	Tied (no low-power sub-interface)

25.10 Open Questions / Future Work

- **`dfi_error` handling.** Currently sampled and ignored. A v2 enhancement would wire it to `irq_overflow` and propagate `SLVERR` on the affected burst.
- **DFI Training Sub-Interface.** Required for DDR3+ at higher data rates. The v1 controller defers this entirely to the PHY (`a7ddrphy` handles its own training at startup, before init).
- **Per-rank `dfi_*_cs_n` width sanity check.** Some PHYs expect `dfi_cs_n` as a flat NR-bit vector per phase, others as a per-phase array. The controller produces the per-phase array form; a wrapper may be needed at the controller↔️PHY boundary for specific PHYs.
- **DFI 3+ migration.** When the DDR3-LPDDR3 family controller arrives, sub-interfaces grew from 6 → 14 (per memory `project_dfi_v6_scope_changes.md`). This MAS will need a new dedicated DFI section. Track in DDR3-LPDDR3 MAS, not here.

26 APB CSR Slave Protocol

Per HAS §2.4 §3 for the per-signal port list and HAS §4.3 for the architectural view. This chapter is the **wire-level contract** for the APB CSR slave specific to this controller.

The CSR is implemented entirely inside `csr_apb_fub`. The CDC topology is documented in MAS §1.3 and the CKE/quiet-point coordination is in §2.13.

26.1 Compliance Profile

Aspect	Support level
APB3 (ARM IHI 0024B)	Full — `psel`, `penable`, `pwrite`, `paddr`, `pwdata`, `pready`, `prdata`, `pslverr`
APB4 `pstrb` (byte enables)	Optional, `SUPPORT_APB_PSTRB = false` default
APB4 `pprot`	Not supported

26.2 Address Space

The slave decodes a 4 KB region (12-bit `paddr`). All registers are 32-bit, naturally aligned on 4-byte boundaries.

Address range	Region
0x000 – 0x00F	Control + Status
0x010 – 0x01F	Timing parameters
0x020 – 0x02F	Mode Register values
0x030 – 0x03F	LPDDR2-specific (PASR, temperature)
0x040 – 0x04F	Scheduler / Refresh / Page tuning
0x050	Init tuning
0x080 – 0x0FF	Per-(rank, bank) observation

Address range	Region
0x100 – 0x1FF	System observation / telemetry
0xFF0 – 0xFFC	Module identification + capability

Detailed register map is in §4.2; the full bit-level layout is in HAS §6.3.

26.3 Behavior

26.3.1 Access Cycles

Standard APB3:

```
SETUP:   psel=1, penable=0   → slave samples paddr/pwdata/pwrite
ENABLE:  psel=1, penable=1   → slave returns pready + prdata + pslverr
IDLE:    psel=0              → no transaction
```

`pready` is asserted in the ENABLE phase. The slave does not insert wait states in v1 — all transactions complete in 2 cycles.

26.3.2 Sub-32-Bit Accesses

All registers are 32-bit. The slave does not support sub-word access:

- APB3 (without `pstrb`): all 32 bits are read or written
- APB4 with `pstrb`: when `SUPPORT_APB_PSTRB = true`, byte-strobe is honored; `pstrb = 0` returns `pslverr`

26.3.3 Error Conditions

The slave asserts `pslverr` (in the ENABLE phase) for:

Condition	Notes
Read from reserved address (gaps in the address space)	Returns `pslverr`; prdata is 0
Write to read-only register	Returns `pslverr`; register unchanged
Write to ranks beyond `NUM_RANKS` (PASR / observation)	Returns `pslverr`
Write to unsupported address-map scheme (per HAS §6.3 ADDR_MAP_TUNING)	Returns `pslverr`
Write to ODT_RULE_OR with un-synthesized	Returns `pslverr`

Condition	Notes
rule	
Init not done, write to traffic-blocking control bit	Honored normally — software may want to pre-load

26.4 CDC Topology

The APB slave is the **only** FUB in the `apb_pclk` domain. The CDC into the controller's `mc_clk` domain happens inside `csr_apb_fub` per MAS §1.3:

- Write side: per-field 4-flop synchronizer with edge-detect strobe into the MC-side staging register
- Read side: per-counter pulse-sync + Gray code

The CDC is transparent to APB software; reads and writes complete in the standard 2-cycle APB access.

26.5 Runtime Override Semantics

Runtime override fields commit at the next quiet point (see §4.3). Software sequence:

1. Write the new value to the staging field via APB (returns immediately)
2. Write `CTRL.config_apply = 1` to request immediate commit (or skip and wait for natural quiet point)
3. Poll `STATUS.config_settled`
4. The new value is now live in the MC-side datapath

See §4.3 for full sequencing detail.

26.6 Performance

Operation	Latency
Single APB read	2 PCLK cycles
Single APB write	2 PCLK cycles
Read with CDC counter sync	2 PCLK cycles (sync transparent — the MC-side counter is sampled at APB read time)
Write + immediate commit	2 PCLK + < 256 MC cycles drain to quiet point
Continuous read polling	One transaction every 2 PCLK

Operation	Latency
	cycles (50 MHz PCLK → 25 M transactions/s)

26.7 Open Questions / Future Work

- **APB4 `pprot` support.** Some SoCs use it for security tagging. Add as `SUPPORT_APB_PPROT` parameter if needed.
- **Burst reads.** Multiple consecutive `paddr` reads currently take 2 PCLK each. Some APB masters support fast-path burst reads. Punt unless software flags it.
- **CDC-bypass for fast register access.** When `apb_pclk == mc_clk`, the CDC adds unnecessary latency. A `BYPASS_CDC` parameter could detect same-domain operation. Punt.

27 Register Map

The full bit-level register map is in HAS §6.3. This MAS chapter is the **implementation-level** view: how the registers are generated, where the source of truth lives, how staging-vs-live works, and how the per-rank generation handles multi-rank.

27.1 Source of Truth

The CSR map is generated from `regs/ddr2_lpddr2_csrs.yaml` (PeakRDL-style YAML). The generator produces:

File	Consumer
`regs/ddr2_lpddr2_csr_pkg.sv`	SV package: offsets, field masks, reset values
`regs/ddr2_lpddr2_csr_block.sv`	The wrapped register file (used inside `csr_apb_fub`)
`regs/ddr2_lpddr2_csrs.h`	C header for SoC software

File	Consumer
`regs/ddr2_lpddr2_csr_block_uvm.sv`	UVM register model for verification
`regs/ddr2_lpddr2_csrs.json`	Machine-readable spec for documentation tools

The HAS-side bit-level table in HAS §6.3 is **derived** from the YAML — it is not the source of truth. If the YAML and HAS disagree, the YAML wins (and a CI check should catch the disagreement).

27.2 Staging vs. Live State

Every R/W field has two storage cells:

1. **APB-side staging register** — written immediately on the APB write, lives in `apb_pclk` domain
2. **MC-side live register** — copied from staging at the next quiet point, lives in `mc_clk` domain

The staging→live commit takes one MC-clock edge. Reads can return either side depending on the field type:

Field type	Read returns
Read-only observation	Live MC-side counter (Gray-synchronized to APB side)
R/W configuration (`*_TUNING`)	Staging by default; switches to live after `CTRL.config_apply` is acknowledged
Read-only echo / capability	Constant value from build parameters

The two-cell architecture means software can read back the value it just wrote (from staging) before the commit completes — useful for sanity-checking the write took effect.

27.3 Per-Rank Register Generation

The YAML uses a `repeat` clause to instantiate per-rank registers. Example:

```
PASR_BANK_MASK_RANK:
  offset: 0x030
  repeat: NUM_RANKS
  stride: 0x004
  access: rw
  fields:
    pasr_banks:
```

```
      bits: 7:0
      reset: 0
```

At NUM_RANKS=1 this produces only PASR_BANK_MASK_RANK0 at 0x030. At NUM_RANKS=4 it produces PASR_BANK_MASK_RANK0..3 at 0x030, 0x034, 0x038, 0x03C.

The generator emits a pslverr decoder for ranks beyond NUM_RANKS-1. The HAS-side register map shows the maximum-config layout; software queries the 0xFF8 capability vector for the actual rank count.

27.4 Per-(Rank, Bank) Register Generation

The observation registers in 0x080–0x0FF use a 2-D repeat:

```
OBS_ROW_HIT_RANK_BANK:
  offset: 0x080
  repeat_outer: NUM_RANKS
  repeat_inner: NUM_BANKS
  outer_stride: NUM_BANKS * 4
  inner_stride: 0x004
  access: ro
  fields:
    row_hit_count:
      bits: 31:0
      source: scheduler.row_hit_ctr[r][b]
```

The 2-D layout means at NR=1, NB=8 the registers consume 0x080–0x09C (8 × 4 bytes). At NR=4, NB=8 they consume 0x080–0x0FF (32 × 4 bytes — saturates the available window).

If a future build needs NR > 4, NB > 8 (256 registers), the window needs to grow — there's a 0x180 spare slot available.

27.5 Field Source Attribution

Each R/W field's MC-side live value drives a specific FUB input (per §1.3 and the per-FUB CSR sections):

CSR field	Drives
SCHED_TUNING.force_inorder	scheduler_fub.cfg_force_inorder_i
SCHED_TUNING.lookahead_active	scheduler_fub.cfg_lookahead_active_i
REFRESH_TUNING.refresh_defer_active	refresh_mgr_fub.cfg_refresh_defer_active_i
REFRESH_TUNING.zqcs_freq_hz	refresh_mgr_fub.cfg_zqcs_freq_hz_i
ADDR_MAP_TUNING.scheme_or	addr_mapper_fub.scheme_active_i
ADDR_MAP_TUNING.xor_seed_runtim	addr_mapper_fub.xor_seed_i

CSR field	Drives
e	
RANK_TUNING.odt_rule_or	odt_ctrl_fub.cfg_odt_rule_or_i
POWER_TUNING.apd_idle_threshold	power_state_fub.cfg_apd_idle_threshold_i
… (many more) …	per-FUB CSR sections

This attribution is also generated from the YAML — the `source:` field per CSR field tells the generator which SV instance hierarchy receives the value.

27.6 Reset Values

Field type	Reset value source
Configuration (`*_TUNING`)	YAML `reset:` field; populated to match build defaults
Observation (`OBS_*`)	0
Build-time echo (`*_obs`)	Build parameter value
Capability vector (`0xFF8`)	Derived from build-time config

The reset values must match the synthesized defaults so that “do nothing” software sees the controller’s intended baseline behavior.

27.7 Generation Pipeline

```
ddr2_lpddr2_csrs.yaml
        |
        v
peakrdl-csr-gen.py    (or equivalent)
        |
        +--> ddr2_lpddr2_csr_pkg.sv
        +--> ddr2_lpddr2_csr_block.sv      <-- instantiated in
csr_apb_fub
        +--> ddr2_lpddr2_csrs.h            <-- consumed by SoC
firmware
        +--> ddr2_lpddr2_csr_block_uvm.sv  <-- consumed by UVM tests
        +--> ddr2_lpddr2_csrs.json         <-- consumed by doc-gen
```

The generator is invoked by the FUB filelist build (`rtl/filelists/fub/csr_apb.f`); the `ddr2_lpddr2_csr_block.sv` is a build artifact, not a checked-in source file. Changes to the YAML trigger regeneration on next build.

27.8 Open Questions / Future Work

- **YAML schema versioning.** When the YAML schema changes (e.g., DDR3+ adds new fields), need a version bumper. Add `schema_version: 1.0` field to track.
- **HAS §6.3 ↔️ YAML CI drift check.** Currently the HAS bit-level tables are hand-maintained from the YAML. A CI check that compares could catch drift. Punt to verification setup.
- **Field-level access timestamps.** When debugging "what wrote this register?", a per-field last-write-timestamp would be useful. Adds area; not in v1.

---

28 Runtime Overrides and Quiet Points

Per HAS §5.1 for the build-time-vs-runtime principle and HAS §6.3 for the field-level register map. This MAS chapter is the **implementation protocol** for how runtime overrides actually commit to the live datapath.

Quiet-point detection lives in `power_state_fub` (§2.13); the override staging registers and APB↔️MC CDC live in `csr_apb_fub` (referenced throughout §2).

28.1 Override Categories

Category	Examples	Commit point
Synthesized-MAX, runtime-active	`lookahead_active`, `refresh_defer_active`	Next quiet point
Synthesized-set, runtime-mux	`scheme_or`, `refpb_policy_or`, `page_policy_or`	Next quiet point
Pure runtime	`zqcs_freq_hz`, `init_timeout_ms`, idle thresholds	Live (no quiet point)
Behavioral kill-switch	`force_inorder`, `happy_enable`	Next quiet point

The “pure runtime” category commits immediately on the APB write because the fields don’t gate any in-flight DRAM state — they’re consumed by counter reload-comparators that sample on the next event boundary anyway.

The other three categories require a quiet point because changing them mid-cycle could corrupt scheduler decisions, refresh sequencing, or address-mapping state.

28.2 Quiet Point Definition

Per §2.13:

```
quiet_point = (txn_queue empty OR all entries PENDING)
            AND (refresh_mgr.dbg_state == IDLE)
            AND (init_engine.init_in_progress == 0)
            AND (cmd_encoder pipeline drained)
            AND (all ranks' power state == ACTIVE)
```

The detector is combinational; `csr_apb_fub` samples it on the rising edge of each MC clock.

28.3 Software-Side Sequence

```
// Pseudocode for runtime override commit
void write_override(uint32_t reg_offset, uint32_t value) {
    // 1. Write the staging field (returns immediately)
    apb_write(reg_offset, value);

    // 2. Request immediate commit (otherwise wait for natural quiet
point)
    apb_write(CTRL, CTRL_CONFIG_APPLY);

    // 3. Poll for completion
    while (!(apb_read(STATUS) & STATUS_CONFIG_SETTLED)) {
        // typically settles in < 256 MC cycles
    }

    // 4. Clear config_apply
    apb_write(CTRL, 0);
}
```

The drain phase between step 2 and step 3 is bounded — typically < 256 MC cycles for a healthy queue, longer if there’s an active refresh batch.

28.4 Hardware-Side Sequence

```
cycle T:    APB write to staging field; staging register updates
immediately
cycle T:    cfg_apply asserts via APB (separate write)
cycle T+1:  csr_apb_fub assert quiet_drain_request
```

```
cycle T+1:  scheduler stops accepting new issues; txn_queue drains
cycle T+1:  refresh_mgr completes any in-flight batch, then idles
cycle T+1:  cmd_encoder pipeline naturally drains over a few cycles
cycle T+1+N (N=drain): power_state_fub.quiet_point = 1
cycle T+1+N+1: csr_apb_fub copies staging → live for ALL R/W fields
with pending writes
cycle T+1+N+1: csr_apb_fub releases quiet_drain_request;
STATUS.config_settled = 1
cycle T+1+N+2: scheduler resumes normal operation with the new live
values
```

The “ALL R/W fields with pending writes” behavior means **batch commit**: software can write multiple staging fields, then issue one `config_apply`, and all commit atomically.

28.5 Batch Commit Example

To swap from `PAGE_POLICY = OPEN` to `PAGE_POLICY = HAPPY_HYBRID` and bump lookahead:

```
// Stage all the changes
apb_write(REFRESH_TUNING, REFRESH_TUNING_PAGE_POLICY_OR_HAPPY_HYBRID);
apb_write(SCHED_TUNING, SCHED_TUNING_LOOKAHEAD_ACTIVE(4) |
SCHED_TUNING_HAPPY_ENABLE);

// Single commit
apb_write(CTRL, CTRL_CONFIG_APPLY);
while (!(apb_read(STATUS) & STATUS_CONFIG_SETTLED)) { /* ... */ }
apb_write(CTRL, 0);

// All three fields are now live, with the same quiet-point edge
```

This is the recommended pattern for any multi-field configuration change.

28.6 Quiet-Point Drain Latency

Empirical bounds:

Scenario	Drain latency
Idle controller (no in-flight)	~5 MC cycles
Light traffic (queue < 4 entries)	~20 MC cycles
Heavy traffic (queue near full)	~100–256 MC cycles
Active refresh batch	+ REFRESH_DEFER_MAX × tRFCpb
Active ZQCS (rare)	+ tZQCS

For software that must complete a config swap in bounded time, time the wait against the worst-case bound:

```
uint32_t timeout = WORST_CASE_DRAIN_CYCLES / mc_clk_to_apb_clk_ratio;
while (timeout-- && !(apb_read(STATUS) & STATUS_CONFIG_SETTLED));
if (!timeout) {
    // Timeout — diagnostic; should never happen on a healthy
controller
}
```

28.7 Conflicting Override Detection

If software writes the same field twice before commit, the second write supersedes the first. The staging register holds the latest value; there's no queue of pending changes.

If software issues `config_apply` while a previous commit is still in flight (`STATUS.config_settled = 0`), the new request is honored — the next quiet point will commit all pending changes.

28.8 Open Questions / Future Work

- **Priority commits.** When refresh fires during a commit drain, refresh wins (per the priority hierarchy in §2.7). The commit waits. For software with hard real-time bounds, an "abort current refresh" mechanism could let commit win — but it violates JEDEC. Punt; rely on the worst-case bound.
- **Per-field commit.** Currently all pending stagings commit at once. A "commit-this-field-only" mode could reduce drain pressure. Adds CSR area; minor benefit. Punt.
- **Notification interrupt.** Software currently polls `STATUS.config_settled`. An IRQ option (`irq_config_settled`) would save polling cycles. Worth considering for SoCs with constrained CPU budget.

---

29 Family-Wide Config-Bit Applicability

Per HAS §5.4 for the family-wide config-bit registry, applicability matrix, and capability vector at 0xFF8. This MAS chapter is the **implementation-level** detail for **this controller** specifically.

29.1 Implementation Status in This Controller

The HAS-side registry enumerates 25+ bits that span DDR2/3/4 and LPDDR2/3/4 controllers. This controller implements the subset relevant to DDR2/LPDDR2. Bits flagged for higher generations are present in the CSR map at the documented offsets but **read as 0 and silently ignore writes**.

29.2 Bit-by-Bit Implementation Notes (DDR2 / LPDDR2)

29.2.1 SCHED_TUNING family

Bit	This controller
`lookahead_active`	Live; drives `scheduler_fub.cfg_lookahead_active_i` (§2.7)
`force_inorder`	Live; drives `scheduler_fub.cfg_force_inorder_i`
`happy_enable`	Live when `PAGE_POLICY == HAPPY_HYBRID`; reads as 0 otherwise (the FUB isn't synthesized)
`age_max_runtime`	Live; clipped to ≤ `AGE_MAX` build-time parameter
`txn_queue_high_water`	Live; drives backpressure threshold
`lookahead_max_obs`	RO echo of build-time `LOOKAHEAD_DEPTH_MAX`
`qos_high_priority`	**Sampled but no behavior in v1**; reserved for v2 QoS feature
`bank_group_balance`	**Reads as 0**; DDR2/LPDDR2 has no bank groups

29.2.2 REFRESH_TUNING family

Bit	This controller
`refpb_policy_or`	Live for LPDDR2 builds; reads as 0 in DDR2 builds (no REFpb)
`page_policy_or`	Live; drives `scheduler` page-policy fallback
`refresh_defer_active`	Live; clipped to ≤ `REFRESH_DEFER_MAX` build-time
`zqcs_freq_hz`	Live; drives periodic ZQCS timer
`fgr_mode`	**Reads as 0**; FGR is DDR3+ feature
`refresh_per_rank`	Tied to 1 when `NUM_RANKS > 1`; reads as 0 at NR=1

29.2.3 ADDR_MAP_TUNING family

Bit	This controller
scheme_or	Live; drives addr_mapper_fub.scheme_active_i (§2.3)
synth_mask_obs	RO echo of build-time ADDR_MAP_SCHEMES_SYNTH
bg_field_position	**Reads as 0**; bank groups are DDR4+ feature
xor_seed_runtime	Live when XOR_HASH scheme is synthesized

29.2.4 INIT_TUNING family

Bit	This controller
zq_retries	Live; drives init_engine.cfg_zq_retries_i (§2.12)
init_timeout_ms	Live; drives per-step timeout
wl_retries	**Reads as 0**; write-leveling is DDR3+ feature
mpr_enable	**Reads as 0**; MPR is DDR3+ feature
ca_train_enable	**Reads as 0**; CA training is LPDDR3+ feature
cbt_enable	**Reads as 0**; CBT is LPDDR4-only feature

29.2.5 POWER_TUNING family

Bit	This controller
apd_idle_threshold	Live; drives power_state_fub.cfg_apd_idle_threshold_i
srf_idle_threshold	Live; drives power_state_fub.cfg_srf_idle_threshold_i
dpd_enable	Live for LPDDR2 builds; reads as 0 in DDR2 builds
pasr_active	RO; OR over per-rank PASR masks (LPDDR2 only)
ckedis_after_sr	**Reads as 0**; DDR3+ feature
low_freq_mode	**Reads as 0**; DDR3+ feature

29.2.6 RANK_TUNING family

Bit	This controller
rank_enable_mask	Live; drives per-rank scheduler enable
odt_rule_or	Live; drives odt_ctrl_fub.cfg_odt_rule_or_i (§2.16)
num_ranks_obs	RO echo of build-time NUM_RANKS
cs_assert_cycles	Live; drives per-command CS_n hold cycles in cmd_encoder

29.2.7 ECC_TUNING family (reserved)

Bit	This controller
`ecc_enable`	**Reads as 0**; inline ECC out of scope for v1
`ecc_correct_mode`	**Reads as 0**

29.3 Capability Vector at 0xFF8 (This Build)

The capability vector advertises what this controller supports:

Bits	Field	Value (DDR2 single-rank build)	Value (LPDDR2 4-rank build)
0	`cap_happy`	1 if `PAGE_POLICY == HAPPY_HYBRID`	1 if `PAGE_POLICY == HAPPY_HYBRID`
1	`cap_bg_balance`	0 (DDR2 has no bank groups)	0 (LPDDR2 has no bank groups)
2	`cap_fgr`	0	0
3	`cap_pasr`	0	1
4	`cap_dpd`	0	1
5	`cap_wl`	0	0
6	`cap_cbt`	0	0
7	`cap_xor_hash`	1 if XOR_HASH is in `ADDR_MAP_SCHEMES_SYNTH`	same
11:8	`cap_max_ranks`	1	4
15:12	`cap_n_phases`	4 (default)	4
19:16	`cap_memtype`	0 (DDR2)	4 (LPDDR2)

Software queries this once at boot to know what to write — writing un-capable bits is silently ignored (reads as 0).

29.4 Soft-N/A Behavior Summary

The “soft-N/A” mechanism — writes to un-capable bits silently ignored, no `pslverr` — is deliberate. It lets the same SoC software (e.g., a Linux kernel driver) write the family-wide config word on any flavor and have the un-applicable bits drop without error. This is the same approach taken by family-wide config registers in commercial controllers.

Exceptions (where `pslverr` IS returned, per §4.1):

- Writes to rank N where `N >= NUM_RANKS` (per-rank registers)
- Writes to address-map scheme not in `ADDR_MAP_SCHEMES_SYNTH`
- Writes to `RANK_TUNING.odt_rule_or` with un-synthesized rule

These are caught at decode time because they reference resources that don't exist at all in the build.

29.5 Open Questions / Future Work

- **`cap_qos`.** v2 QoS feature would add a `cap_qos` bit. Reserve bit 23 of `0xFF8` for it.
- **Capability discovery vs. memory-mapped probe.** Some SoCs prefer to probe memory-mapped capabilities; others trust a single capability word. The current single-word approach is the documented pattern; we keep it.
- **Per-family vs. per-component capability.** Currently the capability vector tells software what THIS controller supports. A finer-grained per-component vector would let software check, e.g., "does the scheduler have a specific feature." Punt — current granularity is sufficient.

---

30 Initialization Sequence

Per HAS §6.2 for the architectural sequence and §2.12 (`init_engine_fub`) for the FUB-level detail. This chapter is the **software-side cookbook** — what bring-up firmware actually does to get from power-on to "DRAM ready for AXI traffic."

30.1 Pre-Reset Setup

Before `mc_rst_n` deasserts, the SoC's PMU should:

1. Apply DRAM power rails (VDD, VDDQ; LPDDR2 also VDD1/VDD2/VDDCA)
2. Wait for power rails to be stable (typically 10–100 µs per JEDEC)

3. Deassert `apb_presetn` (APB slave comes out of reset; CSRs are R/W-able but init_engine is still in IDLE)
4. Load expected timing values into CSRs (TIMINGS_*, MR0..MR3, PASR if LPDDR2 multi-rank)
5. Set `POWER_TUNING` thresholds if auto-APD/SRF behavior is wanted post-init
6. Deassert `mc_rst_n` (controller comes out of reset; sits in POST_RESET; init_engine IDLE)

The pre-loading lets the init engine pick up the values when it runs. If software skips pre-loading, the init engine uses the ROM defaults (which may not match the actual silicon).

30.2 Init Trigger

```
void start_dram_init(void) {
    // Load timings (example values for DDR2-800)
    apb_write(TIMINGS_RC_RCD_RP, TIMING_PACK(60, 5, 5, 18));   // tRC,
tRCD, tRP, tRAS
    apb_write(TIMINGS_RAS_RFC_REFI, REFI_PACK(35, 0xC30));      //
tRFC, tREFI
    apb_write(TIMINGS_RRD_FAW_WTR, RRD_PACK(5, 18, 4, 2));      //
tRRD, tFAW, tWTR, tCCD
    apb_write(TIMINGS_CL_CWL_WR, CL_PACK(5, 5, 6, 0));          // CL,
CWL, tWR, (tRFCpb 0=N/A)

    // Load MR0..MR3 from the board's DRAM datasheet
    apb_write(MR0, mr0_value_for_this_part);
    apb_write(MR1, mr1_value_for_this_part);
    apb_write(MR2, mr2_value_for_this_part);
    apb_write(MR3, mr3_value_for_this_part);

    // (LPDDR2 multi-rank) Per-rank PASR if needed
    for (int r = 0; r < num_ranks; r++) {
        apb_write(PASR_BANK_MASK_RANK0 + r*4, pasr_mask_for_rank[r]);
    }

    // Optional: ZQ retry count
    apb_write(INIT_TUNING, INIT_TUNING_PACK(3, 10));  // 3 retries,
10ms timeout

    // Optional: configure refresh tuning
    apb_write(REFRESH_TUNING, REFRESH_DEFER_ACTIVE(8) |
ZQCS_FREQ_HZ(1));

    // Trigger init
    apb_write(CTRL, CTRL_INIT_START);
```

```
    // Poll for completion
    while (1) {
        uint32_t s = apb_read(STATUS);
        if (s & STATUS_INIT_DONE) {
            break;       // DRAM ready
        }
        if (s & STATUS_INIT_ERROR) {
            // Diagnostic; STATUS.init_step_dbg shows where it failed
            uint8_t step = STATUS_INIT_STEP_DBG(s);
            log_error("DRAM init failed at step %d", step);
            return -EIO;
        }
    }
}
```

30.3 Init Sequence Details

The actual step-by-step JEDEC sequence is driven by the init engine's microprogram ROM (§2.12). Software does not interleave commands during init — the init engine has exclusive control until `STATUS.init_done` rises. Major phases:

1. **Power-up settling** — `tINIT0` ≈ 200 µs (DDR2) or `tINIT1` ≈ 100 µs (LPDDR2)
2. **CKE wiggle / RESET_N pulse** — sequence per memtype
3. **MR0..MR3 loads** — Per-rank when `NUM_RANKS > 1`
4. **ZQ calibration** — DDR2 uses OCD; LPDDR2 uses MR10 ZQ Init
5. **Initial refresh batches** — Drives DRAM into a stable refresh schedule
6. **Final settling delay** — A few hundred cycles to ensure all per-rank state is consistent
7. **`init_done` assertion** — Controller transitions out of POST_RESET

For sim runs, set `SIM_INIT_SCALE = 10000` at elaboration to compress the multi-hundred-µs delays into a few cycles. Silicon always uses `SIM_INIT_SCALE = 1`.

30.4 Multi-Rank Init Differences

For `NUM_RANKS > 1`:

- MRS_LOAD steps iterate over ranks; the init engine's `rank_iter` counter (§2.12) advances one rank per MC clock during the step
- ZQ calibration is per-rank (each rank has its own ZQ resistor reference; calibration happens N times serially)
- Each rank's PASR mask is loaded individually

- Init time grows by ~NR × per-rank steps; typically minimal compared to the multi-hundred-µs delays

Software is **rank-blind** during init programming — the same code works at NR=1, 2, or 4. The differences are absorbed by the per-rank YAML repeat in the CSR map (per §4.2) and the rank_iter loop in the init engine.

30.5 ZQ Retry

If ZQ calibration times out (per `INIT_TUNING.zq_retries`), the init engine retries `zq_retries` times before raising `INIT_ERROR`. Default is 3 retries; raise it if signal-integrity issues cause occasional failures.

30.6 Post-Init Power-State

After `init_done`, the controller is in ACTIVE for all ranks with CKE high. The refresh manager starts its tREFI counter and begins normal refresh scheduling. The scheduler is unblocked and any AXI bursts that accumulated during init begin issuing.

30.7 Software Wait Times

Phase	Wait (real silicon, 200 MHz MC clock)
Power-up settling	200 µs
MRS loads + per-rank iteration	~10 cycles × NR × 4 MRs ≈ 160 cycles
ZQ calibration	~1 µs
Initial refresh batches	~10 µs
Total init duration	~250 µs (single-rank), ~280 µs (4-rank)

The init duration scales weakly with rank count — the per-rank serialization is dwarfed by the fixed settling delays.

30.8 Reset Recovery

If init fails (`STATUS.init_error = 1`):

```
void recover_init_failure(void) {
    uint8_t step = STATUS_INIT_STEP_DBG(apb_read(STATUS));
    log_error("Init failed at step %d", step);

    // Inspect step-specific telemetry (per step table)
    // ...
```

```
    // Soft reset and retry
    apb_write(CTRL, CTRL_INIT_FORCE_RESTART);

    // Wait a few cycles for FSM to settle to IDLE
    for (int i = 0; i < 10; i++) asm volatile("nop");

    // Re-init
    start_dram_init();
}
```

A hard fault (e.g., persistent ZQ calibration failure) requires SoC-level intervention — typically a power-cycle and re-bring-up.

30.9 Open Questions / Future Work

- **Init telemetry capture.** When init fails, telemetry beyond `init_step_dbg` would help (DFI signal trace, per-rank fault detection). The current debug surface is sparse; expand if bring-up calls it out.
- **Init via JTAG.** Some platforms init via a JTAG side-band rather than CPU+APB. The CSR map supports this through any APB-attached transport; no controller changes needed.
- **Partial init for warm reset.** Coming out of self-refresh doesn't require full init. The current init engine doesn't have a "warm reset" path. Add when DDR3+ needs it for frequency change.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

31 Refresh and Power-State Programming

Per HAS §3.4 / §3.5 for architecture and §2.11 / §2.13 for FUB detail. This chapter is the **software-side** view of refresh and power configuration.

31.1 Tuning Refresh Behavior

```
void configure_refresh(refresh_config_t* cfg) {
    // Base tREFI (already set during init; can be retuned at runtime
```

```
via TIMINGS_RAS_RFC_REFI)
    apb_write(TIMINGS_RAS_RFC_REFI, REFI_PACK(cfg->trfc, cfg->trefi));

    // Postponer batch depth (1..REFRESH_DEFER_MAX)
    // 1 = no batching; 8 = maximum batching (default)
    uint32_t v = REFRESH_DEFER_ACTIVE(cfg->defer_active);

    // REFpb policy override (LPDDR2 only)
    if (cfg->refpb_policy == DARP)            v |=
REFPB_POLICY_OR_DARP;
    else if (cfg->refpb_policy == OLDEST)     v |=
REFPB_POLICY_OR_OLDEST;
    else if (cfg->refpb_policy == ROUND_ROBIN) v |=
REFPB_POLICY_OR_ROUND_ROBIN;

    // Periodic ZQCS interval (0 = disable, else Hz)
    v |= ZQCS_FREQ_HZ(cfg->zqcs_freq_hz);

    apb_write(REFRESH_TUNING, v);

    // Commit (quiet-point)
    apb_write(CTRL, CTRL_CONFIG_APPLY);
    while (!(apb_read(STATUS) & STATUS_CONFIG_SETTLED));
    apb_write(CTRL, 0);
}
```

31.1.1 When to Tune

Workload	Recommended config
Streaming (DMA, video)	`defer_active` = 8, ZQCS = 1 Hz
Low-latency bursty (CPU access)	`defer_active` = 1 (no batching), ZQCS = 0.1 Hz
Power-sensitive (sleep-heavy)	`defer_active` = 4, ZQCS = 0.1 Hz
Real-time / safety-critical	`defer_active` = 1, deterministic refresh latency

31.2 LPDDR2 PASR (Partial Array Self-Refresh)

PASR masks DRAM regions that hold no data, reducing refresh power.

```
void set_pasr_per_rank(uint8_t rank, uint8_t bank_mask, uint8_t
seg_mask) {
    apb_write(PASR_BANK_MASK_RANK0 + rank*4, bank_mask);
    apb_write(PASR_SEG_MASK_RANK0 + rank*4,  seg_mask);
```

```
    // Wait for next self-refresh entry to propagate to DRAM via
MR16/MR17
    // OR force immediate via quiet-point:
    apb_write(CTRL, CTRL_CONFIG_APPLY);
    while (!(apb_read(STATUS) & STATUS_CONFIG_SETTLED));
    apb_write(CTRL, 0);
}
```

The PASR mask is propagated lazily — typically at the next self-refresh entry — to avoid an extra bus-blocking MR write during normal operation. Forcing immediate propagation via `config_apply` is the right call when the application has decided "we just freed half the DRAM, refresh-power matters NOW."

31.3 Temperature Compensation (LPDDR2)

LPDDR2 devices expose temperature classification in MR4. The SoC reads this via PHY-side mechanisms (out of scope for the controller; the PHY signals temperature change via interrupt or polled register).

Software updates the controller's tREFI scaling:

```
void update_temperature_class(uint8_t rank, uint8_t temp_class) {
    // temp_class: 0 = nominal, 1 = 2x refresh, 2 = 4x refresh
    // Per-rank because ranks can be at different temperatures
    uint32_t v = apb_read(TEMP_DERATE_RANK0 + rank*4);
    v = (v & ~TEMP_CLASS_MASK) | TEMP_CLASS(temp_class);
    apb_write(TEMP_DERATE_RANK0 + rank*4, v);

    // Live — no quiet point needed (just scales the next t_refi_cnt
reload)
}
```

The refresh manager scales tREFI by the per-rank class on the next reload.

31.4 Self-Refresh Entry

For periods of inactivity, software (or auto-detection) can put the DRAM into self-refresh:

```
void enter_self_refresh(void) {
    // Channel-wide request (v1)
    apb_write(CTRL, CTRL_PWR_REQ_SELF_REFRESH);

    // Poll for SR state
    while (1) {
        uint32_t s = apb_read(STATUS);
        uint8_t pstate = STATUS_POWER_STATE(s);
```

```
        if (pstate == POWER_STATE_SELF_REFRESH) break;
    }
}

void exit_self_refresh(void) {
    apb_write(CTRL, CTRL_PWR_REQ_ACTIVE);

    while (STATUS_POWER_STATE(apb_read(STATUS)) !=
POWER_STATE_ACTIVE);
}
```

Note: any AXI traffic to a rank will auto-wake it from SR. Explicit exit is only needed when the application knows it wants to be ready before issuing.

31.5 Auto Power-Down Tuning

```
void configure_auto_power_down(uint32_t apd_threshold, uint32_t
srf_threshold) {
    apb_write(POWER_TUNING,
              POWER_TUNING_APD(apd_threshold) |
              POWER_TUNING_SRF(srf_threshold));

    apb_write(CTRL, CTRL_CONFIG_APPLY);
    while (!(apb_read(STATUS) & STATUS_CONFIG_SETTLED));
    apb_write(CTRL, 0);
}
```

Threshold	Effect
`apd_threshold = 1024`	Default; ~5 µs idleness → APD
`apd_threshold = 0xFFFF`	Effectively disabled
`srf_threshold = 16M`	Default; ~80 ms idleness → SRF
`srf_threshold = 0xFFFFFF`	Effectively disabled

Per-rank idleness counters mean each rank enters APD/SRF independently if traffic concentrates on a subset.

31.6 DPD (LPDDR2 Deep Power Down)

DPD is the deepest power state on LPDDR2 — DRAM is fully off; software must full-init to exit.

```
void enter_dpd(void) {
    // Sanity: LPDDR2 only
    if (!(apb_read(0xFF8) & CAP_DPD)) {
```

```
        log_error("DPD not supported on this build");
        return;
    }

    apb_write(CTRL, CTRL_PWR_REQ_DPD);

    while (STATUS_POWER_STATE(apb_read(STATUS)) != POWER_STATE_DPD);
}

void exit_dpd(void) {
    // Re-init the rank
    start_dram_init();
}
```

DPD is rare in practice — typical use is "DRAM is fully unused for hours" (long-suspend laptop). The full-init exit takes ~250 µs.

31.7 Open Questions / Future Work

- **Per-rank CSR control.** Channel-wide power request is v1. Per-rank request (`CTRL.pwr_req_low_power_rank_R<R>`) for partial low-power without affecting active ranks would be useful for many workloads. Not in v1; see §2.13 open questions.
- **SR lock.** Software-forced SR can be woken by AXI traffic. A "stay in SR until I say go" mode would be useful for guaranteed-low-power testing. Punt to v2.
- **Auto-tune from telemetry.** The refresh-deferral histogram (`OBS_REFRESH_DEFER_HIST_*`) hints at what `defer_active` should be. Could a kernel/firmware loop auto-tune? Likely yes; needs characterization data first.

---

32 Multi-Rank Programming

Per HAS §2.1 (multi-rank as differentiator), HAS §3.3 / §3.4 / §3.6 (architecture). FUB-level detail in MAS §2.9 (bank machine per-rank),

§2.11 (refresh round-robin + DARP), §2.13 (per-rank power state), §2.14 (per-rank CS_n drive), §2.16 (cross-rank ODT).

32.1 Discovery

Software discovers rank count via the capability vector at 0xFF8:

```
uint8_t get_num_ranks(void) {
    return (apb_read(0xFF8) >> 8) & 0xF;
}
```

This is the actual synthesized NUM_RANKS; software should treat it as the authoritative value rather than assuming a default.

32.2 Per-Rank Mode Register Loads

During init, the controller's microprogram automatically iterates the MRS loads across all ranks. Software just writes MR0..MR3 once, and the init engine handles rank iteration:

```
apb_write(MR0, mr0_value);   // applied to all ranks during init
apb_write(MR1, mr1_value);
apb_write(MR2, mr2_value);
apb_write(MR3, mr3_value);
```

For per-rank MR variations (rare — most multi-rank systems use identical DRAM parts), software can intercept the init engine sequence:

```
// Stop after pre-MRS phase
apb_write(CTRL, CTRL_INIT_START);
while (STATUS_INIT_STEP_DBG(apb_read(STATUS)) < INIT_STEP_PRE_MRS);

// Drive MRS to each rank individually via the test access path
// (not currently exposed in v1; this is a hypothetical extension)

// Resume
```

In v1, the simpler model is "all ranks use the same MR values"; per-rank MR override is a v2 feature.

32.3 Per-Rank PASR (LPDDR2)

Already covered in §5.2. Recap:

```
for (int r = 0; r < num_ranks; r++) {
    apb_write(PASR_BANK_MASK_RANK0 + r*4, mask[r]);
}
```

PASR is **per-rank** because each rank has independent MR16/MR17.

32.4 ODT Rule Selection

The controller's ODT behavior is per HAS §3.6 with three modes; runtime selectable via `RANK_TUNING.odt_rule_or`. See §2.16 for the rule details.

```
void configure_odt_for_dimm(uint8_t mode) {
    // mode: 0 = build default, 1 = DDR2, 2 = LPDDR2, 3 = OFF (NR=1
only)
    uint32_t v = apb_read(RANK_TUNING);
    v = (v & ~ODT_RULE_OR_MASK) | ODT_RULE_OR(mode);
    apb_write(RANK_TUNING, v);

    apb_write(CTRL, CTRL_CONFIG_APPLY);
    while (!(apb_read(STATUS) & STATUS_CONFIG_SETTLED));
    apb_write(CTRL, 0);
}
```

For boards with non-standard impedance budgets, override to a specific rule rather than relying on the build-time default.

32.5 Per-Rank Disable

To disable a rank entirely (e.g., for power testing or to skip a faulty rank):

```
void disable_rank(uint8_t rank) {
    uint32_t v = apb_read(RANK_TUNING);
    v &= ~(1 << (RANK_ENABLE_MASK_SHIFT + rank));
    apb_write(RANK_TUNING, v);

    apb_write(CTRL, CTRL_CONFIG_APPLY);
    while (!(apb_read(STATUS) & STATUS_CONFIG_SETTLED));
    apb_write(CTRL, 0);
}
```

The scheduler suppresses all commands to disabled ranks. AXI bursts addressing a disabled rank are returned with `SLVERR`.

32.6 Address Mapping for Multi-Rank

The rank field's position in the AXI flat address depends on the scheme:

Scheme	Rank field position
`ROW_MAJOR`	High bits, above row
`BANK_INTERLEAVE`	High bits, above row

Scheme	Rank field position
XOR_HASH	High bits (identity), but bank field hashed

For interleaved-rank access (where consecutive cache lines round-robin across ranks for max parallelism), this is currently NOT a built-in scheme. v2 may add RANK_INTERLEAVE if characterization shows the benefit.

For now, software-managed rank interleaving (e.g., the OS striping memory allocations across rank-aligned regions) is the workaround.

32.7 Refresh Behavior with Multi-Rank

Per HAS §3.4 v0.2 and §2.11:

- REFab dispatches per-rank in round-robin (not all-rank simultaneously)
- REFpb selects (rank, bank) tuples via DARP across the full product set
- Per-rank PASR masks are honored independently
- Per-rank `last_ref_age` counters drive DARP fairness

This means a 4-rank system has 4× the refresh time concentrated, but distributed across the timeline so any single rank only blocks for tRFC per refresh — non-target ranks keep operating.

32.8 Power State Per-Rank

Per §2.13:

- Each rank has its own FSM (ACTIVE / APD / SRF / DPD)
- Channel-wide CSR `CTRL.pwr_req_*` applies to all ranks in v1
- Per-rank auto-APD / SRF triggers based on per-rank idleness (AXI traffic to rank r resets rank r's idle counter)
- v1: software can't request "low-power on rank 1 only"; v2 adds per-rank request

Workloads that consolidate to rank 0 will see ranks 1+ auto-enter low-power without software intervention.

32.9 Per-Rank Observation

Telemetry registers exist per (rank, bank):

```
uint32_t get_row_hit_rate(uint8_t rank, uint8_t bank) {
    return apb_read(OBS_ROW_HIT_RANK0_BANK0 + (rank * num_banks +
```

```
bank) * 4);
}
```

At NR=1 NB=8, 8 such registers. At NR=4 NB=8, 32. Software queries `num_ranks` and `num_banks` from the capability vector to know how many to iterate.

32.10 Multi-Rank Bring-Up Checklist

Step	Why
Verify `0xFF8.cap_max_ranks` matches expected DIMM rank count	Build / board mismatch
Check `0xFF8.cap_pasr` if using LPDDR2	Need PASR for LPDDR2 bring-up
Set `ODT_RULE_MULTIRANK` per board impedance design	Signal integrity
Verify per-rank ZQ calibration succeeds during init	Each rank's drive impedance
Sweep `OBS_REF_LATENCY_RANK<R>_BANK<N>` across rank workload mix	Per-rank refresh fairness
Stress-test rank-switching (read alternating ranks)	tRTRS / tCS timing
Verify per-rank auto-APD triggers correctly	Power telemetry

32.11 Open Questions / Future Work

- **Per-rank MR override.** v2 feature; some DIMMs have rank-specific tuning (rare).
- **Per-rank CSR power control.** v2 feature; in v1 it's channel-wide.
- **RANK_INTERLEAVE address scheme.** v2; needs characterization to confirm benefit.
- **3DS / LRDIMM logical-rank support.** DDR3+ feature; not in DDR2/LPDDR2 anyway.

33 Error Handling

The controller's error reporting paths and the recommended software response model. Per HAS §2.4 §4 for the IRQ port list and the per-FUB CSR-hook sections in MAS Ch 2.

33.1 Error Categories

Category	Detection	Reporting
Init failure	`init_engine_fub` WAIT_FOR_BIT timeout or ZQ retries exhausted	`STATUS.init_error = 1`, `irq_init_error` pulse
AXI queue overflow	Hard backpressure exceeded (rare)	`STATUS.txn_queue_full_pulses` + `irq_overflow`
Refresh deadline miss	`refresh_owed` exceeds safety margin	`irq_overflow`
DFI rddata-valid timeout	RD issued, no rddata arrived in CL × N + slack	`irq_overflow` + `SLVERR` on the AXI R
AXI burst boundary violation	Per §3.1	`SLVERR` on B/R; no IRQ
ZQ calibration failure (silicon issue)	DFI signal-integrity check via training	Caught at PHY layer, not this controller
Multi-bit DRAM error	Out of scope (no inline ECC in v1)	Would need ECC sideband

33.2 IRQ Topology

Per HAS §2.4 §4:

IRQ	Source	Latch behavior
`irq_init_done`	`init_engine.END` step	One-cycle pulse
`irq_init_error`	`init_engine.INIT_ERROR` state	Latched until CSR-cleared
`irq_overflow`	OR of queue overflow, refresh miss,	Latched

IRQ	Source	Latch behavior
	rddata timeout	

IRQs assert in the `mc_clk` domain. The SoC's interrupt controller is responsible for any re-synchronization to its own clock if different.

33.3 Software Response Patterns

33.3.1 Init Failure

```
void handle_init_error_irq(void) {
    uint32_t s = apb_read(STATUS);
    uint8_t step = STATUS_INIT_STEP_DBG(s);

    log_error("DRAM init failed at step %d", step);

    // Inspect step-specific telemetry
    switch (step) {
        case INIT_STEP_WAIT_DFI_INIT_COMPLETE:
            log_error("PHY init did not complete — check PHY config");
            break;
        case INIT_STEP_ZQ_CAL:
            log_error("ZQ calibration failed — check VDDQ supply,
board impedance");
            break;
        case INIT_STEP_MRS_LOAD:
            log_error("MRS load failed — verify CSR-loaded MR
values");
            break;
        // ... more cases per step table ...
    }

    // Clear the latched error
    apb_write(STATUS, STATUS_INIT_ERROR_CLEAR);   // W1C semantics

    // Attempt re-init
    apb_write(CTRL, CTRL_INIT_FORCE_RESTART);
}
```

33.3.2 AXI Queue Overflow

```
void handle_overflow_irq(void) {
    uint32_t s = apb_read(STATUS);
    uint32_t pulses = apb_read(TXN_QUEUE_FULL_PULSES);

    log_warn("AXI queue overflow: %u pulses", pulses);
```

```
    // Read the high-water threshold and current depth
    uint32_t sched = apb_read(SCHED_TUNING);
    log_warn("High-water threshold: %u, current depth: %u",
             (sched >> 16) & 0xFF, STATUS_TXN_QUEUE_OCC(s));

    // Mitigation: drop the high-water threshold to backpressure
earlier
    sched = (sched & ~0xFF0000) | ((current_depth - 4) << 16);
    apb_write(SCHED_TUNING, sched);
    apb_write(CTRL, CTRL_CONFIG_APPLY);
    while (!(apb_read(STATUS) & STATUS_CONFIG_SETTLED));
}
```

33.3.3 Refresh Deadline Miss

```
void handle_refresh_miss(void) {
    uint32_t owed = apb_read(REFRESH_PENDING_MAX);
    log_error("Refresh deadline missed; owed = %u", owed);

    // Inspect refresh manager state
    uint32_t state = STATUS_REFRESH_MGR_STATE(apb_read(STATUS));
    log_error("Refresh FSM stuck in state %u", state);

    // Mitigation: temporarily disable batching to catch up
    uint32_t v = apb_read(REFRESH_TUNING);
    v = (v & ~REFRESH_DEFER_ACTIVE_MASK) | REFRESH_DEFER_ACTIVE(1);
    apb_write(REFRESH_TUNING, v);
    apb_write(CTRL, CTRL_CONFIG_APPLY);
    while (!(apb_read(STATUS) & STATUS_CONFIG_SETTLED));

    // After catch-up, optionally re-enable batching
}
```

33.4 STATUS_HISTORY for Bring-Up

The STATUS_HISTORY register captures the last 8 power-state transitions (4 bits each). Useful when the controller is oscillating between states (e.g., entering and exiting APD repeatedly):

```
void dump_state_history(void) {
    uint32_t hist = apb_read(STATUS_HISTORY);
    for (int i = 0; i < 8; i++) {
        uint8_t state = (hist >> (i * 4)) & 0xF;
        log_info("State[-%d]: %s", i, power_state_name(state));
    }
}
```

This is more useful than polling current state — it captures fast transitions that polling would miss.

33.5 Diagnostic Sequence for Suspected Bug

```
1. Read STATUS.init_done (must be 1)
2. Read STATUS.power_state (should be ACTIVE)
3. Read STATUS_HISTORY (look for oscillation)
4. Read STATUS.txn_queue_occ + STATUS.refresh_pending_max (check for
backlog)
5. Read OBS_AXI_R_LATENCY_P99 (tail latency telemetry)
6. Read OBS_ROW_HIT_RANK<R>_BANK<N> (per-bank traffic distribution)
7. Read OBS_REF_LATENCY_RANK<R>_BANK<N> (per-bank refresh fairness)
8. Check IRQ status; clear and re-arm if needed
```

This is the bring-up team's first-pass diagnostic flow.

33.6 Soft Reset vs. Re-Init

`CTRL.soft_reset` (bit 31 of `CTRL`) asserts an internal soft reset that re-clears all FUB state without affecting the CSR contents. Use this when the controller is in an inconsistent state but the software's CSR config is correct:

```c
void soft_reset(void) {
    apb_write(CTRL, CTRL_SOFT_RESET);
    // Self-clearing; wait a few cycles
    for (int i = 0; i < 16; i++) asm volatile("nop");

    // Re-trigger init (CSRs are preserved)
    apb_write(CTRL, CTRL_INIT_START);
}
```

For hard cases (silicon bug, persistent failure), the SoC PMU should drive the asynchronous reset and software should re-bring-up from scratch.

33.7 Open Questions / Future Work

- **Per-rank error reporting.** When a fault is localized to one rank (signal integrity, board fault), it would be useful to report which rank. Currently the IRQs are channel-wide. v2 enhancement.
- **Error histogram.** A short rolling history of recent errors (last 16) would help post-mortem analysis. Costs CSR area; punt.
- **Auto-recovery vs. report-only.** The controller could auto-recover from some errors (e.g., refresh deadline miss → drop batching automatically). Currently report-only; software must intervene.

34 Build-Time Configuration Reference

Per HAS §5.2 for the full parameter table and HAS §5.1 for the build-vs-runtime philosophy. This chapter is the **build-script-author's** view: which parameters can be tuned at synthesis time and how each flows through the filelists, includes, and FUB instances.

34.1 Setting Parameters

Build parameters are passed via the top-level module's parameter port at instantiation:

```
ddr2_lpddr2_ctrl #(
    .MEMTYPE              ("LPDDR2"),
    .N_PHASES             (4),
    .NUM_RANKS            (2),
    .NUM_BANKS            (8),
    .AXI_DATA_WIDTH       (128),
    .AXI_ID_WIDTH         (4),
    .AXI_ADDR_WIDTH       (32),
    .ROW_WIDTH            (14),
    .COL_WIDTH            (10),
    .DFI_DATA_WIDTH       (32),
    .DFI_ADDR_WIDTH       (20),       // 20 for LPDDR2; 14 for DDR2
    .SCHEDULER_MODE       ("OOO"),
    .LOOKAHEAD_DEPTH_MAX  (4),
    .PAGE_POLICY          ("HAPPY_HYBRID"),
    .ODT_RULE_MULTIRANK   ("JEDEC_DDR2"),  // forced OFF when
NUM_RANKS == 1
    .REFRESH_DEFER_MAX    (8),
    .REFPB_POLICY         ("DARP")
    // ...
) u_memctrl ( ... );
```

For synthesis-script-driven overrides:

```
# Vivado example
set_property generic {
    NUM_RANKS=4
    PAGE_POLICY=HAPPY_HYBRID
```

```
    PAGE_PREDICTOR_TABLE_BITS=12
} [get_filesets sources_1]
```

34.2 Filelist Composition

The build's filelist depends on parameter values. Conditional FUBs are included via top-level generate; the filelists include them unconditionally but the synthesis tool elides unused logic. Recommendations:

Parameter combination	Filelist effect
`MEMTYPE = "DDR2"`	LPDDR2 encoder code dead-code-eliminated
`MEMTYPE = "LPDDR2"`	DDR2 encoder code dead-code-eliminated
`PAGE_POLICY = "HAPPY_HYBRID"`	`page_predictor_fub.sv` instantiated
`PAGE_POLICY != "HAPPY_HYBRID"`	`page_predictor_fub` not instantiated; tied output
`SCHEDULER_MODE = "INORDER"`	OoO comparator network not synthesized
`NUM_RANKS = 1`	`odt_ctrl_fub` collapses to tie-off; cross-rank logic dead

For aggressive area control, hand-curate the filelist to exclude unused FUBs entirely (rather than relying on tool's dead-code-elimination).

34.3 rtl/includes/ddr2_lpddr2_config.svh

This header is generated from the build parameters and contains derived constants used across FUBs:

```
`define DDR2_LPDDR2_CONFIG_SVH

`define MEMTYPE_DDR2   "DDR2"
`define MEMTYPE_LPDDR2 "LPDDR2"

// Build-time
`define BUILD_NUM_RANKS  4
`define BUILD_NUM_BANKS  8

// Derived
`define RANK_WIDTH       2     // $clog2(NUM_RANKS)
`define BANK_WIDTH       3     // $clog2(NUM_BANKS)

// Sanity
`if (`BUILD_NUM_RANKS == 1) && (`ODT_RULE_MULTIRANK != "OFF")
```

```
    `error "ODT_RULE_MULTIRANK must be OFF when NUM_RANKS = 1"
`endif
```

The header is regenerated whenever the build parameters change.

34.4 Parameter Sanity Assertions

The top-level generate block contains elaboration-time assertions:

```
generate
    if (NUM_RANKS == 1 && ODT_RULE_MULTIRANK != "OFF")
        $error("ODT_RULE_MULTIRANK must be OFF when NUM_RANKS = 1");

    if (MEMTYPE == "LPDDR2" && DFI_ADDR_WIDTH < 20)
        $error("LPDDR2 requires DFI_ADDR_WIDTH >= 20 (CA-bus
packing)");

    if (MEMTYPE == "DDR2" && DFI_ADDR_WIDTH < ROW_WIDTH)
        $error("DDR2 requires DFI_ADDR_WIDTH >= ROW_WIDTH");

    if (LOOKAHEAD_DEPTH_MAX < 0 || LOOKAHEAD_DEPTH_MAX > 4)
        $error("LOOKAHEAD_DEPTH_MAX must be 0..4");

    if (REFRESH_DEFER_MAX < 1 || REFRESH_DEFER_MAX > 8)
        $error("REFRESH_DEFER_MAX must be 1..8 (JEDEC ceiling)");

    if (NUM_RANKS != 1 && NUM_RANKS != 2 && NUM_RANKS != 4)
        $error("NUM_RANKS must be 1, 2, or 4");

    // ... more sanity checks ...
endgenerate
```

These catch invalid configs at synthesis time rather than letting them silently produce broken silicon.

34.5 Recommended Build Profiles

34.5.1 Profile A — Tutorial / Smallest Area

```
MEMTYPE              = "DDR2"
N_PHASES             = 2
NUM_RANKS            = 1
PAGE_POLICY          = "OPEN"           // no HAPPY predictor
SCHEDULER_MODE       = "INORDER"        // no OoO logic
LOOKAHEAD_DEPTH_MAX  = 0
TXN_QUEUE_DEPTH      = 4
```

```
ODT_RULE_MULTIRANK   = "OFF"
ADDR_MAP_SCHEMES_SYNTH = {ROW_MAJOR}     // one scheme only
```

Estimated area: ~7,000 LUTs on XC7A100T. Suitable for the smallest embedded SoCs.

34.5.2 Profile B — Typical Embedded SoC

```
MEMTYPE              = "DDR2"            // or "LPDDR2" depending on
board
N_PHASES             = 4
NUM_RANKS            = 1
PAGE_POLICY          = "HAPPY_HYBRID"
PAGE_PREDICTOR_TABLE_BITS = 12
SCHEDULER_MODE       = "OOO"
LOOKAHEAD_DEPTH_MAX  = 4
TXN_QUEUE_DEPTH      = 16
REFRESH_DEFER_MAX    = 8
ADDR_MAP_SCHEMES_SYNTH = {ROW_MAJOR, BANK_INTERLEAVE, XOR_HASH}
```

Estimated area: ~12,000 LUTs. The Nexys A7 bring-up profile.

34.5.3 Profile C — Multi-Rank DIMM

```
MEMTYPE              = "DDR2"
N_PHASES             = 4
NUM_RANKS            = 4
NUM_BANKS            = 8
PAGE_POLICY          = "HAPPY_HYBRID"
ODT_RULE_MULTIRANK   = "JEDEC_DDR2"
SCHEDULER_MODE       = "OOO"
LOOKAHEAD_DEPTH_MAX  = 4
TXN_QUEUE_DEPTH      = 32
REFRESH_DEFER_MAX    = 8
REFPB_POLICY         = "DARP"           // not used in DDR2; reserved
if MEMTYPE switches
```

Estimated area: ~22,000 LUTs (bank machines dominate due to 32 instances). For multi-rank DIMM characterization platforms.

34.5.4 Profile D — LPDDR2 Single-Rank Mobile

```
MEMTYPE              = "LPDDR2"
N_PHASES             = 4
NUM_RANKS            = 1
NUM_BANKS            = 8
PAGE_POLICY          = "HAPPY_HYBRID"
SCHEDULER_MODE       = "OOO"
REFPB_POLICY         = "DARP"
PAGE_PREDICTOR_TABLE_BITS = 12
POWER_TUNING_DEFAULT  = LPDDR2 with DPD enabled
```

Estimated area: ~13,000 LUTs. Mobile-class designs.

34.6 Synthesis Tool Notes

34.6.1 Vivado / 7-Series

- `(* ram_style = "block" *)` on the `page_predictor` table → BRAM18
- `(* keep = "TRUE" *)` on `dbg_*` outputs to preserve telemetry through optimization
- `synth_design -directive AreaOptimized_high` for size-constrained builds

34.6.2 Open-Source Toolchain (Yosys + nextpnr)

- Generic synthesis works for the controller; `page_predictor` table falls into BRAM via Yosys's auto-detect
- For nextpnr-xilinx, the `--no-promote-globals` flag is helpful for the bank-state aggregation buses

34.7 Open Questions / Future Work

- **Hand-curated filelist for ultra-small builds.** When `SCHEDULER_MODE = "INORDER"`, the synthesizer can elide most of the OoO logic; but a curated filelist could exclude the entire FR-FCFS comparator file from the build. Minor area win.
- **Parameter validation script.** A standalone script that takes the parameters and reports the expected utilization could help bring-up. Punt.
- **Configurable assertions for sim vs synth.** Some assertions help in sim but bloat the synthesized netlist. A `BUILD_FOR_SIM` parameter could control. Not in v1.

35 Runtime Configuration Reference

Per HAS §6.3 for the full CSR map. Per §4.3 for the staging-vs-live commit protocol. This chapter is the **driver author's** cookbook for runtime tunables: what to tune, when, and what to watch for.

35.1 Bring-Up Configuration Order

When bring-up software first comes up after init, recommended order:

1. **Verify capability vector** — `apb_read(0xFF8)`. Confirm `cap_max_ranks`, `cap_n_phases`, `cap_memtype` match expectations.
2. **Set address-map scheme** — `ADDR_MAP_TUNING.scheme_or` to match the SoC's physical-address layout.
3. **Set page policy** — `REFRESH_TUNING.page_policy_or` if not using build default.
4. **Set lookahead depth** — `SCHED_TUNING.lookahead_active`. Start at MAX, tune down if area-sensitive.
5. **Set refresh deferral** — `REFRESH_TUNING.refresh_defer_active`. Start at MAX; tune for workload.
6. **Set ZQCS frequency** — `REFRESH_TUNING.zqcs_freq_hz`. 1 Hz default; 0 to disable.
7. **Set ODT rule** (multi-rank only) — `RANK_TUNING.odt_rule_or` per board design.
8. **Set power thresholds** — `POWER_TUNING.apd_idle_threshold`, `POWER_TUNING.srf_idle_threshold`.

Issue `CTRL.config_apply = 1` once at the end to commit all in a single quiet-point drain (per §4.3 batch commit).

35.2 Characterization Sweep Order

For tuning on a specific workload:

Sweep order	Knob	Why first
1	`ADDR_MAP_TUNING.scheme_or`	Largest impact on row-hit rate
2	`ADDR_MAP_TUNING.xor_seed_runtime`	For XOR_HASH, hunt for

Sweep order	Knob	Why first
		pathological collisions
3	SCHED_TUNING.lookahead_active	Direct trade between issue rate and lookahead area
4	REFRESH_TUNING.page_policy_or	OPEN vs CLOSE vs HAPPY for the workload mix
5	SCHED_TUNING.happy_enable	A/B test the predictor (HAPPY only)
6	REFRESH_TUNING.refresh_defer_active	Trade refresh latency vs sustained BW
7	SCHED_TUNING.age_max_runtime	Anti-starvation tuning
8	SCHED_TUNING.txn_queue_high_water	Backpressure timing

Re-run at each step; observe telemetry (next section) to decide which way to tune.

35.3 Telemetry to Watch

Telemetry register	What it tells you	Tune action
OBS_AXI_R_LATENCY_AVG	Average AXI read latency	Tune scheduler / lookahead / page policy
OBS_AXI_R_LATENCY_P99	Tail read latency	Tune anti-starvation, age_max
OBS_AXI_W_LATENCY_AVG	Average AXI write latency	Tune CWL alignment / wr_data_path
OBS_ROW_HIT_RANK<R>_BANK<N>	Per-bank row-hit rate	Tune address mapping, page policy
OBS_REF_LATENCY_RANK<R>_BANK<N>	Per-bank refresh blocking	Tune refresh deferral / REFpb policy
OBS_TXN_QUEUE_DEPTH_MAX/AVG	Queue pressure	Tune high water mark
OBS_REFRESH_PENDING_MAX	How close to refresh-deadline-miss	Lower refresh_defer_active
OBS_REFRESH_DEFER_HIST_0..3	Refresh batch histogram	Validate refresh_defer_active choice

Telemetry register	What it tells you	Tune action
OBS_PAGE_PRED_ACCURACY	HAPPY accuracy (when enabled)	Tune warmup / hysteresis
OBS_INORDER_FALLBACKS	How often OoO would have helped	Decide if force_inorder costs too much
OBS_LOOKAHEAD_HITS	Conclusive-lookahead rate	Decide if increasing lookahead_active helps
OBS_XRANK_SWITCHES	Cross-rank traffic frequency (NR>1)	Address mapping might cluster traffic better
OBS_TFAW_HITS_RANK<R>	Per-rank tFAW pressure	Workload-dependent — high values flag thermal limits
OBS_TWTR_HITS	Write-to-read turnaround pressure	Workload mix issue
OBS_ODT_PULSE_PCT_R<R> (NR>1)	Fraction of cycles per-rank ODT is on	Power telemetry

35.4 Workload-Specific Recipes

35.4.1 Streaming (DMA, video, audio capture)

```
// Maximize row-hit, batch refresh
apb_write(SCHED_TUNING, LOOKAHEAD_ACTIVE(4) | HAPPY_ENABLE);
apb_write(REFRESH_TUNING, REFRESH_DEFER_ACTIVE(8) |
PAGE_POLICY_OR_OPEN | ZQCS_FREQ_HZ(1));
apb_write(ADDR_MAP_TUNING, SCHEME_OR(BANK_INTERLEAVE));
apb_write(CTRL, CTRL_CONFIG_APPLY);
```

35.4.2 Low-Latency Bursty (CPU)

```
// Prioritize first-beat latency, minimal refresh deferral
apb_write(SCHED_TUNING, LOOKAHEAD_ACTIVE(2) | HAPPY_ENABLE);
apb_write(REFRESH_TUNING, REFRESH_DEFER_ACTIVE(1) |
PAGE_POLICY_OR_HAPPY | ZQCS_FREQ_HZ(0.1));
apb_write(ADDR_MAP_TUNING, SCHEME_OR(XOR_HASH));
apb_write(CTRL, CTRL_CONFIG_APPLY);
```

35.4.3 Real-Time / Safety-Critical

```
// Deterministic; predictable refresh, no OoO
apb_write(SCHED_TUNING, FORCE_INORDER | LOOKAHEAD_ACTIVE(0));
apb_write(REFRESH_TUNING, REFRESH_DEFER_ACTIVE(1) |
```

```
PAGE_POLICY_OR_CLOSE);
apb_write(CTRL, CTRL_CONFIG_APPLY);
```

35.4.4 Power-Sensitive (Sleep-Heavy)

```
apb_write(SCHED_TUNING, LOOKAHEAD_ACTIVE(4) | HAPPY_ENABLE);
apb_write(REFRESH_TUNING, REFRESH_DEFER_ACTIVE(4));
apb_write(POWER_TUNING, APD(512) | SRF(8000000));   // aggressive
auto-low-power
apb_write(CTRL, CTRL_CONFIG_APPLY);
```

35.4.5 Multi-Rank DIMM Workload Distribution

```
// LPDDR2 with 4 ranks, DARP refresh, JEDEC_DDR2 ODT
apb_write(REFRESH_TUNING, REFPB_POLICY_OR(DARP) |
REFRESH_DEFER_ACTIVE(8));
apb_write(RANK_TUNING, ODT_RULE_OR(JEDEC_DDR2));
apb_write(POWER_TUNING, APD(2048) | SRF(16000000));   // larger
thresholds for multi-rank
apb_write(CTRL, CTRL_CONFIG_APPLY);
```

35.5 Telemetry-Driven Auto-Tuning Loop

For SoCs with firmware capable of background loops:

```
void periodic_autotune(void) {
    static uint8_t defer = 8;
    uint32_t pending_max = apb_read(REFRESH_PENDING_MAX);

    if (pending_max > REFRESH_DEFER_MAX * 7 / 8) {
        // Approaching deadline miss; back off batching
        if (defer > 1) defer--;
    }
    else if (pending_max < REFRESH_DEFER_MAX / 4) {
        // Plenty of headroom; allow more batching
        if (defer < REFRESH_DEFER_MAX) defer++;
    }

    uint32_t v = apb_read(REFRESH_TUNING);
    v = (v & ~REFRESH_DEFER_ACTIVE_MASK) |
REFRESH_DEFER_ACTIVE(defer);
    apb_write(REFRESH_TUNING, v);
    apb_write(CTRL, CTRL_CONFIG_APPLY);
    while (!(apb_read(STATUS) & STATUS_CONFIG_SETTLED));
    apb_write(CTRL, 0);
}
```

Run this every 100 ms or so. The CSR commits are cheap (small quiet-point drain) and the workload-dependent tuning is automatic.

35.6 Open Questions / Future Work

- **Recommended-default switching by workload identifier.** Some SoCs identify the workload (graphics vs CPU vs idle). Could the controller accept a “profile select” CSR that switches all knobs at once? Adds CSR area; useful for firmware-controlled systems. Punt.
- **Telemetry-based ML tuning.** Long-horizon ML could optimize the entire knob set per workload. Out of scope for the controller; would live in firmware.
- **Per-application priority via QoS.** v2 feature using `awqos`/`arqos`. The hooks are in the scheduler interface; just not wired in v1.